

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Core Python File Report

Hierarchical Reinforcement Learning for Opportunistic Aircraft Maintenance

A line-by-line guide to the maintenance tutorial

Soroush Asadian
Student Number: 270334

Supervisor: Emanuele Petriconi

28 October 2026

Contents

1 Purpose, source version, and reading instructions **7**

2 What changed since the earlier code report **7**

3 Cleanup applied on 28 October 2026 **8**

3.1 Source size and retained methods 8

3.2 Shared observation validation 8

3.3 Direct critic evaluation 9

3.4 Action history and input validity 9

3.5 Verification scope and documentation 9

4 The environment, policy, trainer, and evaluator **9**

4.1 Four responsibilities 9

4.2 The notebook's library trainer versus the custom fleet trainers 10

4.3 One day is the physical time unit 10

5 State, observations, identities, and maintenance actions **10**

5.1 Hidden plant and observed belief 10

5.2 Bitwise selection and aircraft plates 11

5.3 Full eight-field action and duration 12

5.4 Reservations, costs, and observation updates 12

6 Single-agent policies, shared parameters, and multi-agent information **12**

6.1 Four independent design questions 12

6.2 Why one shared actor is not the original joint fleet controller 13

6.3 Why single-agent flat PPO is not DCMAC despite similar network counts . . 13

6.4 Why DDMAC has N networks and still needs global observations 13

6.5 Critic sharing is a separate choice 14

7 Every implemented method and its concrete differences **14**

7.1 flat 15

7.2 conditional 15

7.3 shared 15

7.4 options 15

7.5 learned_options 15

7.6 mappo_aircraft 15

7.7 mappo_component 15

7.8 dcmac_aircraft 16

7.9 ddmac_aircraft 16

7.10 dcmac_component 16

7.11 ddmac_component . . . 16
7.12 concurrent_options . . . 17
7.13 planning_hierarchy . . . 17
7.14 aggregate_targets . . . 17
7.15 maxq . . . 17
7.16 option_critic . . . 17
7.17 hierarchical_vdn . . . 17
7.18 hrl_flat . . . 17

8 Training equations and what their axes mean **18**
8.1 Fleet-joint PPO and per-agent MAPPO . . . 18
8.2 Replay-based DCMAC and DDMAC . . . 18
8.3 Aircraft option durations and completion values . . . 18
8.4 Option-critic termination and two-level value-decomposition networks . . . 19

9 Comparison design, verification, and remaining work **19**
9.1 Which comparisons isolate which changes . . . 19
9.2 Recorded software checks . . . 19
9.3 Important physical and methodological limits . . . 20

10 Source walkthrough conventions **20**
10.1 Module header and imports . . . 21
10.2 MiniMaintenanceEnv . . . 22
10.3 MiniMaintenanceEnv.___init___ . . . 23
10.4 MiniMaintenanceEnv.reset . . . 24
10.5 MiniMaintenanceEnv.step . . . 25
10.6 MiniMaintenanceEnv._observation . . . 26
10.7 MiniMaintenanceEnv.render_text . . . 27
10.8 random_policy . . . 27
10.9 threshold_policy . . . 28
10.10rollout . . . 28
10.11Module declarations and integration . . . 29
10.12valid_action . . . 30
10.13all_actions . . . 31
10.14ConditionalMaintenanceEnv . . . 31
10.15ConditionalMaintenanceEnv.___init___ . . . 32
10.16ConditionalMaintenanceEnv._observation . . . 32
10.17ConditionalMaintenanceEnv.reset . . . 33
10.18ConditionalMaintenanceEnv.step . . . 33
10.19Module declarations and integration . . . 35
10.20GymMaintenanceEnv . . . 35

10.21GymMaintenanceEnv.___init___ . . . 36
10.22GymMaintenanceEnv._encode . . . 36
10.23GymMaintenanceEnv.reset . . . 37
10.24GymMaintenanceEnv.step . . . 37
10.25GymMaintenanceEnv.render . . . 38
10.26GymMaintenanceEnv.close . . . 38
10.27Module declarations and integration . . . 38
10.28Config . . . 39
10.29Config.___post_init___ . . . 41
10.30Plan . . . 42
10.31Plan.crews . . . 43
10.32Job . . . 43
10.33FleetMaintenanceEnv . . . 44
10.34FleetMaintenanceEnv.___init___ . . . 46
10.35FleetMaintenanceEnv.reset . . . 47
10.36FleetMaintenanceEnv.split_observation . . . 49
10.37FleetMaintenanceEnv.idle_action . . . 50
10.38FleetMaintenanceEnv.available_crew_mask . . . 50
10.39FleetMaintenanceEnv.resource_state . . . 50
10.40FleetMaintenanceEnv.resource_details . . . 51
10.41FleetMaintenanceEnv.sensor_noise . . . 51
10.42FleetMaintenanceEnv._sensor_model . . . 52
10.43FleetMaintenanceEnv._clip_measurements . . . 53
10.44FleetMaintenanceEnv._invert_sensor_metrics . . . 53
10.45FleetMaintenanceEnv.observation_metrics . . . 54
10.46FleetMaintenanceEnv._observation . . . 54
10.47FleetMaintenanceEnv.validate_row . . . 56
10.48FleetMaintenanceEnv._start_job . . . 56
10.49FleetMaintenanceEnv._update_filter . . . 58
10.50FleetMaintenanceEnv.step . . . 60
10.51FleetMaintenanceEnv.render . . . 66
10.52threshold_action . . . 66
10.53Module declarations and integration . . . 68
10.54FlattenFleetAction . . . 69
10.55FlattenFleetAction.___init___ . . . 69
10.56FlattenFleetAction.action . . . 70
10.57FlattenFleetAction.reverse_action . . . 70
10.58Module declarations and integration . . . 70
10.59_fleet_observation_tensor . . . 71
10.60FleetActorCritic . . . 71

10.61FleetActorCritic.___init___ 72
10.62FleetActorCritic.value 74
10.63FleetActorCritic.act 74
10.64OptionActorCritic 78
10.65OptionActorCritic.___init___ 78
10.66OptionActorCritic.value 79
10.67OptionActorCritic.act 79
10.68LearnedOptionActorCritic 80
10.69LearnedOptionActorCritic.___init___ 80
10.70LearnedOptionActorCritic.value 81
10.71LearnedOptionActorCritic.act 81
10.72Module declarations and integration 82
10.73mappo_agent_inputs 82
10.74MultiAgentFleetEnv 84
10.75MultiAgentFleetEnv.___init___ 85
10.76MultiAgentFleetEnv.observations 86
10.77MultiAgentFleetEnv.reset 87
10.78MultiAgentFleetEnv.compose_action 87
10.79MultiAgentFleetEnv.step 91
10.80MultiAgentActorCritic 92
10.81MultiAgentActorCritic.___init___ 92
10.82MultiAgentActorCritic.agent_inputs 94
10.83MultiAgentActorCritic.policy 94
10.84MultiAgentActorCritic.active_agents 96
10.85MultiAgentActorCritic.value 97
10.86MultiAgentActorCritic.act 97
10.87MultiAgentActorCritic.action_dict 97
10.88Module declarations and integration 98
10.89request_assignment 98
10.90DCMACActorCritic 99
10.91DCMACActorCritic.___init___ 100
10.92DCMACActorCritic.agent_inputs 101
10.93DCMACActorCritic._busy_fields 102
10.94DCMACActorCritic.active_agents 103
10.95DCMACActorCritic.policy 103
10.96DCMACActorCritic.value 104
10.97DCMACActorCritic.act 105
10.98dcmac_losses 105
10.99train_dcmac 107
10.100mappo_gae 113

10.101 mappo_policy_loss . . . 114
10.102 train_mappo . . . 115
10.103 Module declarations and integration . . . 120
10.104 hrl_mlp . . . 120
10.105 hrl_catalog_mask . . . 121
10.106 hrl_categorical . . . 122
10.107 AircraftOptionEnv . . . 124
10.108 AircraftOptionEnv.__init__ . . . 125
10.109 AircraftOptionEnv.reset . . . 125
10.110 AircraftOptionEnv.decision_mask . . . 126
10.111 AircraftOptionEnv.step . . . 126
10.112 AircraftPlanningHierarchy . . . 129
10.113 AircraftPlanningHierarchy.__init__ . . . 130
10.114 AircraftPlanningHierarchy.features . . . 131
10.115 AircraftPlanningHierarchy.value . . . 132
10.116 AircraftPlanningHierarchy.actor_parameters . . . 132
10.117 AircraftPlanningHierarchy.act . . . 133
10.118 AircraftPlanningHierarchy.catalog_actions . . . 137
10.119 hrl_option_target . . . 138
10.120 train_aircraft_hrl . . . 139
10.121 MAXQMaintenance . . . 144
10.122 MAXQMaintenance.__init__ . . . 145
10.123 MAXQMaintenance.state . . . 146
10.124 MAXQMaintenance.task_children . . . 146
10.125 MAXQMaintenance.value . . . 147
10.126 MAXQMaintenance.q . . . 148
10.127 MAXQMaintenance.update_completion . . . 148
10.128 MAXQMaintenance.execute . . . 149
10.129 MAXQMaintenance.get_extra_state . . . 152
10.130 MAXQMaintenance.set_extra_state . . . 152
10.131 train_maxq . . . 152
10.132 FleetOptionCritic . . . 154
10.133 FleetOptionCritic.__init__ . . . 154
10.134 FleetOptionCritic.features . . . 155
10.135 FleetOptionCritic.q_values . . . 156
10.136 FleetOptionCritic.beta . . . 156
10.137 FleetOptionCritic.policy . . . 157
10.138 option_critic_target . . . 158
10.139 hrl_epsilon_choice . . . 158
10.140 train_option_critic . . . 159

10.141 HierarchicalVDN . . . 164
10.142 HierarchicalVDN.___init___ . . . 164
10.143 HierarchicalVDN.features . . . 165
10.144 HierarchicalVDN.high_values . . . 165
10.145 HierarchicalVDN.low_values . . . 166
10.146 HierarchicalVDN.mix . . . 166
10.147 HierarchicalVDN.choose_goals . . . 167
10.148 HierarchicalVDN.choose_actions . . . 168
10.149 train_hierarchical_vdn . . . 169
10.150 evaluate_hrl . . . 173
10.151 Module declarations and integration . . . 177
10.152 execute_option . . . 177
10.153 Module declarations and integration . . . 180
10.154 make_model . . . 180
10.155 load_checkpoint . . . 181
10.156 mdp_gae . . . 182
10.157 train . . . 183
10.158 evaluate . . . 188
10.159 main . . . 191
10.160 Module declarations and integration . . . 194

11 Closing interpretation **194**

1. Purpose, source version, and reading instructions

This revised second report documents the core Python implementation as of 28 October 2026 of the aircraft-maintenance tutorial, including the environment and all implemented learning methods. It supersedes the implementation descriptions in the report dated 22 October 2026. That earlier report correctly described a single fleet controller at its own source version; its statement that multi-agent learning was not implemented no longer describes the current code.

We retain the method comparisons and the complete source walkthrough from the second report dated 26 October 2026. The revision follows the subsequent code cleanup. The simulator remains a research tutorial with synthetic assumptions, and the software checks do not establish operational validity.

Code scope. As in the previous report, the detailed walkthrough covers `maintenance_tutorial_annotated_core.py`. The notebook assembles this file with the separate exercises/solutions and behavioral-test files into one in-memory module. Their integration and verification are explained, but the separate files and every notebook cell are not additional line-by-line appendices. All code needed to read the core walkthrough is embedded in this TeX file. The institutional artwork is also embedded and is extracted automatically when this document is compiled.

Line numbers. Each listing displays the corresponding line numbers in the current annotated Python file, with gaps only where blank source lines are omitted. Every nonblank retained code, import, decorator, declaration, documentation, and ordinary comment line has a numbered explanation. Blank lines only separate source visually. Continuation lines are explained individually. The generated explanation-only comments were removed from the Python file during cleanup. Their removal changes the amount of inline documentation; the explanations in this report remain available separately.

How to read. Begin with the observation, action, time, and information conventions. Then read the method comparisons before the long source walkthrough. A network count alone does not determine whether a controller is single-agent, multi-agent, or hierarchical. The decisive questions are which information it uses, which decisions it owns, how its probability or value is represented, and which update is applied.

Frozen source. `maintenance_tutorial_annotated_core.py`; 2331 physical lines; 2096 nonblank retained lines explained. SHA-256:

`19414e8b5b720969b5323defec5419c57d39a6acc39cdf135ced315d8bf4d66c`

The source snapshot is frozen for this report; later edits require regeneration of the line mapping.

2. What changed since the earlier code report

1. Aircraft have stable fleet-local plate labels, such as `AC-001`, linking their array indices, actions, maintenance jobs, and event diagnostics. These labels do not by themselves create a persistent flight-history database or a fleet-status bitmask.

2. Crew members and workshops have identities and individual duration factors. Crew selection is an identity bitmask. Selected crew work rates add; a workshop factor changes component work duration. A job stores cumulative component-completion days so accepted scheduling assumptions are used during actual execution.

3. Observation version 3 retains 70 values per aircraft and extends public information with

individual resource availability and factor encodings. Fleet and resource counts are configurable at model construction.

4. Multi-agent proximal policy optimization (MAPPO) supports aircraft ownership with shared actor weights and component-type ownership with six independent actors. Its centralized training critic produces one value per agent; actors use restricted observed information.

5. Deep centralized multi-agent actor–critic (DCMAC) and deep decentralized multi-agent actor–critic (DDMAC) have separate off-policy fleet adaptations under aircraft and component-type ownership. They receive global observed beliefs, use replay and joint importance-weighted temporal-difference (TD) updates, and have one scalar fleet critic.

6. A daily aircraft-option wrapper permits new plans for free aircraft while other aircraft remain in committed repairs. Additional methods implement separate learned planning stages, replacement-budget targets, a single-aircraft MAXQ-0 baseline, safe planning-mode option-critic, and a two-level value-decomposition adaptation. A restricted-catalog flat comparator accompanies them.

7. The common model factory, training dispatcher, evaluation, checkpoints, runner, tests, and comprehensive notebook recognize the additions. The notebook has ten learning parts and 122 executed code cells; the behavioral suite records 45 passing tests after the cleanup.

Not implemented by these changes. Explicit flight legs, flight-hour duration, time/cycle-dependent wear, thermal/dust/salt condition inputs, standardized common-reference post-landing measurements, and the richer bird/hail encounter model remain pending. The current weather multiplier and incident constants must not be described as the completed literature-based upgrade.

3. Cleanup applied on 28 October 2026

3.1. Source size and retained methods

The core decreased from 4,273 to 2,331 physical lines, a reduction of 1,942 lines (45.4%). We removed 1,936 generated `# Explanation:` comments and nine duplicate imports. The reduction therefore comes mainly from inline documentation. No existing function or method was removed. All 18 learned methods, network constructors, registered layers, reward formulas, physical transitions, observation/action layouts, training losses and trainer functions were retained.

Eight existing function or method bodies changed, and one private helper was added. The changes below concern repeated input handling and unnecessary intermediate work. Registered layers that are unused in particular modes remain present so existing checkpoint keys and parameter initialization order remain compatible.

3.2. Shared observation validation

`_fleet_observation_tensor` converts observations to float32 tensors, adds a batch dimension to a single observation, and checks the expected two-dimensional width. MAPPO input preparation, DCMAC input preparation and the two hierarchy feature adapters reuse this helper. Their shape requirements and error messages remain the same. The helper validates the global observation without constructing local actor features.

3.3. Direct critic evaluation

`AircraftPlanningHierarchy.value` and `FleetOptionCritic.q_values` now pass the validated observation directly to their existing critic networks. Previously, these calls constructed actor context through `features()` and discarded that context. The option-value path retains its original output reshape. Neither change removes a critic or changes its layers.

An isolated forward-call benchmark recorded speedups of 1.87 to 2.03 times at seven aircraft and 7.88 to 8.17 times at 50 aircraft. It used batches of 32, one Torch CPU thread, disabled gradients, alternating old/new measurement order and five timing blocks. These measurements concern the two critic calls only. The calculated avoided actor-context allocation was 519,680 bytes at seven aircraft and 22,976,000 bytes at 50 aircraft. These values are not measured peak-memory reductions, and no GPU, backward-pass or full-training speedup was established.

The direct calls also change an extension point: an external subclass or monkeypatch that intercepts `features()` will no longer intercept these critic methods. The built-in implementations do not rely on that interception.

3.4. Action history and input validity

`FleetActorCritic.act` skips the previous-choice buffer in flat mode because that mode does not use it to condition logits. Other modes still update the history needed by later fields, but skip the final update after the last field. Feasibility masks, action probabilities, action reconstruction and sampling are retained. The history update remains out of place because earlier conditioning layers may retain their inputs for backpropagation.

`FleetMaintenanceEnv.reset` now explicitly requires finite custom initial health, in addition to the existing shape and range checks. NaN values previously passed the range comparisons and now raise `ValueError`. Positive and negative infinity already failed the previous range check. Valid finite arrays in $[0, 1]$ retain their previous initialization behavior.

3.5. Verification scope and documentation

The recorded checks support preserved built-in behavior for the valid inputs they exercise. They do not prove equivalence for every configuration. The saved verification JSON does not include source hashes or environment versions, so it is a record of the earlier verification rather than a fully self-contained reproducibility package. This report embeds the current code and records its source hash for navigation and version identification.

The teaching explanations remain in the reports. Current listings and explanation labels below use the cleaned source line numbers. Older reports and historical timing manifests retain their original version context. The abstract was not regenerated; this technical report has no abstract section.

4. The environment, policy, trainer, and evaluator

4.1. Four responsibilities

The **environment** owns hidden health, synthetic measurements, estimated condition, inventory, crews, workshops, committed jobs, rewards, and the physical clock. A policy submits a decision; the simulator decides whether and how it can be executed. The policy does not directly write true health or change a running repair.

The **policy** maps permitted information to decisions. An actor produces action probabilities. A value learner estimates future return, either as a state value or an action/option value. Reinforcement learning (RL) updates these estimates from interaction; hierarchical reinforcement learning (HRL) introduces specified planning levels or temporally extended behavior. A physically nested component structure does not alone define an HRL algorithm [1, 2].

The **trainer** collects transitions, constructs targets, computes losses, and updates network parameters or tables. Different trainers in this file use different algorithms: proximal policy optimization (PPO), MAPPO, replay-based actor–critic, MAXQ completion-value learning, option-critic gradients, or hierarchical value-decomposition targets. They share a simulator, not one universal loss [3].

The **evaluator** creates fresh seeded episodes and requests fixed-weight decisions. It does not call an optimizer. It reconstructs the required option state at episode start. For neural categorical policies it normally chooses the most likely action; option-critic also uses a deterministic threshold for termination. These deterministic evaluation conventions differ from the stochastic choices used during training.

4.2. The notebook's library trainer versus the custom fleet trainers

Stable-Baselines3 (SB3) PPO is used in the introductory notebook example with `GymMaintenanceEnv`. The custom fleet models are independently constructed by `make_model` and updated by the appropriate custom training function. SB3 does not initialize these fleet weights and then pass them to a second optimizer.

The custom implementations make conditional action likelihoods, masks, per-agent statistics, option durations, and completion values explicit. Standard SB3 policies do not automatically implement those project-specific interfaces. This is an implementation and teaching choice, not a claim that SB3 cannot be extended. `FlattenFleetAction` only reshapes an action; it supplies neither conditional sampling nor a hierarchical training algorithm [4].

4.3. One day is the physical time unit

`FleetMaintenanceEnv.step` advances one day. Each eligible aircraft performs one modeled operating flight that day; a grounded aircraft does not. This is not an explicit route or multi-leg schedule. Maintenance occupies several days. Other aircraft continue operating while a job progresses.

Grounding, engine choice, bundle choice, crew selection, and workshop selection may be calculated through several Python calls before that one day advances. Those calls are computational planning stages. The older `execute_option` instead advances the entire fleet through several days before returning to its manager. The new `AircraftOptionEnv` always returns after one day and tracks each aircraft's committed physical option independently. These two interfaces therefore provide different manager decision opportunities.

5. State, observations, identities, and maintenance actions

5.1. Hidden plant and observed belief

For N aircraft, hidden health has shape $(N, 2, 6)$. The six component types are fan, low-pressure compressor (LPC), high-pressure compressor (HPC), combustor, high-pressure tur-

bine (HPT), and low-pressure turbine (LPT). Health is a normalized teaching variable, not a calibrated component map. A noisy synthetic efficiency/temperature model feeds a bootstrap particle filter. The controller observes estimates and uncertainty summaries, not the realized incident severity or hidden health array.

Each aircraft contributes 70 local values. The public vector has

$$P = 10 + 2C + 2W, \qquad D = 70N + P,$$

where C is crew count, W workshop count, and D total observed width. With seven aircraft, three crew members and two workshops, $P = 20$ and $D = 510$. The earlier report's 500-value example belonged to the older ten-public-value layout.

Local indices	Contents	Interpretation
0–11	Estimated health means	Two engines times six components.
12–23	Estimated standard deviations	Component uncertainty, not wear rates.
24–35	Lower health quantiles	Used for conservative urgency and grounding.
36	Exposure divided by 1.5	Fixed synthetic episode context; not flight hours.
37–38	Busy flag and remaining-job fraction	Whether a committed repair occupies the aircraft.
39–44	Committed plan fields	Engines, two bundles, crew identity mask, workshop and cyclic order start, all normalized.
45–56	Efficiency proxies	Synthetic readings, not directly observed physical health modifiers.
57–68	Normalized temperature proxies	Encoded through the fixed Celsius range.
69	Reading age divided by horizon	Increases while grounded because no new flight measurement is collected.

Public indices 0–9 contain day, free-resource fractions, replenishment timing and six stock values. Next come crew free flags, crew factor encodings, workshop free flags and workshop factor encodings. A duration factor f is encoded as $1/(1+f)$; its meaning remains a maintenance-time factor, not a direct reward bonus.

5.2. Bitwise selection and aircraft plates

For a component index c, `1 << c` sets one bit. Decimal 5 is 000101_2, selecting fan and HPC on one engine. The two engine bits identify which engine bundles are nonempty; they do not replace the component bundles. Similarly, `0b101` selects crew members 0 and 2. Bitwise AND tests membership, bitwise OR combines selections, and `bit_count()` counts selected members.

The workshop is a single integer index, not a bitmask. Aircraft plates are stable strings mapped to array indices, not a one-hot observation or a binary fleet status. Identity, availability, and operating history are separate concepts. Reset retains the plate/index mapping but starts a new episode history.

5.3. Full eight-field action and duration

The original daily fleet policies choose

```
[ground, engine_mask, bundle0, bundle1,
 crew_mask, workshop_index, order_start, priority]
```

for every aircraft. Their category counts are

$$[2, 4, 64, 64, 2^C, W, 12, N].$$

The 64 bundle categories are the 2^6 possible subsets. The crew head grows as 2^C, an explicit combinatorial limitation. A nonempty new repair requires a nonzero crew identity mask. Priority determines proposal processing order. `order_start` selects a cyclic order of engine/component slots, not an arbitrary permutation of selected tasks.

For crew set S, component labor L_c, and workshop factor f_w,

$$R_S = \sum_{i \in S} \frac{1}{f_i}, \qquad d_c = \max\left(1, \left\lceil \frac{L_c f_w}{R_S} \right\rceil\right), \qquad d_{job} = d_{setup} + \sum_c d_c.$$

Setup is fixed; component tasks execute sequentially. The stored cumulative completion days determine when each component is restored. Faster crew or workshop factors can reduce work time; they do not remove setup or create a new negative cost term. These are synthetic additive-work-rate assumptions.

5.4. Reservations, costs, and observation updates

Accepted plans reserve identified crew members and a workshop and consume selected spares. A busy aircraft's new row cannot replace the plan. Components restore to health one at their completion thresholds. The aircraft remains grounded until the entire job completes, and readings remain old until another flight.

The current cost includes setup, selected engines, parts, unused-life cost, downtime, failures, and rejected requests. Daily reward is $(10\, flights - cost)/(10N)$. The unsafe-flight hold uses the minimum observed lower health quantile over all components; it does not require HPC to fail first. Job duration is derived from the package and resources, rather than freely selected as a duration action. Spares are selected indirectly through replacement content; procurement and ordering are not learned actions.

6. Single-agent policies, shared parameters, and multi-agent information

6.1. Four independent design questions

Multi-agent reinforcement learning (MARL) requires specifying decision ownership and learning, not merely counting network objects. The following choices should be stated separately:

1. **Ownership:** one joint fleet controller, one agent per aircraft, or one agent per component type across aircraft.
2. **Information:** global observed fleet belief, aircraft-local information plus public variables, or a component-restricted view.

3. **Parameters:** shared actor weights, a central actor with distinct output blocks, or separately parameterized actors.

4. **Learning:** a fleet-joint PPO objective, per-agent MAPPO clipping, replay-based actor–critic, or a hierarchical/value-decomposition update.

Centralized training with decentralized execution (CTDE) is an information pattern, not a competing algorithm alongside PPO or DCMAC. In the present MAPPO methods the critic uses the full observed fleet input during training and the actors can execute from their restricted inputs. However, a fixed centralized broker still allocates resources. Actor proposal generation is decentralized; complete maintenance allocation is not fully decentralized.

6.2. Why one shared actor is not the original joint fleet controller

The original `conditional` policy includes a learned whole-fleet encoding when constructing each aircraft row and combines action likelihoods into one fleet likelihood. Aircraft MAPPO applies one shared actor to separate local/public observations and retains a log probability and advantage per agent. Thus the same network count can conceal different information and update rules.

The original `shared` mode is an important qualification: it also reuses local/public actor weights. It remains in the original fleet-joint update and full-action interface. It is closer in information structure to aircraft MAPPO, but it is not the new MAPPO objective or request-only action interface. In fact, `flat`, `conditional`, and `shared` all reuse their actor and field heads across aircraft. The name `shared` chiefly distinguishes omission of learned global actor context.

6.3. Why single-agent flat PPO is not DCMAC despite similar network counts

The `flat` baseline already has factorized categorical fields; it is not a network enumerating every possible joint action. DCMAC also uses factorized outputs, but the implemented DCMAC family defines aircraft/component control units, uses the replacement-request interface and fixed broker, and trains a global actor and scalar critic from replay with a joint truncated importance ratio. The original fleet PPO instead learns eight fields and optimizes clipped on-policy-style fleet likelihoods. A one-actor/one-critic count does not equate these implementations [5].

6.4. Why DDMAC has N networks and still needs global observations

Aircraft DDMAC sets its actor count from `len(env.aircraft_plates)`. Each list-comprehension iteration constructs a new network, so seven aircraft create seven independent parameter sets. All actors nevertheless receive the same global observed fleet vector; each owns only its own replacement request. This follows the global-belief actor convention discussed in the cited DDMAC formulation, rather than local-information execution [6].

DCMAC's single global actor has different output blocks for different units. Its shared trunk is not the same as repeatedly applying an identical local actor. Component-type DDMAC keeps six actors as fleet size changes, but each actor's inputs and output count still grow with N. Changing N constructs a new model and normally requires retraining; it does not dynamically add networks to an already running checkpoint.

6.5. Critic sharing is a separate choice

The original fleet PPO and DCMAC/DDMAC have one scalar state-value critic. MAPPO and the new planning-PPO modes use one centralized network with a separate output per agent/aircraft. This is one network with multiple value heads, not N separate critic networks. Option-critic has option-action values, MAXQ stores primitive/completion values, and hierarchical value-decomposition networks (VDN) sum learned agent Q values. They should not all be called the same critic architecture.

Separate local critics, a separate global critic per agent, independent local-input aircraft MAPPO actors, and shared component MAPPO actors are not additional configuration choices in the current registry. Their conceptual feasibility should not be reported as implementation.

7. Every implemented method and its concrete differences

For the table, a semi-Markov decision process (SMDP) accounts for actions spanning variable numbers of primitive days. Generalized advantage estimation (GAE) combines bootstrapped TD residuals. N denotes the configurable aircraft count; scalar V means one fleet state-value output, whereas an N-output V retains one baseline per aircraft.

Method	Actor information	Networks / value	Update
`flat`	Global + local	1 reused actor; scalar V	Joint PPO
`conditional`	Global + local	1 reused actor; scalar V	Joint PPO
`shared`	Local + public	1 reused actor; scalar V	Joint PPO
`options`	Global	4-choice manager; scalar V	SMDP PPO
`learned_options`	Global + local	Manager + conditioned worker	SMDP PPO
`mappo_aircraft`	Local + public	1 shared actor; N-output V	Per-agent PPO
`mappo_component`	Component view + public	6 actors; 6-output V	Per-agent PPO
`dcmac_aircraft`	Global belief	1 factorized actor; scalar V	Replay TD
`ddmac_aircraft`	Global belief	N actors; scalar V	Replay TD
`dcmac_component`	Global belief	1 factorized actor; scalar V	Replay TD
`ddmac_component`	Global belief	6 actors; scalar V	Replay TD
`concurrent_options`	Global + local	Shared manager; N-output V	Aircraft SMDP PPO
`planning_hierarchy`	Global + local	3 shared stage nets; N-output V	Aircraft SMDP PPO
`aggregate_targets`	Global + local	Manager + worker; N-output V	Aircraft SMDP PPO
`maxq`	Quantized global belief	Tabular primitive V and C	MAXQ-0
`option_critic`	Global + local	Worker + Q + beta	Online TD + termination
`hierarchical_vdn`	Local + public	N high + N low Q nets	Two-level additive TD

Method	Actor information	Networks / value	Update
`hrl_flat`	Global + local	Shared catalog actor; N-output V	Aircraft SMDP PPO

7.1. flat

Original flat field logits; later validity masks still depend on earlier choices.

Decision authority. Full fleet plan. **Information and parameters.** Global + local; 1 reused actor; scalar V. **Learning.** Joint PPO.

7.2. conditional

Previous field choices additionally condition later logits.

Decision authority. Full fleet plan. **Information and parameters.** Global + local; 1 reused actor; scalar V. **Learning.** Joint PPO.

7.3. shared

No global actor encoding; still one fleet-joint training likelihood.

Decision authority. Full fleet plan. **Information and parameters.** Local + public; 1 reused actor; scalar V. **Learning.** Joint PPO.

7.4. options

Learned manager over four fixed threshold routines; fleet-wide decision suspension.

Decision authority. Fleet option. **Information and parameters.** Global; 4-choice manager; scalar V. **Learning.** SMDP PPO.

7.5. learned_options

Both levels learn; one full plan is held by the fixed repair executor.

Decision authority. Fleet option + plan. **Information and parameters.** Global + local; Manager + conditioned worker. **Learning.** SMDP PPO.

7.6. mappo_aircraft

N aircraft decision roles; two 64-way bundle heads per role.

Decision authority. 2 bundles / aircraft. **Information and parameters.** Local + public; 1 shared actor; N-output V. **Learning.** Per-agent PPO.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

7.7. mappo_component

Six component types; each actor makes N four-way engine decisions.

Decision authority. Engine masks / type. **Information and parameters.** Component view + public; 6 actors; 6-output V. **Learning.** Per-agent PPO.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

7.8. dcmac_aircraft

Output blocks represent N aircraft agents; no local-view execution.

Decision authority. 2 bundles / aircraft. **Information and parameters.** Global belief; 1 factorized actor; scalar V. **Learning.** Replay TD.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

7.9. ddmac_aircraft

An independent global-input network is constructed for every aircraft.

Decision authority. 2 bundles / aircraft. **Information and parameters.** Global belief; N actors; scalar V. **Learning.** Replay TD.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

The independent actor count follows ownership: N for aircraft assignment, six for component-type assignment. Each network still receives the global observed vector; the word decentralized here describes parameter separation, not local information.

7.10. dcmac_component

Six output blocks; N four-way fields within each component block.

Decision authority. Engine masks / type. **Information and parameters.** Global belief; 1 factorized actor; scalar V. **Learning.** Replay TD.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

7.11. ddmac_component

One independent global-input actor for each component type.

Decision authority. Engine masks / type. **Information and parameters.** Global belief; 6 actors; scalar V. **Learning.** Replay TD.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

The independent actor count follows ownership: N for aircraft assignment, six for component-type assignment. Each network still receives the global observed vector; the word decentralized here describes parameter separation, not local information.

7.12. concurrent_options

Fixed condition-based worker; free aircraft replan while others remain busy.

Decision authority. Operate / maintain. **Information and parameters.** Global + local; Shared manager; N-output V. **Learning.** Aircraft SMDP PPO.

A free aircraft starts a decision; its return spans the actual time until it becomes free again. Other free aircraft continue deciding on intervening days. Structural masks do not guarantee inventory acceptance.

7.13. planning_hierarchy

Three conditioned planning stages, followed by committed execution.

Decision authority. Ground / engine / bundle. **Information and parameters.** Global + local; 3 shared stage nets; N-output V. **Learning.** Aircraft SMDP PPO.

A free aircraft starts a decision; its return spans the actual time until it becomes free again. Other free aircraft continue deciding on intervening days. Structural masks do not guarantee inventory acceptance.

7.14. aggregate_targets

Goals 0/1/2/12 limit component instances; targets do not ensure broker acceptance.

Decision authority. Budget then catalog. **Information and parameters.** Global + local; Manager + worker; N-output V. **Learning.** Aircraft SMDP PPO.

A free aircraft starts a decision; its return spans the actual time until it becomes free again. Other free aircraft continue deciding on intervening days. Structural masks do not guarantee inventory acceptance.

7.15. maxq

One aircraft only; child completion is learned with actual duration.

Decision authority. Recursive task tree. **Information and parameters.** Quantized global belief; Tabular primitive V and C. **Learning.** MAXQ-0.

7.16. option_critic

Three modes; safe beta is zero while physical repair remains committed.

Decision authority. Mode and catalog. **Information and parameters.** Global + local; Worker + Q + beta. **Learning.** Online TD + termination.

7.17. hierarchical_vdn

Three-day upper clock, daily lower clock, disclosed advantage reward shaping.

Decision authority. Budget and catalog. **Information and parameters.** Local + public; N high + N low Q nets. **Learning.** Two-level additive TD.

7.18. hrl_flat

Comparator for staged planning using the same catalog, broker and decision timing.

Decision authority. 22-entry catalog. **Information and parameters.** Global + local; Shared catalog actor; N-output V. **Learning.** Aircraft SMDP PPO.

A free aircraft starts a decision; its return spans the actual time until it becomes free again. Other free aircraft continue deciding on intervening days. Structural masks do not guarantee inventory acceptance.

8. Training equations and what their axes mean

8.1. Fleet-joint PPO and per-agent MAPPO

The original full-action fleet methods sum log probabilities across fields and aircraft. PPO compares the same stored fleet action under current and collection-time policies. MAPPO sums only fields belonging to one agent, preserving the agent axis before clipping. Their ratios are respectively

$$r_t^{fleet} = \exp\left(\sum_i \log \pi_i^{new} - \sum_i \log \pi_i^{old}\right), \qquad r_{t,i} = \exp\left(\log \pi_i^{new} - \log \pi_i^{old}\right).$$

Clipping the product is not the same operation as clipping each ratio. For positive unit advantages and individual ratios 2 and 0.5, the averaged per-agent clipped surrogate is $(1.2 + 0.5)/2 = 0.85$, while the joint ratio is one. This explains a concrete code difference without attributing fleet performance to a benchmark paper [7].

8.2. Replay-based DCMAC and DDMAC

The code stores original submitted proposal likelihoods even when the broker defers the request. For replay behavior policy μ and current policy π, it calculates

$$w = \min\left(2, \exp\left[\sum_i \log \pi_i(a_i \mid b) - \sum_i \log \mu_i(a_i \mid b)\right]\right),$$

$$y = r + \gamma(1-d)V(b'), \qquad \delta = y - V(b).$$

The actor minimizes $-\mathbb{E}[w \log \pi(a \mid b)\,\mathrm{stopgrad}(\delta)]$, and the critic minimizes $\frac{1}{2}\mathbb{E}[w\delta^2]$. Weights and next-state targets are detached. The cap 2 and small-network settings are tutorial choices. Replay truncation trades variance for bias; no replay state-distribution correction is implemented. These weights correct policy likelihood mismatch, not oversampling of rare hazard scenarios.

8.3. Aircraft option durations and completion values

For rewards accumulated over k physical days, the new aircraft-option target is

$$y = \sum_{j=0}^{k-1} \gamma^j r_j + \gamma^k (1-d) V(b_{next}).$$

A training-budget cutoff can bootstrap from a still-busy state; it does not cancel the job or claim option completion. Pending decisions retain their collection-time likelihoods across rollout updates. This is a delayed-decision PPO adaptation, not an assertion that all later samples are freshly on-policy. The critic also learns daily targets, including busy states.

MAXQ-0 uses a different decomposition, $Q(i, s, a) = V(a, s) + C(i, s, a)$, where primitive V values estimate immediate reward and C estimates future reward needed to complete the parent after the child. The code updates C with the parent's continuation value discounted over the actual child duration. Quantizing observed beliefs can alias states, so exact-state guarantees are not claimed [8].

8.4. Option-critic termination and two-level value-decomposition networks

Option-critic constructs a continuation value

$$U(b', o) = (1 - \beta_{safe})Q_\Omega(b', o) + \beta_{safe} \max_{o'} Q_\Omega(b', o').$$

Here $\beta_{safe} = 0$ while physical repair is committed. The internal policy receives a detached TD advantage. The termination network uses an option-advantage loss with a 0.01 margin at permitted nonterminal free-aircraft states only. This learns planning-mode termination without acquiring permission to interrupt repair [9].

The hierarchical value-decomposition network (VDN) adaptation sums agent values at the high and low levels. The high target uses a three-day-or-shorter macro return. The low reward adds $0.1\delta_H/k$, using the detached high-level TD advantage δ_H over macro duration k. This coupling changes the low-level objective. The implementation draws on hierarchical cooperative coordination, but does not reproduce the complete hierarchical cooperative multi-agent reinforcement learning with dual coordination mechanism (HAVEN) or the nonlinear value mixer used by QMIX [10, 11].

9. Comparison design, verification, and remaining work

9.1. Which comparisons isolate which changes

The closest new representation comparison is `hrl_flat` versus `planning_hierarchy`: both have global/local inputs, the same 22 catalog requests, the same fixed resource broker, the same aircraft-specific decision opportunities, critic shape, duration accounting and PPO update. Their network parameterization and parameter counts differ. Aggregate targets retain the same union of possible requests but change the upper action semantics. The concurrent-options worker is fixed and therefore restricts realized choices further.

The six MAPPO/DCMAC/DDMAC variants share their broader replacement-request interface and broker, but differ in actor information, sharing, critic outputs, or update. They cannot be treated as a single-factor agent-count ablation. The original full-action PPO methods also learn resource fields, whereas all request methods and the new catalog methods delegate those fields. Older fleet options reduce manager opportunities. VDN changes macro timing and shaped reward. MAXQ supports one aircraft only. Equal simulation days do not equalize gradient updates, network size, or wall-clock effort.

9.2. Recorded software checks

The cleanup verification records 45 passing behavioral tests, including the earlier checks of environment contracts, identities, resource reservations, committed work, observation boundaries, likelihoods, targets, gradients and checkpoint round trips. Two additional tests cover direct critic evaluation without actor-context construction and rejection of nonfinite initial health. These are software assertions, not 45 independent before-and-after equivalence experiments.

For each of the 18 learned methods, the old and cleaned source were also trained for 64 simulated days with training seed 771, horizon 12, 16 particles and rollout size 16 where applicable. MAXQ used one aircraft and the other methods used two. Training logs and final state dictionaries matched exactly, and evaluation records matched at seed 10421. One historical seed-0 checkpoint per method also loaded with identical configuration and state and produced matching evaluation records at seed 10000. Those checkpoint comparisons used seven aircraft except for single-aircraft MAXQ. They do not cover every saved checkpoint or exact optimizer/replay resumption.

The two optimized critic paths produced exactly matching outputs and input gradients on one three-aircraft observation. This check does not establish equality for every input, device or batch shape. The earlier execution of all 122 code cells in the comprehensive notebook remains a historical result. The cleanup changed 52 Markdown cells, while its code cells, outputs and execution counts were retained. Neither that notebook nor the full presentation experiment was rerun for this report revision.

The presentation experiment used 1,024 training days per seed, three training seeds and 20 evaluation seeds. Its saved timings describe the earlier source snapshot. The short cleanup comparisons establish compatibility within the tested cases; they do not establish convergence, a performance ranking or improved end-to-end training time.

9.3. Important physical and methodological limits

The particle filter returns condition summaries, not a remaining-useful-life forecast. It resamples component marginals independently, so common physical weather does not create a fully dependency-aware Bayesian filter. Temperature still includes the aircraft-specific exposure offset. Bird/hail events are sampled independently per engine, with uniform severities and fixed component weight vectors; there is no aircraft encounter stage, explicit no-damage branch, or separate operability state.

The 22-entry menu is structurally defined, not a guarantee of stock feasibility. For example, one aircraft initially has one spare per component type, so a request for the same component on both engines needs more stock than is present under that configuration. The broker can defer it. Repair order changes component restoration timing, but the current whole-job grounding rule can leave total return unchanged for a fixed package. An earlier read-only two-task probe observed exactly that behavior.

Calendar-based staffing, graph/attention/hypergraph policies, learned peer messaging, arbitrary task permutations, procurement decisions, rolling-horizon or mixed-integer baselines, a fleet exact-search oracle, calibrated hazard parameters, rare-event confidence intervals, and unchanged-weight transfer across fleet sizes remain outside the implementation described here. The report does not promote a code prototype into an empirical literature result.

10. Source walkthrough conventions

The following sections cover the current core in source order. A class introduction states what it owns. Each function introduction explains its purpose, inputs, side effects and calls where relevant. Listings reproduce the nonblank lines of the cleaned source, with current source labels preserved through a line mapping. The prose describes every nonblank retained physical line, including continuation lines. For a multi-line expression, later fragments must be read together with the initiating statement; they are not independent operations.

10.1. Module header and imports

Source lines 1–21.

```
# Maintenance tutorial core; full line-by-line explanations are in the code reports.
# Updated simulator: fixed aircraft identities and individually assigned heterogeneous maintenance resources.
"""One supporting module for the comprehensive RL/HRL maintenance course.
OBSERVATION_VERSION=3 adds individual resource availability and duration factors.
Fleet action field 4 is now a crew-selection bitmask, not a crew count minus one.
The first examples and user's exercise answers are namespaced by tutorial part.
The fleet classes/functions keep their public names. No import depends on Code.
All figures are teaching assumptions rather than real turbofan calibration.
"""
from __future__ import annotations
# ======================================================================
# mini_maintenance.py
# ======================================================================
import random
from typing import TypeAlias
Observation: TypeAlias = tuple[int, int]
```

Line-by-line explanation.

1 Source comment: Maintenance tutorial core; full line-by-line explanations are in the code reports. This line documents the following code and does not execute.

2 Source comment: Updated simulator: fixed aircraft identities and individually assigned heterogeneous maintenance resources. This line documents the following code and does not execute.

3 Documentation text: One supporting module for the comprehensive RL/HRL maintenance course.. This states the contract or qualification for `module`; it does not update the simulator or model.

5 Documentation text: OBSERVATION_VERSION=3 adds individual resource availability and duration factors.. This states the contract or qualification for `module`; it does not update the simulator or model.

6 Documentation text: Fleet action field 4 is now a crew-selection bitmask, not a crew count minus one.. This states the contract or qualification for `module`; it does not update the simulator or model.

7 Documentation text: The first examples and user's exercise answers are namespaced by tutorial part.. This states the contract or qualification for `module`; it does not update the simulator or model.

8 Documentation text: The fleet classes/functions keep their public names. No import depends on Code.. This states the contract or qualification for `module`; it does not update the simulator or model.

9 Documentation text: All figures are teaching assumptions rather than real turbofan calibration.. This states the contract or qualification for `module`; it does not update the simulator or model.

10 Close the documentation string for `module`. This delimiter ends the descriptive text and performs no simulator or learning update.

11 Import `from __future__ import annotations`. The imported names are used by the definitions below; importing does not start a training run.

14 A visual section separator in the source. It has no executable effect.

15 Source comment: mini_maintenance.py This line documents the following code and does not execute.

16 A visual section separator in the source. It has no executable effect.

19 Import `import random`. Random sampling supports the small simulator and reproducible policies.

20 Import `from typing import TypeAlias`. The imported names are used by the definitions below; importing does not start a training run.

21 Set `Observation` by select the indexed/sliced entries of tuple.

10.2. MiniMaintenanceEnv

A one-item teaching Markov decision process. Operate and replace actions expose the basic observation, transition, reward and episode contract before the fleet model is introduced.

Source lines 23–39.

```
23 class MiniMaintenanceEnv:
24     """One aircraft with one replaceable item and two high-level actions.
26     Actions:
27         0 = operate for one day
28         1 = ground for one day and replace the item
30     Health is an integer in {0, 1, 2, 3}; 3 is new and 0 is failed.
31     A successful operation earns +5. If operation ends at health 0, it earns
32     -12 instead. Grounding/replacement earns -4 and restores health to 3.
33     Operating wear is 1, plus one extra unit with probability shock_prob.
35     The episode ends after ``horizon`` days. Failure itself does not end it:
36     the policy can ground and restore the item on a later day.
37     """
38     OPERATE = 0
39     GROUND_AND_REPLACE = 1
```

Line-by-line explanation.

23 Define `MiniMaintenanceEnv` with the stated inheritance. A one-item teaching Markov decision process. Operate and replace actions expose the basic observation, transition, reward and episode contract before the fleet model is introduced.

24 Documentation text: One aircraft with one replaceable item and two high-level actions.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

26 Documentation text: Actions:. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

27 Documentation text: 0 = operate for one day. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

28 Documentation text: 1 = ground for one day and replace the item. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

30 Documentation text: Health is an integer in {0, 1, 2, 3}; 3 is new and 0 is failed.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

31 Documentation text: A successful operation earns +5. If operation ends at health 0, it earns. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

32 Documentation text: -12 instead. Grounding/replacement earns -4 and restores health to 3.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

33 Documentation text: Operating wear is 1, plus one extra unit with probability shock_prob.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

35 Documentation text: The episode ends after “horizon“ days. Failure itself does not end it:. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

36 Documentation text: the policy can ground and restore the item on a later day.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

37 Close the documentation string for `MiniMaintenanceEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

38 Set `OPERATE` by use the literal 0.

39 Set `GROUND_AND_REPLACE` by use the literal 1.

10.3. MiniMaintenanceEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 41–51.

```
41      def __init__(self, horizon: int=12, shock_prob: float=0.0) -> None:
42          if not isinstance(horizon, int) or horizon <= 0:
43              raise ValueError('horizon must be a positive integer')
44          if not 0.0 <= shock_prob <= 1.0:
45              raise ValueError('shock_prob must be between 0 and 1')
46          self.horizon = horizon
47          self.shock_prob = float(shock_prob)
48          self._rng = random.Random()
49          self.health = 3
50          self.day = 0
51          self._needs_reset = True
```

Line-by-line explanation.

41 Define `__init__(self, horizon: int=12, shock_prob: float=0.0) -> None`. This ___init___ method is part of MiniMaintenanceEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

42 Test `not isinstance(horizon, int) or horizon <= 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

43 Raise `ValueError('horizon must be a positive integer')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

44 Test `not 0.0 <= shock_prob <= 1.0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

45 Raise `ValueError('shock_prob must be between 0 and 1')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

46 Set `self.horizon` by evaluate horizon.

47 Set `self.shock_prob` by convert the result to a Python floating-point number.

48 Set `self._rng` by call random.Random with the shown positional and keyword arguments.

49 Set `self.health` by use the literal 3. It holds hidden physical component condition.

50 Set `self.day` by use the literal 0.

51 Set `self._needs_reset` by use the literal True.

10.4. MiniMaintenanceEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 53–66.

```
53     def reset(self, *, seed: int | None=None, initial_health: int=3):
54         """Start an episode and return (observation, info).
56         A seed makes subsequent stochastic wear reproducible. Calling reset
57         without a seed keeps the current generator stream.
58         """
59         if initial_health not in (0, 1, 2, 3):
60             raise ValueError('initial_health must be 0, 1, 2, or 3')
61         if seed is not None:
62             self._rng.seed(seed)
63         self.health = initial_health
64         self.day = 0
65         self._needs_reset = False
66         return (self._observation(), {'day': self.day})
```

Line-by-line explanation.

53 Define `reset(self, *, seed: int | None=None, initial_health: int=3)`. This reset method is part of MiniMaintenanceEnv.reset; its parameters and defaults are the interface used by the calls in this section.

54 Documentation text: Start an episode and return (observation, info).. This states the contract or qualification for `MiniMaintenanceEnv.reset`; it does not update the simulator or model.

56 Documentation text: A seed makes subsequent stochastic wear reproducible. Calling reset. This states the contract or qualification for `MiniMaintenanceEnv.reset`; it does not update the simulator or model.

57 Documentation text: without a seed keeps the current generator stream.. This states the contract or qualification for `MiniMaintenanceEnv.reset`; it does not update the simulator or model.

58 Close the documentation string for `MiniMaintenanceEnv.reset`. This delimiter ends the descriptive text and performs no simulator or learning update.

59 Test `initial_health not in (0, 1, 2, 3)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

60 Raise `ValueError('initial_health must be 0, 1, 2, or 3')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

61 Test `seed is not None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

62 Call self._rng.seed with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

63 Set `self.health` by evaluate initial_health. It holds hidden physical component condition.

64 Set `self.day` by use the literal 0.

65 Set `self._needs_reset` by use the literal False.

66 Return `(self._observation(), {'day': self.day})` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.5. MiniMaintenanceEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 68–85.

```
68      def step(self, action: int):
69          """Return (next_observation, reward, terminated, truncated, info)."""
70          if self._needs_reset:
71              raise RuntimeError('call reset() before step(), including after an episode')
72          if action not in (self.OPERATE, self.GROUND_AND_REPLACE):
73              raise ValueError('action must be 0 (operate) or 1 (ground/replace)')
74          previous_health = self.health
75          if action == self.OPERATE:
76              extra_wear = int(self._rng.random() < self.shock_prob)
77              self.health = max(0, self.health - 1 - extra_wear)
78              failed = self.health == 0
79              reward = -12.0 if failed else 5.0
80              info = {'event': 'failed_operation' if failed else 'successful_operation', 'wear':
                ↪  previous_health - self.health, 'failed': failed, 'grounded': False}
81          else:
82              self.health = 3
83              reward = -4.0
84              info = {'event': 'grounded_and_replaced', 'wear': 0, 'failed': False, 'grounded': True}
85          self.day += 1
```

Line-by-line explanation.

68 Define `step(self, action: int)`. This step method is part of MiniMaintenanceEnv.step; its parameters and defaults are the interface used by the calls in this section.

69 Documentation text: Return (next_observation, reward, terminated, truncated, info).. This states the contract or qualification for `MiniMaintenanceEnv.step`; it does not update the simulator or model.

70 Test `self._needs_reset`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

71 Raise `RuntimeError('call reset() before step(), including after an episode')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

72 Test `action not in (self.OPERATE, self.GROUND_AND_REPLACE)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

73 Raise `ValueError('action must be 0 (operate) or 1 (ground/replace)')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

74 Set `previous_health` by evaluate self.health.

75 Test `action == self.OPERATE`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

76 Set `extra_wear` by convert the result to a Python integer.

77 Set `self.health` by call max with the shown positional and keyword arguments. It holds hidden physical component condition.

78 Set `failed` by evaluate the comparison self.health == 0.

79 Set `reward` by select -12.0 when failed, otherwise 5.0.

80 Set `info` by construct the shown dict in its stated order.

81 Execute this alternative only when the corresponding preceding condition did not select its branch.

82 Set `self.health` by use the literal 3. It holds hidden physical component condition.

83 Set `reward` by evaluate -4.0.

84 Set `info` by construct the shown dict in its stated order.

85 Add `1` to `self.day`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Source lines 86–90.

```
86          terminated = self.day >= self.horizon
87          truncated = False
88          self._needs_reset = terminated
89          info['day'] = self.day
90          return (self._observation(), reward, terminated, truncated, info)
```

Line-by-line explanation.

86 Set `terminated` by evaluate the comparison self.day >= self.horizon.

87 Set `truncated` by use the literal False.

88 Set `self._needs_reset` by evaluate terminated.

89 Set `info['day']` by evaluate self.day.

90 Return `(self._observation(), reward, terminated, truncated, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.6. MiniMaintenanceEnv._observation

Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

Source lines 92–93.

```
92      def _observation(self) -> Observation:
93          return (self.health, self.horizon - self.day)
```

Line-by-line explanation.

92 Define `_observation(self) -> Observation`. Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

93 Return `(self.health, self.horizon - self.day)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.7. MiniMaintenanceEnv.render_text

This method implements MiniMaintenanceEnv.render_text in the class described above. Read its inputs, validity branches and returned objects together; persistent writes use the instance or simulator records.

Source lines 95–97.

```
95      def render_text(self) -> str:
96          """Small human-readable display; useful for tracing a policy."""
97          return f'day={self.day:2d}  health={self.health}  steps_left={self.horizon - self.day}'
```

Line-by-line explanation.

95 Define `render_text(self) -> str`. This render_text method is part of MiniMaintenanceEnv.render_text; its parameters and defaults are the interface used by the calls in this section.

96 Documentation text: Small human-readable display; useful for tracing a policy.. This states the contract or qualification for `MiniMaintenanceEnv.render_text`; it does not update the simulator or model.

97 Return `f'day={self.day:2d} health={self.health} steps_left={self.horizon - self.day}'` to the caller by evaluate f'day={self.day:2d} health={self.health} steps_left={self.horizon - self.day}'. No further statements in this call execute.

10.8. random_policy

Select one of the tiny environment actions with an explicitly supplied random generator.

Source lines 99–101.

```
99  def random_policy(observation: Observation, *, rng: random.Random) -> int:
100     """Random baseline, independent of health."""
101     return rng.randrange(2)
```

Line-by-line explanation.

99 Define `random_policy(observation: Observation, *, rng: random.Random) -> int`. Select one of the tiny environment actions with an explicitly supplied random generator.

100 Documentation text: Random baseline, independent of health.. This states the contract or qualification for `random_policy`; it does not update the simulator or model.

101 Return `rng.randrange(2)` to the caller by call rng.randrange with the shown positional and keyword arguments. No further statements in this call execute.

10.9. threshold_policy

Use observed tiny-environment health to choose operation or replacement without learning.

Source lines 103–106.

```
103 def threshold_policy(observation: Observation) -> int:
104     """Simple rule baseline: replace at health 0 or 1."""
105     health, _steps_left = observation
106     return MiniMaintenanceEnv.GROUND_AND_REPLACE if health <= 1 else MiniMaintenanceEnv.OPERATE
```

Line-by-line explanation.

103 Define `threshold_policy(observation: Observation) -> int`. Use observed tiny-environment health to choose operation or replacement without learning.

104 Documentation text: Simple rule baseline: replace at health 0 or 1.. This states the contract or qualification for `threshold_policy`; it does not update the simulator or model.

105 Set `(health, _steps_left)` by evaluate observation.

106 Return `MiniMaintenanceEnv.GROUND_AND_REPLACE if health <= 1 else MiniMaintenanceEnv.OPERATE` to the caller by select MiniMaintenanceEnv.GROUND_AND_REPLACE when health <= 1, otherwise MiniMaintenanceEnv.OPERATE. No further statements in this call execute.

10.10. rollout

Run a tiny-environment episode, retaining transitions so state, action, reward and termination can be inspected.

Source lines 108–119.

```
108 def rollout(env: MiniMaintenanceEnv, policy, *, seed: int=0):
109     """Execute one episode; policy receives observation and returns an action."""
110     observation, _info = env.reset(seed=seed)
111     rows: list[dict] = []
112     while True:
113         action = policy(observation)
114         next_observation, reward, terminated, truncated, info = env.step(action)
115         rows.append({'observation': observation, 'action': action, 'reward': reward,
            ↪ 'next_observation': next_observation, 'event': info['event'], 'terminated': terminated,
            ↪ 'truncated': truncated})
116         observation = next_observation
117         if terminated or truncated:
118             break
119     return rows
```

Line-by-line explanation.

108 Define `rollout(env: MiniMaintenanceEnv, policy, *, seed: int=0)`. Run a tiny-environment episode, retaining transitions so state, action, reward and termination can be inspected.

109 Documentation text: Execute one episode; policy receives observation and returns an action.. This states the contract or qualification for `rollout`; it does not update the simulator or model.

110 Set `(observation, _info)` by initialize a new episode using the stated seed/options.

111 Set `rows` by construct the shown list in its stated order.

112 Repeat the indented block while `True` remains true; this condition controls the simulation/collection loop here.

113 Set `action` by call policy with the shown positional and keyword arguments. It holds the sampled or stored action whose probability is evaluated.

114 Set `(next_observation, reward, terminated, truncated, info)` by execute this object's step with the shown arguments.

115 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

116 Set `observation` by evaluate next_observation. It holds the policy-visible belief and public context.

117 Test `terminated or truncated`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

118 Exit the nearest loop immediately; execution resumes after that loop.

119 Return `rows` to the caller by evaluate rows. No further statements in this call execute.

10.11. Module declarations and integration

Source lines 122–128.

```
122 # ============================================================================
123 # hierarchical_maintenance.py
124 # ============================================================================
127 from itertools import product
128 TOY_COMPONENTS = ('fan', 'compressor')
```

Line-by-line explanation.

122 A visual section separator in the source. It has no executable effect.

123 Source comment: hierarchical_maintenance.py This line documents the following code and does not execute.

124 A visual section separator in the source. It has no executable effect.

127 Import `from itertools import product`. The imported names are used by the definitions below; importing does not start a training run.

128 Set `TOY_COMPONENTS` by construct the shown tuple in its stated order.

10.12. valid_action

Check the conditional tuple contract before a small-example transition can execute.

Source lines 130–146.

```
130 def valid_action(action: tuple[int, int, int]) -> bool:
131     """Action is (ground, engine_mask, component_mask).
133     Engine bits 0 and 1 represent engines 0 and 1. Component bits 0-1
134     represent engine 0; bits 2-3 represent engine 1. A selected engine must
135     contain at least one selected component, and no unselected engine may
136     receive component work.
137     """
138     if len(action) != 3 or any((type(x) is not int for x in action)):
139         return False
140     ground, engines, components = action
141     if ground == 0:
142         return engines == 0 and components == 0
143     if ground != 1 or not 1 <= engines <= 3 or (not 1 <= components <= 15):
144         return False
145     selected_engines = sum((1 << engine for engine in range(2) if components & 3 << 2 * engine))
146     return engines == selected_engines
```

Line-by-line explanation.

130 Define `valid_action(action: tuple[int, int, int]) -> bool`. Check the conditional tuple contract before a small-example transition can execute.

131 Documentation text: Action is (ground, engine_mask, component_mask).. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

133 Documentation text: Engine bits 0 and 1 represent engines 0 and 1. Component bits 0-1. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

134 Documentation text: represent engine 0; bits 2-3 represent engine 1. A selected engine must. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

135 Documentation text: contain at least one selected component, and no unselected engine may. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

136 Documentation text: receive component work.. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

137 Close the documentation string for `valid_action`. This delimiter ends the descriptive text and performs no simulator or learning update.

138 Test `len(action) != 3 or any((type(x) is not int for x in action))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

139 Return `False` to the caller by use the literal False. No further statements in this call execute.

140 Set `(ground, engines, components)` by evaluate action.

141 Test `ground == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

142 Return `engines == 0 and components == 0` to the caller by evaluate engines == 0 and components == 0. No further statements in this call execute.

143 Test `ground != 1 or not 1 <= engines <= 3 or (not 1 <= components <= 15)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

144 Return `False` to the caller by use the literal False. No further statements in this call execute.

145 Set `selected_engines` by add the terms of the supplied iterable.

146 Return `engines == selected_engines` to the caller by evaluate the comparison engines == selected_engines. No further statements in this call execute.

10.13. all_actions

Enumerate all structurally valid tuples in the small conditional example.

Source lines 148–150.

```
148 def all_actions():
149     """List the 16 feasible composite actions, including operate."""
150     return [(0, 0, 0)] + [(1, engines, components) for engines, components in product(range(1, 4),
    ↪  range(1, 16)) if valid_action((1, engines, components))]
```

Line-by-line explanation.

148 Define `all_actions()`. Enumerate all structurally valid tuples in the small conditional example.

149 Documentation text: List the 16 feasible composite actions, including operate.. This states the contract or qualification for `all_actions`; it does not update the simulator or model.

150 Return `[(0, 0, 0)] + [(1, engines, components) for engines, components in product(range(1, 4), range(1, 16)) if valid_action((1, engines, components))]` to the caller by add [(0, 0, 0)] and [(1, engines, components) for engines, components in product(range(1, 4), range(1, 16)) if valid_action((1, engines, components))]. No further statements in this call execute.

10.14. ConditionalMaintenanceEnv

A small structural-action example. Grounding gates engine and component choices; it demonstrates valid conditional tuples, rather than the later fleet resource scheduler.

Source lines 152–153.

```
152 class ConditionalMaintenanceEnv:
153     """Tiny deterministic simulator for one aircraft with two engines."""
```

Line-by-line explanation.

152 Define `ConditionalMaintenanceEnv` with the stated inheritance. A small structural-action example. Grounding gates engine and component choices; it demonstrates valid conditional tuples, rather than the later fleet resource scheduler.

153 Documentation text: Tiny deterministic simulator for one aircraft with two engines.. This states the contract or qualification for `ConditionalMaintenanceEnv`; it does not update the simulator or model.

10.15. ConditionalMaintenanceEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 155–161.

```
155     def __init__(self, horizon: int=6):
156         if not isinstance(horizon, int) or horizon <= 0:
157             raise ValueError('horizon must be positive')
158         self.horizon = horizon
159         self.health = [3, 3, 3, 3]
160         self.day = 0
161         self._needs_reset = True
```

Line-by-line explanation.

155 Define `__init__(self, horizon: int=6)`. This ___init___ method is part of ConditionalMaintenanceEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

156 Test `not isinstance(horizon, int) or horizon <= 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

157 Raise `ValueError('horizon must be positive')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

158 Set `self.horizon` by evaluate horizon.

159 Set `self.health` by construct the shown list in its stated order. It holds hidden physical component condition.

160 Set `self.day` by use the literal 0.

161 Set `self._needs_reset` by use the literal True.

10.16. ConditionalMaintenanceEnv._observation

Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

Source lines 163–164.

```
163     def _observation(self):
164         return tuple(self.health) + (self.horizon - self.day,)
```

Line-by-line explanation.

163 Define `_observation(self)`. Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

164 Return `tuple(self.health) + (self.horizon - self.day,)` to the caller by add tuple(self.health) and (self.horizon - self.day,). No further statements in this call execute.

10.17. ConditionalMaintenanceEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 166–172.

```
166     def reset(self, initial_health=(3, 3, 3, 3)):
167         if len(initial_health) != 4 or any((h not in range(4) for h in initial_health)):
168             raise ValueError('provide four health values in 0..3')
169         self.health = list(initial_health)
170         self.day = 0
171         self._needs_reset = False
172         return (self._observation(), {})
```

Line-by-line explanation.

166 Define `reset(self, initial_health=(3, 3, 3, 3))`. This reset method is part of ConditionalMaintenanceEnv.reset; its parameters and defaults are the interface used by the calls in this section.

167 Test `len(initial_health) != 4 or any((h not in range(4) for h in initial_health))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

168 Raise `ValueError('provide four health values in 0..3')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

169 Set `self.health` by call list with the shown positional and keyword arguments. It holds hidden physical component condition.

170 Set `self.day` by use the literal 0.

171 Set `self._needs_reset` by use the literal False.

172 Return `(self._observation(), {})` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.18. ConditionalMaintenanceEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 174–191.

```
174     def step(self, action):
175         if self._needs_reset:
176             raise RuntimeError('reset before stepping')
177         if not valid_action(action):
178             raise ValueError(f'infeasible hierarchical action: {action}')
179         ground, engines, components = action
180         if ground:
181             for index in range(4):
182                 if components & 1 << index:
183                     self.health[index] = 3
184             n_engines = engines.bit_count()
185             n_components = components.bit_count()
186             reward = -3.0 - 2.0 * n_engines - 1.0 * n_components
187             event = 'grounded_and_replaced'
188         else:
```

```
189             self.health = [max(0, h - 1) for h in self.health]
190             reward = -15.0 if 0 in self.health else 8.0
191             event = 'failed_operation' if reward < 0 else 'successful_operation'
```

Line-by-line explanation.

174 Define `step(self, action)`. This step method is part of ConditionalMaintenanceEnv.step; its parameters and defaults are the interface used by the calls in this section.

175 Test `self._needs_reset`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

176 Raise `RuntimeError('reset before stepping')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

177 Test `not valid_action(action)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

178 Raise `ValueError(f'infeasible hierarchical action: {action}')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

179 Set `(ground, engines, components)` by evaluate action.

180 Test `ground`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

181 Iterate `index` over `range(4)`. The following indented statements run once for each selected element.

182 Test `components & 1 << index`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

183 Set `self.health[index]` by use the literal 3. It holds hidden physical component condition.

184 Set `n_engines` by call engines.bit_count with the shown positional and keyword arguments.

185 Set `n_components` by call components.bit_count with the shown positional and keyword arguments.

186 Set `reward` by subtract -3.0 - 2.0 * n_engines and 1.0 * n_components.

187 Set `event` by use the literal 'grounded_and_replaced'.

188 Execute this alternative only when the corresponding preceding condition did not select its branch.

189 Set `self.health` by construct the collection by iterating the shown comprehension and applying its filters. It holds hidden physical component condition.

190 Set `reward` by select -15.0 when 0 in self.health, otherwise 8.0.

191 Set `event` by select 'failed_operation' when reward $<$ 0, otherwise 'successful_operation'.

Source lines 192–195.

```
192         self.day += 1
193         terminated = self.day >= self.horizon
194         self._needs_reset = terminated
195         return (self._observation(), reward, terminated, False, {'event': event})
```

Line-by-line explanation.

192 Add `1` to `self.day`, and store the result back in that same target. The change is visible to subsequent statements in this block.

193 Set `terminated` by evaluate the comparison self.day >= self.horizon.

194 Set `self._needs_reset` by evaluate terminated.

195 Return `(self._observation(), reward, terminated, False, {'event': event})` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.19. Module declarations and integration

Source lines 198–205.

```
198 # ================================================================
199 # gym_maintenance.py
200 # ================================================================
203 import gymnasium as gym
204 import numpy as np
205 from gymnasium import spaces
```

Line-by-line explanation.

198 A visual section separator in the source. It has no executable effect.

199 Source comment: gym_maintenance.py This line documents the following code and does not execute.

200 A visual section separator in the source. It has no executable effect.

203 Import `import gymnasium as gym`. The imported names are used by the definitions below; importing does not start a training run.

204 Import `import numpy as np`. The imported names are used by the definitions below; importing does not start a training run.

205 Import `from gymnasium import spaces`. Gymnasium supplies the environment/wrapper contracts.

10.20. GymMaintenanceEnv

A Gymnasium adapter for the small example. It declares library-compatible spaces and reset/step signatures so a library trainer can interact with the teaching process.

Source lines 207–213.

```
207 class GymMaintenanceEnv(gym.Env):
208     """Same decisions and rewards as MiniMaintenanceEnv, with Gymnasium spaces.
210     The observation is [health / 3, days_left / horizon]. Normalization keeps
211     both input features between zero and one for a neural-network policy.
212     """
213     metadata = {'render_modes': ['ansi']}
```

Line-by-line explanation.

207 Define `GymMaintenanceEnv` with the stated inheritance. A Gymnasium adapter for the small example. It declares library-compatible spaces and reset/step signatures so a library trainer can interact with the teaching process.

208 Documentation text: Same decisions and rewards as MiniMaintenanceEnv, with Gymnasium spaces.. This states the contract or qualification for `GymMaintenanceEnv`; it does not update the simulator or model.

210 Documentation text: The observation is [health / 3, days_left / horizon]. Normalization keeps. This states the contract or qualification for `GymMaintenanceEnv`; it does not update the simulator or model.

211 Documentation text: both input features between zero and one for a neural-network policy.. This states the contract or qualification for `GymMaintenanceEnv`; it does not update the simulator or model.

212 Close the documentation string for `GymMaintenanceEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

213 Set `metadata` by construct the shown dict in its stated order.

10.21. GymMaintenanceEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 215–219.

```
215     def __init__(self, horizon: int=12, shock_prob: float=0.0):
216         super().__init__()
217         self.core = MiniMaintenanceEnv(horizon=horizon, shock_prob=shock_prob)
218         self.action_space = spaces.Discrete(2)
219         self.observation_space = spaces.Box(low=np.zeros(2, dtype=np.float32), high=np.ones(2,
        ↪  dtype=np.float32), dtype=np.float32)
```

Line-by-line explanation.

215 Define `__init__(self, horizon: int=12, shock_prob: float=0.0)`. This ___init___ method is part of GymMaintenanceEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

216 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

217 Set `self.core` by call MiniMaintenanceEnv with the shown positional and keyword arguments.

218 Set `self.action_space` by call spaces.Discrete with the shown positional and keyword arguments.

219 Set `self.observation_space` by call spaces.Box with the shown positional and keyword arguments.

10.22. GymMaintenanceEnv._encode

Convert the small example's observation into the declared Gymnasium observation encoding.

Source lines 221–223.

```
221     def _encode(self, observation):
222         health, days_left = observation
223         return np.array([health / 3.0, days_left / self.core.horizon], dtype=np.float32)
```

Line-by-line explanation.

221 Define `_encode(self, observation)`. Convert the small example's observation into the declared Gymnasium observation encoding.

222 Set `(health, days_left)` by evaluate observation.

223 Return `np.array([health / 3.0, days_left / self.core.horizon], dtype=np.float32)` to the caller by call np.array with the shown positional and keyword arguments. No further statements in this call execute.

10.23. GymMaintenanceEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 225–229.

```
225     def reset(self, *, seed=None, options=None):
226         super().reset(seed=seed)
227         initial_health = 3 if options is None else options.get('initial_health', 3)
228         observation, info = self.core.reset(seed=seed, initial_health=initial_health)
229         return (self._encode(observation), info)
```

Line-by-line explanation.

225 Define `reset(self, *, seed=None, options=None)`. This reset method is part of GymMaintenanceEnv.reset; its parameters and defaults are the interface used by the calls in this section.

226 Initialize a new episode using the stated seed/options. The exact receiver and arguments are shown on this source line.

227 Set `initial_health` by select 3 when options is None, otherwise options.get('initial_health', 3).

228 Set `(observation, info)` by initialize a new episode using the stated seed/options.

229 Return `(self._encode(observation), info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.24. GymMaintenanceEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 231–233.

```
231     def step(self, action):
232         observation, reward, terminated, truncated, info = self.core.step(int(action))
233         return (self._encode(observation), float(reward), terminated, truncated, info)
```

Line-by-line explanation.

231 Define `step(self, action)`. This step method is part of GymMaintenanceEnv.step; its parameters and defaults are the interface used by the calls in this section.

232 Set `(observation, reward, terminated, truncated, info)` by execute this object's step with the shown arguments.

233 Return `(self._encode(observation), float(reward), terminated, truncated, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.25. GymMaintenanceEnv.render

This method implements GymMaintenanceEnv.render in the class described above. Read its inputs, validity branches and returned objects together; persistent writes use the instance or simulator records.

Source lines 235–236.

```
235     def render(self):
236         return self.core.render_text()
```

Line-by-line explanation.

235 Define `render(self)`. This render method is part of GymMaintenanceEnv.render; its parameters and defaults are the interface used by the calls in this section.

236 Return `self.core.render_text()` to the caller by call self.core.render_text with the shown positional and keyword arguments. No further statements in this call execute.

10.26. GymMaintenanceEnv.close

This method implements GymMaintenanceEnv.close in the class described above. Read its inputs, validity branches and returned objects together; persistent writes use the instance or simulator records.

Source lines 238–239.

```
238     def close(self):
239         pass
```

Line-by-line explanation.

238 Define `close(self)`. This close method is part of GymMaintenanceEnv.close; its parameters and defaults are the interface used by the calls in this section.

239 Use an explicit empty statement; it performs no state or parameter update.

10.27. Module declarations and integration

Source lines 242–260.

```
242 # ======================================================================
243 # fleet_maintenance/fleet_env.py
244 # ======================================================================
247 from dataclasses import dataclass
248 import math
249 COMPONENTS = ('fan', 'LPC', 'HPC', 'combustor', 'HPT', 'LPT')
```

```
250 WEAR = np.array([0.002, 0.0025, 0.004, 0.002, 0.003, 0.0025])
251 LABOR_DAYS = np.array([2, 3, 4, 2, 3, 3])
252 PART_COST = np.array([8.0, 10.0, 16.0, 8.0, 12.0, 10.0])
253 EFFICIENCY_BASE = 0.72
254 EFFICIENCY_GAIN = 0.24
255 TEMPERATURE_BASE_C = np.array([40.0, 120.0, 350.0, 900.0, 650.0, 450.0])
256 TEMPERATURE_HEALTH_GAIN_C = 120.0
257 TEMPERATURE_EXPOSURE_GAIN_C = 30.0
258 TEMPERATURE_MIN_C = -50.0
259 TEMPERATURE_MAX_C = 1400.0
260 OBSERVATION_VERSION = 3
```

Line-by-line explanation.

242 A visual section separator in the source. It has no executable effect.

243 Source comment: fleet_maintenance/fleet_env.py This line documents the following code and does not execute.

244 A visual section separator in the source. It has no executable effect.

247 Import `from dataclasses import dataclass`. Dataclasses provide structured configuration and repair records.

248 Import `import math`. Scalar mathematics supports finite-value checks and rounded durations.

249 Set `COMPONENTS` by construct the shown tuple in its stated order.

250 Set `WEAR` by call np.array with the shown positional and keyword arguments.

251 Set `LABOR_DAYS` by call np.array with the shown positional and keyword arguments.

252 Set `PART_COST` by call np.array with the shown positional and keyword arguments.

253 Set `EFFICIENCY_BASE` by use the literal 0.72.

254 Set `EFFICIENCY_GAIN` by use the literal 0.24.

255 Set `TEMPERATURE_BASE_C` by call np.array with the shown positional and keyword arguments.

256 Set `TEMPERATURE_HEALTH_GAIN_C` by use the literal 120.0.

257 Set `TEMPERATURE_EXPOSURE_GAIN_C` by use the literal 30.0.

258 Set `TEMPERATURE_MIN_C` by evaluate -50.0.

259 Set `TEMPERATURE_MAX_C` by use the literal 1400.0.

260 Set `OBSERVATION_VERSION` by use the literal 3.

10.28. Config

Immutable simulation inputs and validation. Fleet size changes array sizes and model construction; changing it does not automatically resize an already trained checkpoint.

Source lines 262–279.

```
262 @dataclass(frozen=True)
263 class Config:
264     aircraft: int = 7
265     horizon: int = 90
266     crews: int = 3
```

```
    workshops: int = 2
    # Factors below one speed up work; empty profiles select synthetic defaults.
    crew_time_factors: tuple[float, ...] = ()
    workshop_time_factors: tuple[float, ...] = ()
    particles: int = 64
    setup_days: int = 7
    replenish_days: int = 14
    bird_probability: float = 0.002
    hail_probability: float = 0.003
    weather_probability: float = 0.03
    observation_noise: float = 0.025
    wear_scale: float = 1.0
    mandatory_health: float = 0.08
```

Line-by-line explanation.

262 Apply the decorator `dataclass(frozen=True)` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

263 Define `Config` with the stated inheritance. Immutable simulation inputs and validation. Fleet size changes array sizes and model construction; changing it does not automatically resize an already trained checkpoint.

264 Set `aircraft` by use the literal 7.

265 Set `horizon` by use the literal 90.

266 Set `crews` by use the literal 3.

267 Set `workshops` by use the literal 2.

268 Source comment: Factors below one speed up work; empty profiles select synthetic defaults. This line documents the following code and does not execute.

269 Set `crew_time_factors` by construct the shown tuple in its stated order.

270 Set `workshop_time_factors` by construct the shown tuple in its stated order.

271 Set `particles` by use the literal 64. It holds samples representing uncertain component health.

272 Set `setup_days` by use the literal 7.

273 Set `replenish_days` by use the literal 14.

274 Set `bird_probability` by use the literal 0.002.

275 Set `hail_probability` by use the literal 0.003.

276 Set `weather_probability` by use the literal 0.03.

277 Set `observation_noise` by use the literal 0.025.

278 Set `wear_scale` by use the literal 1.0.

279 Set `mandatory_health` by use the literal 0.08.

Source lines 280–280.

```
    initial_health: float = 0.88
```

Line-by-line explanation.

280 Set `initial_health` by use the literal 0.88.

10.29. Config.__post_init__

Validate configuration values after dataclass initialization, reject malformed resource profiles, and normalize accepted profiles to immutable tuples.

Source lines 282–299.

```
282     def __post_init__(self):
283         for k in ('aircraft', 'horizon', 'crews', 'workshops', 'particles', 'setup_days',
        ↪  'replenish_days'):
284             if not isinstance(getattr(self, k), int) or getattr(self, k) < 1:
285                 raise ValueError(f'{k} must be a positive integer')
286         for name, count in (('crew_time_factors', self.crews), ('workshop_time_factors',
        ↪  self.workshops)):
287             factors = getattr(self, name)
288             if not isinstance(factors, (tuple, list)):
289                 raise ValueError(f'{name} must be a tuple or list of duration factors')
290             if len(factors) and (len(factors) != count or any(not isinstance(x, (int, float)) or
            ↪  isinstance(x, bool) or not math.isfinite(x) or x <= 0 for x in factors)):
291                 raise ValueError(f'{name} must be empty or contain {count} finite positive numbers')
292             # Accepted-job resource profiles must remain immutable.
293             object.__setattr__(self, name, tuple(factors))
294         for k in ('bird_probability', 'hail_probability', 'weather_probability'):
295             if not 0 <= getattr(self, k) <= 1:
296                 raise ValueError(f'{k} must be in [0,1]')
297         if not 0 < self.observation_noise or not 0 < self.wear_scale:
298             raise ValueError('noise and wear_scale must be positive')
299         if not 0 <= self.mandatory_health < 1 or not 0 < self.initial_health <= 1:
```

Line-by-line explanation.

282 Define `__post_init__(self)`. Validate configuration values after dataclass initialization, reject malformed resource profiles, and normalize accepted profiles to immutable tuples.

283 Iterate `k` over `('aircraft', 'horizon', 'crews', 'workshops', 'particles', 'setup_days', 'replenish_days')`. The following indented statements run once for each selected element.

284 Test `not isinstance(getattr(self, k), int) or getattr(self, k) < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

285 Raise `ValueError(f'{k} must be a positive integer')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

286 Iterate `(name, count)` over `(('crew_time_factors', self.crews), ('workshop_time_factors', self.workshops))`. The following indented statements run once for each selected element.

287 Set `factors` by call getattr with the shown positional and keyword arguments.

288 Test `not isinstance(factors, (tuple, list))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

289 Raise `ValueError(f'{name} must be a tuple or list of duration factors')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

290 Test `len(factors) and (len(factors) != count or any((not isinstance(x, (int, float)) or isinstance(x, bool) or (not math.isfinite(x)) or (x <= 0) for x in factors)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

291 Raise `ValueError(f'{name} must be empty or contain {count} finite positive numbers')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

292 Source comment: Accepted-job resource profiles must remain immutable. This line documents the following code and does not execute.

293 Call object.___setattr___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

294 Iterate `k` over `('bird_probability', 'hail_probability', 'weather_probability')`. The following indented statements run once for each selected element.

295 Test `not 0 <= getattr(self, k) <= 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

296 Raise `ValueError(f'{k} must be in [0,1]')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

297 Test `not 0 < self.observation_noise or not 0 < self.wear_scale`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

298 Raise `ValueError('noise and wear_scale must be positive')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

299 Test `not 0 <= self.mandatory_health < 1 or not 0 < self.initial_health <= 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 300–300.

```
300             raise ValueError('invalid health bounds')
```

Line-by-line explanation.

300 Raise `ValueError('invalid health bounds')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

10.30. Plan

The decoded full-action request. It records aircraft, grounding, engine/component selections, crew identities, workshop, cyclic order and priority before acceptance.

Source lines 302–310.

```
302 @dataclass(frozen=True)
303 class Plan:
304     aircraft: int
305     engine_mask: int
306     bundles: tuple[int, int]
307     crew_mask: int
308     workshop: int
309     order_start: int
310     priority: int
```

Line-by-line explanation.

302 Apply the decorator `dataclass(frozen=True)` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

303 Define `Plan` with the stated inheritance. The decoded full-action request. It records aircraft, grounding, engine/component selections, crew identities, workshop, cyclic order and priority before acceptance.

304 Declare `aircraft` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

305 Declare `engine_mask` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

306 Declare `bundles` with type `tuple[int, int]`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

307 Declare `crew_mask` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

308 Declare `workshop` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

309 Declare `order_start` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

310 Declare `priority` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

10.31. Plan.crews

Count selected crew identities using the number of set bits in the crew-selection mask.

Source lines 312–314.

```
312     @property
313     def crews(self):
314         return self.crew_mask.bit_count()
```

Line-by-line explanation.

312 Apply the decorator `property` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

313 Define `crews(self)`. Count selected crew identities using the number of set bits in the crew-selection mask.

314 Return `self.crew_mask.bit_count()` to the caller by call self.crew_mask.bit_count with the shown positional and keyword arguments. No further statements in this call execute.

10.32. Job

The accepted, committed repair record. Remaining time, ordered tasks, component-completion offsets and assigned resources belong to the simulator after acceptance.

Source lines 316–322.

```
@dataclass
class Job:
    plan: Plan
    tasks: tuple[tuple[int, int], ...]
    remaining: int
    total_duration: int
    completion_days: tuple[int, ...]
```

Line-by-line explanation.

316 Apply the decorator `dataclass` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

317 Define `Job` with the stated inheritance. The accepted, committed repair record. Remaining time, ordered tasks, component-completion offsets and assigned resources belong to the simulator after acceptance.

318 Declare `plan` with type `Plan`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

319 Declare `tasks` with type `tuple[tuple[int, int], ...]`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

320 Declare `remaining` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

321 Declare `total_duration` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

322 Declare `completion_days` with type `tuple[int, ...]`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

10.33. FleetMaintenanceEnv

The physical fleet simulator. It owns true health, synthetic sensors, particle beliefs, inventory, heterogeneous resources, jobs, stable aircraft identities, event metrics and the daily clock. Policy networks read observations and cannot replace committed job definitions.

Source lines 324–339.

```
class FleetMaintenanceEnv(gym.Env):
    """Action row: ground, engines, bundle0, bundle1, crew_mask, shop, order, priority.
    Engine mask bits 0/1 select engines. Each bundle has six component bits.
    Crew bit i selects member CR-(i+1); workshop remains a single resource index.
    Crew work rates add; workshop factors multiply component work, not setup.
    Resource factors are synthetic teaching assumptions, not calibrated estimates.
    aircraft_plates[i] identifies aircraft i in every state array and action row.
    Plates are synthetic fleet-local identifiers, independent of operating status.
    Reset starts a new simulated history while retaining the index-to-plate mapping.
    Resources are assigned to complete plans, in selected priority order.
    Unfunded/conflicting proposals are rejected and logged; no partial repairs.
    A busy aircraft's row is ignored: the committed plan cannot be overwritten.
    Mandatory grounding uses observed lower health quantiles, not hidden health.
    """
    metadata = {'render_modes': ['ansi']}
```

Line-by-line explanation.

Define `FleetMaintenanceEnv` with the stated inheritance. The physical fleet simulator. It owns true health, synthetic sensors, particle beliefs, inventory, heterogeneous resources, jobs, stable aircraft identities, event metrics and the daily clock. Policy networks read observations and cannot replace committed job definitions.

Documentation text: Action row: ground, engines, bundle0, bundle1, crew_mask, shop, order, priority.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Engine mask bits 0/1 select engines. Each bundle has six component bits.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Crew bit i selects member CR-(i+1); workshop remains a single resource index.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Crew work rates add; workshop factors multiply component work, not setup.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Resource factors are synthetic teaching assumptions, not calibrated estimates.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: aircraft_plates[i] identifies aircraft i in every state array and action row.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Plates are synthetic fleet-local identifiers, independent of operating status.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Reset starts a new simulated history while retaining the index-to-plate mapping.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Resources are assigned to complete plans, in selected priority order.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Unfunded/conflicting proposals are rejected and logged; no partial repairs.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: A busy aircraft's row is ignored: the committed plan cannot be overwritten.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Documentation text: Mandatory grounding uses observed lower health quantiles, not hidden health.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

Close the documentation string for `FleetMaintenanceEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

Set `metadata` by construct the shown dict in its stated order.

10.34. FleetMaintenanceEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 341–356.

```
341     def __init__(self, config: Config | None=None, render_mode=None):
342         super().__init__()
343         self.config = config or Config()
344         self.render_mode = render_mode
345         n = self.config.aircraft
346         self.aircraft_plates = tuple(f'AC-{i + 1:03d}' for i in range(n))
347         self.crew_ids = tuple(f'CR-{i + 1:03d}' for i in range(self.config.crews))
348         self.workshop_ids = tuple(f'WS-{i + 1:03d}' for i in range(self.config.workshops))
349         self.crew_time_factors = tuple(self.config.crew_time_factors) or tuple((0.8, 1.0, 1.25)[i % 3]
            ↪  for i in range(self.config.crews))
350         self.workshop_time_factors = tuple(self.config.workshop_time_factors) or tuple((0.8, 1.0,
            ↪  1.2)[i % 3] for i in range(self.config.workshops))
351         self.row_dims = np.array([2, 4, 64, 64, 1 << self.config.crews, self.config.workshops, 12, n])
352         self.action_space = spaces.MultiDiscrete(np.tile(self.row_dims, (n, 1)))
353         self.local_size = 70
354         self.public_size = 10 + 2 * self.config.crews + 2 * self.config.workshops
355         self.observation_space = spaces.Box(0.0, 1.0, shape=(n * self.local_size + self.public_size,),
            ↪  dtype=np.float32)
356         self._done = True
```

Line-by-line explanation.

341 Define `__init__(self, config: Config | None=None, render_mode=None)`. This ___init___ method is part of FleetMaintenanceEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

342 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

343 Set `self.config` by evaluate config or Config().

344 Set `self.render_mode` by evaluate render_mode.

345 Set `n` by evaluate self.config.aircraft.

346 Set `self.aircraft_plates` by call tuple with the shown positional and keyword arguments.

347 Set `self.crew_ids` by call tuple with the shown positional and keyword arguments.

348 Set `self.workshop_ids` by call tuple with the shown positional and keyword arguments.

349 Set `self.crew_time_factors` by evaluate tuple(self.config.crew_time_factors) or tuple(((0.8, 1.0, 1.25)[i % 3] for i in range(self.config.crews))).

350 Set `self.workshop_time_factors` by evaluate tuple(self.config.workshop_time_factors) or tuple(((0.8, 1.0, 1.2)[i % 3] for i in range(self.config.workshops))).

351 Set `self.row_dims` by call np.array with the shown positional and keyword arguments.

352 Set `self.action_space` by call spaces.MultiDiscrete with the shown positional and keyword arguments.

353 Set `self.local_size` by use the literal 70.

354 Set `self.public_size` by add 10 + 2 * self.config.crews and 2 * self.config.workshops.

355 Set `self.observation_space` by call spaces.Box with the shown positional and keyword arguments.

356 Set `self._done` by use the literal True.

10.35. FleetMaintenanceEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 358–375.

```
358     def reset(self, *, seed=None, options=None):
359         super().reset(seed=seed)
360         cfg = self.config
361         # Physical events, belief updates, and exposure tapes use separate random streams.
362         seeds = np.random.SeedSequence(seed).spawn(3)
363         self.physical_rng = np.random.default_rng(seeds[0])
364         self.filter_rng = np.random.default_rng(seeds[1])
365         tape_rng = np.random.default_rng(seeds[2])
366         self.day = 0
367         self.health = np.clip(cfg.initial_health + self.physical_rng.normal(0, 0.025, (cfg.aircraft,
        ↪  2, 6)), 0.1, 1)
368         if options and 'health' in options:
369             health = np.asarray(options['health'], dtype=float)
370             if health.shape != self.health.shape or not np.isfinite(health).all() or np.any((health <
            ↪  0) | (health > 1)):
371                 raise ValueError('initial health must have shape (aircraft,2,6), in [0,1]')
372             self.health = health.copy()
373         self.exposure = tape_rng.uniform(0.75, 1.35, cfg.aircraft)
374         self.measurements = self._sensor_model(self.health, self.exposure[:, None, None])
375         self.measurements += self.physical_rng.normal(0, 1, self.measurements.shape) *
        ↪  self.sensor_noise
```

Line-by-line explanation.

358 Define `reset(self, *, seed=None, options=None)`. This reset method is part of FleetMaintenanceEnv.reset; its parameters and defaults are the interface used by the calls in this section.

359 Initialize a new episode using the stated seed/options. The exact receiver and arguments are shown on this source line.

360 Set `cfg` by evaluate self.config.

361 Source comment: Physical events, belief updates, and exposure tapes use separate random streams. This line documents the following code and does not execute.

362 Set `seeds` by call np.random.SeedSequence(seed).spawn with the shown positional and keyword arguments.

363 Set `self.physical_rng` by call np.random.default_rng with the shown positional and keyword arguments.

364 Set `self.filter_rng` by call np.random.default_rng with the shown positional and keyword arguments.

365 Set `tape_rng` by call np.random.default_rng with the shown positional and keyword arguments.

366 Set `self.day` by use the literal 0.

367 Set `self.health` by bound the numeric result elementwise. It holds hidden physical component condition.

368 Test `options and 'health' in options`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

369 Set `health` by call np.asarray with the shown positional and keyword arguments. It holds hidden physical component condition.

370 Reject an incorrect health shape, any nonfinite value, or any value outside [0,1]. The explicit finite check newly excludes NaN, which range comparisons previously missed.

371 Raise `ValueError('initial health must have shape (aircraft,2,6), in [0,1]')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

372 Set `self.health` by make a separate array/dictionary copy. It holds hidden physical component condition.

373 Set `self.exposure` by call tape_rng.uniform with the shown positional and keyword arguments.

374 Set `self.measurements` by call self._sensor_model with the shown positional and keyword arguments.

375 Add `self.physical_rng.normal(0, 1, self.measurements.shape) * self.sensor_noise` to `self.measurements`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Source lines 376–389.

```
        self._clip_measurements(self.measurements)
        self.last_flight_day = np.zeros(cfg.aircraft, dtype=int)
        prior = self._invert_sensor_metrics(self.measurements, self.exposure[:, None, None])
        self.particles = np.clip(prior[..., None] + self.filter_rng.normal(0, 0.04,
        ↪  (*self.health.shape, cfg.particles)), 0, 1)
        self.jobs: dict[int, Job] = {}
        self.stock = np.full(6, cfg.aircraft, dtype=int)
        self.event_tape = tape_rng.random((cfg.horizon, cfg.aircraft, 2, 2))
        self.severity_tape = tape_rng.uniform(0.1, 0.35, (cfg.horizon, cfg.aircraft, 2, 2))
        self.weather_tape = tape_rng.random(cfg.horizon)
        self.wear_tape = tape_rng.normal(1.0, 0.12, (cfg.horizon, cfg.aircraft, 2, 6))
        self.noise_tape = tape_rng.normal(0, cfg.observation_noise, (cfg.horizon, cfg.aircraft, 2, 6,
        ↪  2))
        self.metrics = {k: 0.0 for k in ('return', 'cost', 'flight_failures', 'flights',
        ↪  'downtime_aircraft_days', 'groundings', 'replacements', 'rejections', 'bird_events',
        ↪  'hail_events', 'weather_exposures', 'unsafe_attempts')}
        self._done = False
        return (self._observation(), {'day': self.day, 'aircraft_plates': self.aircraft_plates,
        ↪  'resources': self.resource_details()})
```

Line-by-line explanation.

376 Call self._clip_measurements with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

377 Set `self.last_flight_day` by call np.zeros with the shown positional and keyword arguments.

378 Set `prior` by call self._invert_sensor_metrics with the shown positional and keyword arguments.

379 Set `self.particles` by bound the numeric result elementwise. It holds samples representing uncertain component health.

380 Set `self.jobs` by construct the shown dict in its stated order. It holds accepted repair records held by the simulator.

381 Set `self.stock` by call np.full with the shown positional and keyword arguments. It holds spare inventory by component type.

382 Set `self.event_tape` by call tape_rng.random with the shown positional and keyword arguments.

383 Set `self.severity_tape` by call tape_rng.uniform with the shown positional and keyword arguments.

384 Set `self.weather_tape` by call tape_rng.random with the shown positional and keyword arguments.

385 Set `self.wear_tape` by call tape_rng.normal with the shown positional and keyword arguments.

386 Set `self.noise_tape` by call tape_rng.normal with the shown positional and keyword arguments.

387 Set `self.metrics` by construct the collection by iterating the shown comprehension and applying its filters.

388 Set `self._done` by use the literal False.

389 Return `(self._observation(), {'day': self.day, 'aircraft_plates': self.aircraft_plates, 'resources': self.resource_details()})` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.36. FleetMaintenanceEnv.split_observation

Separate the flattened observed vector into N local aircraft rows and the public resource/inventory vector. This is a view/layout operation, not access to true health.

Source lines 391–393.

```
391     def split_observation(self, observation):
392         a = np.asarray(observation)
393         return (a[:-self.public_size].reshape(self.config.aircraft, self.local_size),
            ↪  a[-self.public_size:])
```

Line-by-line explanation.

391 Define `split_observation(self, observation)`. Separate the flattened observed vector into N local aircraft rows and the public resource/inventory vector. This is a view/layout operation, not access to true health.

392 Set `a` by call np.asarray with the shown positional and keyword arguments.

393 Return `(a[:-self.public_size].reshape(self.config.aircraft, self.local_size), a[-self.public_size:])` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.37. FleetMaintenanceEnv.idle_action

Construct the default fleet action, preserving the required representation for aircraft already in committed repair.

Source lines 395–396.

```
395     def idle_action(self):
396         return np.zeros((self.config.aircraft, 8), dtype=np.int64)
```

Line-by-line explanation.

395 Define `idle_action(self)`. Construct the default fleet action, preserving the required representation for aircraft already in committed repair.

396 Return `np.zeros((self.config.aircraft, 8), dtype=np.int64)` to the caller by call np.zeros with the shown positional and keyword arguments. No further statements in this call execute.

10.38. FleetMaintenanceEnv.available_crew_mask

Set one bit for each currently free crew identity. This mask represents availability, while the selected crew mask specifies an assignment.

Source lines 398–402.

```
398     def available_crew_mask(self):
399         available = (1 << self.config.crews) - 1
400         for job in self.jobs.values():
401             available &= ~job.plan.crew_mask
402         return available
```

Line-by-line explanation.

398 Define `available_crew_mask(self)`. Set one bit for each currently free crew identity. This mask represents availability, while the selected crew mask specifies an assignment.

399 Set `available` by subtract 1 « self.config.crews and 1.

400 Iterate `job` over `self.jobs.values()`. The following indented statements run once for each selected element.

401 Bitwise-AND with `~job.plan.crew_mask` in `available`, and store the result back in that same target. The change is visible to subsequent statements in this block.

402 Return `available` to the caller by evaluate available. No further statements in this call execute.

10.39. FleetMaintenanceEnv.resource_state

Determine which identified resources are reserved by active jobs and which remain free.

Source lines 404–406.

```
404     def resource_state(self):
405         busy_shops = {j.plan.workshop for j in self.jobs.values()}
406         return (self.available_crew_mask().bit_count(), [s for s in range(self.config.workshops) if s
            ↪  not in busy_shops])
```

Line-by-line explanation.

404 Define `resource_state(self)`. Determine which identified resources are reserved by active jobs and which remain free.

405 Set `busy_shops` by evaluate {j.plan.workshop for j in self.jobs.values()}.

406 Return `(self.available_crew_mask().bit_count(), [s for s in range(self.config.workshops) if s not in busy_shops])` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.40. FleetMaintenanceEnv.resource_details

Expose identified crew/workshop availability and configured time factors for diagnostics and reporting.

Source lines 408–409.

```
408     def resource_details(self):
409         return {'crew_ids': self.crew_ids, 'crew_time_factors': self.crew_time_factors,
            ↪  'available_crew_mask': self.available_crew_mask(), 'workshop_ids': self.workshop_ids,
            ↪  'workshop_time_factors': self.workshop_time_factors, 'available_workshops':
            ↪  tuple(self.resource_state()[1])}
```

Line-by-line explanation.

408 Define `resource_details(self)`. Expose identified crew/workshop availability and configured time factors for diagnostics and reporting.

409 Return `{'crew_ids': self.crew_ids, 'crew_time_factors': self.crew_time_factors, 'available_crew_mask': self.available_crew_mask(), 'workshop_ids': self.workshop_ids, 'workshop_time_factors': self.workshop_time_factors, 'available_workshops': tuple(self.resource_state()[1])}` to the caller by construct the shown dict in its stated order. No further statements in this call execute.

10.41. FleetMaintenanceEnv.sensor_noise

Return the configured efficiency/temperature uncertainty scales used by the synthetic sensor likelihood.

Source lines 411–414.

```
411     @property
412     def sensor_noise(self):
413         """Independent sensor noise scales in efficiency fraction and Celsius."""
414         return np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C]) * self.config.observation_noise
```

Line-by-line explanation.

411 Apply the decorator `property` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

412 Define `sensor_noise(self)`. Return the configured efficiency/temperature uncertainty scales used by the synthetic sensor likelihood.

413 Documentation text: Independent sensor noise scales in efficiency fraction and Celsius.. This states the contract or qualification for `FleetMaintenanceEnv.sensor_noise`; it does not update the simulator or model.

414 Return `np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C]) * self.config.observation_noise` to the caller by multiply np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C]) and self.config.observation_noise. No further statements in this call execute.

10.42. FleetMaintenanceEnv._sensor_model

Map hidden normalized health and exposure to synthetic efficiency and temperature means. This is a teaching observation model rather than a turbofan performance map.

Source lines 416–421.

```
416     @staticmethod
417     def _sensor_model(health, exposure):
418         """Toy performance mapping; health decreases efficiency and raises heat."""
419         efficiency = EFFICIENCY_BASE + EFFICIENCY_GAIN * health
420         temperature = TEMPERATURE_BASE_C + TEMPERATURE_HEALTH_GAIN_C * (1 - health) +
            ↪  TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1)
421         return np.stack([efficiency, temperature], axis=-1)
```

Line-by-line explanation.

416 Apply the decorator `staticmethod` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

417 Define `_sensor_model(health, exposure)`. Map hidden normalized health and exposure to synthetic efficiency and temperature means. This is a teaching observation model rather than a turbofan performance map.

418 Documentation text: Toy performance mapping; health decreases efficiency and raises heat.. This states the contract or qualification for `FleetMaintenanceEnv._sensor_model`; it does not update the simulator or model.

419 Set `efficiency` by add EFFICIENCY_BASE and EFFICIENCY_GAIN * health.

420 Set `temperature` by add TEMPERATURE_BASE_C + TEMPERATURE_HEALTH_GAIN_C * (1 - health) and TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1).

421 Return `np.stack([efficiency, temperature], axis=-1)` to the caller by call np.stack with the shown positional and keyword arguments. No further statements in this call execute.

10.43. FleetMaintenanceEnv._clip_measurements

Bound synthetic sensor values to the accepted measurement ranges before encoding/likelihood use.

Source lines 423–426.

```
423     @staticmethod
424     def _clip_measurements(measurements):
425         np.clip(measurements[..., 0], 0, 1, out=measurements[..., 0])
426         np.clip(measurements[..., 1], TEMPERATURE_MIN_C, TEMPERATURE_MAX_C, out=measurements[..., 1])
```

Line-by-line explanation.

423 Apply the decorator `staticmethod` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

424 Define `_clip_measurements(measurements)`. Bound synthetic sensor values to the accepted measurement ranges before encoding/likelihood use.

425 Bound the numeric result elementwise. The exact receiver and arguments are shown on this source line.

426 Bound the numeric result elementwise. The exact receiver and arguments are shown on this source line.

10.44. FleetMaintenanceEnv._invert_sensor_metrics

Map observed synthetic performance back to an approximate health-scale estimate used in filter initialization or correction.

Source lines 428–433.

```
428     @staticmethod
429     def _invert_sensor_metrics(measurements, exposure):
430         """Initialization only: fuse two noisy inverse mappings with equal weight."""
431         from_efficiency = (measurements[..., 0] - EFFICIENCY_BASE) / EFFICIENCY_GAIN
432         from_temperature = 1 - (measurements[..., 1] - TEMPERATURE_BASE_C -
            ↪  TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1)) / TEMPERATURE_HEALTH_GAIN_C
433         return np.clip(0.5 * (from_efficiency + from_temperature), 0, 1)
```

Line-by-line explanation.

428 Apply the decorator `staticmethod` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

429 Define `_invert_sensor_metrics(measurements, exposure)`. Map observed synthetic performance back to an approximate health-scale estimate used in filter initialization or correction.

430 Documentation text: Initialization only: fuse two noisy inverse mappings with equal weight.. This states the contract or qualification for `FleetMaintenanceEnv._invert_sensor_metrics`; it does not update the simulator or model.

431 Set `from_efficiency` by divide measurements[..., 0] - EFFICIENCY_BASE and EFFICIENCY_GAIN.

Set `from_temperature` by subtract 1 and (measurements[..., 1] - TEMPERATURE_BASE_C - TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1)) / TEMPERATURE_HEALTH_GAIN_C.

Return `np.clip(0.5 * (from_efficiency + from_temperature), 0, 1)` to the caller by bound the numeric result elementwise. No further statements in this call execute.

10.45. FleetMaintenanceEnv.observation_metrics

Decode the observed synthetic performance fields for diagnostics without exposing hidden physical condition.

Source lines 435–438.

```
    def observation_metrics(self, observation):
        """Decode observed sensor metrics without exposing hidden health."""
        rows, _ = self.split_observation(observation)
        return {'efficiency': rows[:, 45:57].reshape(self.config.aircraft, 2, 6).copy(),
        ↪  'temperature_c': rows[:, 57:69].reshape(self.config.aircraft, 2, 6) * (TEMPERATURE_MAX_C -
        ↪  TEMPERATURE_MIN_C) + TEMPERATURE_MIN_C, 'reading_age_days': np.rint(rows[:, 69] *
        ↪  self.config.horizon).astype(int)}
```

Line-by-line explanation.

Define `observation_metrics(self, observation)`. Decode the observed synthetic performance fields for diagnostics without exposing hidden physical condition.

Documentation text: Decode observed sensor metrics without exposing hidden health.. This states the contract or qualification for `FleetMaintenanceEnv.observation_metrics`; it does not update the simulator or model.

Set `(rows, _)` by call self.split_observation with the shown positional and keyword arguments.

Return `{'efficiency': rows[:, 45:57].reshape(self.config.aircraft, 2, 6).copy(), 'temperature_c': rows[:, 57:69].reshape(self.config.aircraft, 2, 6) * (TEMPERATURE_MAX_C - TEMPERATURE_MIN_C) + TEMPERATURE_MIN_C, 'reading_age_days': np.rint(rows[:, 69] * self.config.horizon).astype(int)}` to the caller by construct the shown dict in its stated order. No further statements in this call execute.

10.46. FleetMaintenanceEnv._observation

Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

Source lines 440–454.

```
    def _observation(self):
        mean = self.particles.mean(-1)
        std = self.particles.std(-1)
        low = np.quantile(self.particles, 0.1, axis=-1)
        rows = []
        for a in range(self.config.aircraft):
            j = self.jobs.get(a)
```

```
447             committed = [j.plan.engine_mask / 3, j.plan.bundles[0] / 63, j.plan.bundles[1] / 63,
                ↪  j.plan.crew_mask / ((1 << self.config.crews) - 1), j.plan.workshop / max(1,
                ↪  self.config.workshops - 1), j.plan.order_start / 11] if j else [0.0] * 6
448             rows.append(np.concatenate([mean[a].ravel(), std[a].ravel(), low[a].ravel(),
                ↪  [self.exposure[a] / 1.5, float(j is not None), min(1, j.remaining / 60) if j else 0],
                ↪  committed, self.measurements[a, :, :, 0].ravel(), ((self.measurements[a, :, :, 1] -
                ↪  TEMPERATURE_MIN_C) / (TEMPERATURE_MAX_C - TEMPERATURE_MIN_C)).ravel(), [(self.day -
                ↪  self.last_flight_day[a]) / self.config.horizon]]))
449         free, shops = self.resource_state()
450         public = [self.day / self.config.horizon, free / self.config.crews, len(shops) /
            ↪  self.config.workshops, (self.config.replenish_days - self.day %
            ↪  self.config.replenish_days) / self.config.replenish_days, *np.clip(self.stock /
            ↪  self.config.aircraft, 0, 1)]
451         crew_mask = self.available_crew_mask()
452         public.extend([float(bool(crew_mask & (1 << i))) for i in range(self.config.crews)] + [1 / (1
            ↪  + factor) for factor in self.crew_time_factors])
453         public.extend([float(i in shops) for i in range(self.config.workshops)] + [1 / (1 + factor)
            ↪  for factor in self.workshop_time_factors])
454         return np.asarray(np.concatenate([np.asarray(rows).ravel(), public]), dtype=np.float32)
```

Line-by-line explanation.

440 Define `_observation(self)`. Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

441 Set `mean` by average the selected entries.

442 Set `std` by call self.particles.std with the shown positional and keyword arguments.

443 Set `low` by call np.quantile with the shown positional and keyword arguments.

444 Set `rows` by construct the shown list in its stated order.

445 Iterate `a` over `range(self.config.aircraft)`. The following indented statements run once for each selected element.

446 Set `j` by call self.jobs.get with the shown positional and keyword arguments.

447 Set `committed` by select [j.plan.engine_mask / 3, j.plan.bundles[0] / 63, j.plan.bundles[1] / 63, j.plan.crew_mask / ((1 « self.config.crews) - 1), j.plan.workshop / max(1, self.config.workshops - 1), j.plan.order_start / 11] when j, otherwise [0.0] * 6.

448 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

449 Set `(free, shops)` by call self.resource_state with the shown positional and keyword arguments.

450 Set `public` by construct the shown list in its stated order.

451 Set `crew_mask` by call self.available_crew_mask with the shown positional and keyword arguments. It holds the binary selection of individual crew identities.

452 Call public.extend with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

453 Call public.extend with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

454 Return `np.asarray(np.concatenate([np.asarray(rows).ravel(), public]), dtype=np.float32)` to the caller by call np.asarray with the shown positional and keyword arguments. No further statements in this call execute.

10.47. FleetMaintenanceEnv.validate_row

Decode and validate a single aircraft action row, including bitmask/category limits and the structural contract for a proposed plan.

Source lines 456–465.

```
456     def validate_row(self, aircraft, row):
457         ground, engines, b0, b1, crew_mask, shop, order, priority = map(int, row)
458         if not ground:
459             return None if engines == b0 == b1 == 0 else 'work_without_grounding'
460         actual = int(b0 != 0) + 2 * int(b1 != 0)
461         if not actual or engines != actual:
462             return 'inconsistent_engine_bundle'
463         if not 0 < crew_mask < (1 << self.config.crews):
464             return 'invalid_crew_mask'
465         return None
```

Line-by-line explanation.

456 Define `validate_row(self, aircraft, row)`. Decode and validate a single aircraft action row, including bitmask/category limits and the structural contract for a proposed plan.

457 Set `(ground, engines, b0, b1, crew_mask, shop, order, priority)` by call map with the shown positional and keyword arguments.

458 Test `not ground`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

459 Return `None if engines == b0 == b1 == 0 else 'work_without_grounding'` to the caller by select None when engines == b0 == b1 == 0, otherwise 'work_without_grounding'. No further statements in this call execute.

460 Set `actual` by add int(b0 != 0) and 2 * int(b1 != 0).

461 Test `not actual or engines != actual`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

462 Return `'inconsistent_engine_bundle'` to the caller by use the literal 'inconsistent_engine_bundle'. No further statements in this call execute.

463 Test `not 0 < crew_mask < 1 « self.config.crews`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

464 Return `'invalid_crew_mask'` to the caller by use the literal 'invalid_crew_mask'. No further statements in this call execute.

465 Return `None` to the caller by use the literal None. No further statements in this call execute.

10.48. FleetMaintenanceEnv._start_job

Check proposed work against aircraft/resource/stock availability; accept and reserve a committed job or record rejection. Store resource-derived task completion timing at acceptance.

Source lines 467–484.

```
467     def _start_job(self, aircraft, row):
468         _, engines, b0, b1, crew_mask, shop, order, priority = map(int, row)
469         plan = Plan(aircraft, engines, (b0, b1), crew_mask, shop, order, priority)
470         tasks = [(e, c) for e, b in enumerate((b0, b1)) for c in range(6) if b & 1 << c]
471         tasks.sort(key=lambda x: (x[0] * 6 + x[1] - order) % 12)
472         needed = np.bincount([c for _, c in tasks], minlength=6)
473         _, shops = self.resource_state()
474         if crew_mask & self.available_crew_mask() != crew_mask or shop not in shops:
475             return ('resource_conflict', 0.0)
476         if np.any(needed > self.stock):
477             return ('spare_shortage', 0.0)
478         work_rate = sum(1 / factor for i, factor in enumerate(self.crew_time_factors) if crew_mask &
        ↪  (1 << i))
479         duration = self.config.setup_days
480         completion_days = []
481         for _, component in tasks:
482             duration += max(1, math.ceil(LABOR_DAYS[component] * self.workshop_time_factors[shop] /
            ↪  work_rate))
483             completion_days.append(duration)
484         self.stock -= needed
```

Line-by-line explanation.

467 Define `_start_job(self, aircraft, row)`. Check proposed work against aircraft/resource/stock availability; accept and reserve a committed job or record rejection. Store resource-derived task completion timing at acceptance.

468 Set `(_, engines, b0, b1, crew_mask, shop, order, priority)` by call map with the shown positional and keyword arguments.

469 Set `plan` by call Plan with the shown positional and keyword arguments.

470 Set `tasks` by construct the collection by iterating the shown comprehension and applying its filters.

471 Call tasks.sort with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

472 Set `needed` by call np.bincount with the shown positional and keyword arguments.

473 Set `(_, shops)` by call self.resource_state with the shown positional and keyword arguments.

474 Test `crew_mask & self.available_crew_mask() != crew_mask or shop not in shops`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

475 Return `('resource_conflict', 0.0)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

476 Test `np.any(needed > self.stock)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

477 Return `('spare_shortage', 0.0)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

478 Set `work_rate` by add the terms of the supplied iterable.

479 Set `duration` by evaluate self.config.setup_days. It holds the number of primitive days in this decision.

480 Set `completion_days` by construct the shown list in its stated order.

481 Iterate `(_, component)` over `tasks`. The following indented statements run once for each selected element.

482 Add `max(1, math.ceil(LABOR_DAYS[component] * self.workshop_time_factors[shop] / work_rate))` to `duration`, and store the result back in that same target. This updates the number of primitive days in this decision.

483 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

484 Subtract `needed` from `self.stock`, and store the result back in that same target. This updates spare inventory by component type.

Source lines 485–488.

```
485         self.jobs[aircraft] = Job(plan, tuple(tasks), duration, duration, tuple(completion_days))
486         cost = 12.0 + engines.bit_count() * 5 + sum((PART_COST[c] + 3 * self.health[aircraft, e, c]
            ↪  for e, c in tasks))
487         self.metrics['groundings'] += 1
488         return (None, cost)
```

Line-by-line explanation.

485 Set `self.jobs[aircraft]` by call Job with the shown positional and keyword arguments. It holds accepted repair records held by the simulator.

486 Set `cost` by add 12.0 + engines.bit_count() * 5 and sum((PART_COST[c] + 3 * self.health[aircraft, e, c] for e, c in tasks)).

487 Add `1` to `self.metrics['groundings']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

488 Return `(None, cost)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.49. FleetMaintenanceEnv._update_filter

Predict component-health particles under the assumed wear/shock prior, score the synthetic measurements and resample. This is approximate independent marginal filtering, not an RUL forecast.

Source lines 490–508.

```
490     def _update_filter(self, aircraft, measurements):
491         """Bootstrap particle filter with noisy observations and shock-mixture prior.
493         The estimator does not receive true event type, severity or hidden health.
494         It is a deliberately simple independent-component filter, not Emanuele's.
495         """
496         cfg = self.config
497         p = self.particles[aircraft]
498         wear = WEAR[None, :, None] * self.exposure[aircraft] * cfg.wear_scale
499         p -= wear * self.filter_rng.normal(1, 0.12, p.shape)
500         bird = self.filter_rng.random((2, 1, cfg.particles)) < cfg.bird_probability
501         hail = self.filter_rng.random((2, 1, cfg.particles)) < cfg.hail_probability
502         bseverity = self.filter_rng.uniform(0.1, 0.35, (2, 1, cfg.particles))
503         hseverity = self.filter_rng.uniform(0.1, 0.35, (2, 1, cfg.particles))
504         p -= bird * bseverity * np.array([1, 0.5, 0.15, 0, 0, 0])[None, :, None]
505         p -= hail * hseverity * np.array([0.7, 0.4, 0.3, 0.1, 0.1, 0.1])[None, :, None]
506         p -= (self.filter_rng.random((1, 1, cfg.particles)) < cfg.weather_probability) * wear * 2
507         np.clip(p, 0, 1, out=p)
508         predicted_efficiency = EFFICIENCY_BASE + EFFICIENCY_GAIN * p
```

Line-by-line explanation.

490 Define `_update_filter(self, aircraft, measurements)`. Predict component-health particles under the assumed wear/shock prior, score the synthetic measurements and resample. This is approximate independent marginal filtering, not an RUL forecast.

491 Documentation text: Bootstrap particle filter with noisy observations and shock-mixture prior.. This states the contract or qualification for `FleetMaintenanceEnv._update_filter`; it does not update the simulator or model.

493 Documentation text: The estimator does not receive true event type, severity or hidden health.. This states the contract or qualification for `FleetMaintenanceEnv._update_filter`; it does not update the simulator or model.

494 Documentation text: It is a deliberately simple independent-component filter, not Emanuele's.. This states the contract or qualification for `FleetMaintenanceEnv._update_filter`; it does not update the simulator or model.

495 Close the documentation string for `FleetMaintenanceEnv._update_filter`. This delimiter ends the descriptive text and performs no simulator or learning update.

496 Set `cfg` by evaluate self.config.

497 Set `p` by select the indexed/sliced entries of self.particles.

498 Set `wear` by multiply WEAR[None, :, None] * self.exposure[aircraft] and cfg.wear_scale.

499 Subtract `wear * self.filter_rng.normal(1, 0.12, p.shape)` from `p`, and store the result back in that same target. The change is visible to subsequent statements in this block.

500 Set `bird` by evaluate the comparison self.filter_rng.random((2, 1, cfg.particles)) < cfg.bird_probability.

501 Set `hail` by evaluate the comparison self.filter_rng.random((2, 1, cfg.particles)) < cfg.hail_probability.

502 Set `bseverity` by call self.filter_rng.uniform with the shown positional and keyword arguments.

503 Set `hseverity` by call self.filter_rng.uniform with the shown positional and keyword arguments.

504 Subtract `bird * bseverity * np.array([1, 0.5, 0.15, 0, 0, 0])[None, :, None]` from `p`, and store the result back in that same target. The change is visible to subsequent statements in this block.

505 Subtract `hail * hseverity * np.array([0.7, 0.4, 0.3, 0.1, 0.1, 0.1])[None, :, None]` from `p`, and store the result back in that same target. The change is visible to subsequent statements in this block.

506 Subtract `(self.filter_rng.random((1, 1, cfg.particles)) < cfg.weather_probability) * wear * 2` from `p`, and store the result back in that same target. The change is visible to subsequent statements in this block.

507 Bound the numeric result elementwise. The exact receiver and arguments are shown on this source line.

508 Set `predicted_efficiency` by add EFFICIENCY_BASE and EFFICIENCY_GAIN * p.

Source lines 509–518.

```
509         predicted_temperature = TEMPERATURE_BASE_C[None, :, None] + TEMPERATURE_HEALTH_GAIN_C * (1 -
            ↪  p) + TEMPERATURE_EXPOSURE_GAIN_C * (self.exposure[aircraft] - 1)
510         efficiency_error = (predicted_efficiency - measurements[:, :, 0, None]) / self.sensor_noise[0]
511         temperature_error = (predicted_temperature - measurements[:, :, 1, None]) /
            ↪  self.sensor_noise[1]
512         logits = -0.5 * (efficiency_error ** 2 + temperature_error ** 2)
513         weights = np.exp(logits - logits.max(-1, keepdims=True))
514         weights /= weights.sum(-1, keepdims=True)
515         for e in range(2):
516             for c in range(6):
517                 idx = self.filter_rng.choice(cfg.particles, cfg.particles, p=weights[e, c])
518                 p[e, c] = np.clip(p[e, c, idx] + self.filter_rng.normal(0, 0.006, cfg.particles), 0, 1)
```

Line-by-line explanation.

509 Set `predicted_temperature` by add TEMPERATURE_BASE_C[None, :, None] + TEMPERATURE_HEALTH_GAIN_C * (1 - p) and TEMPERATURE_EXPOSURE_GAIN_C * (self.exposure[aircraft] - 1).

510 Set `efficiency_error` by divide predicted_efficiency - measurements[:, :, 0, None] and self.sensor_noise[0].

511 Set `temperature_error` by divide predicted_temperature - measurements[:, :, 1, None] and self.sensor_noise[1].

512 Set `logits` by multiply -0.5 and efficiency_error ** 2 + temperature_error ** 2.

513 Set `weights` by exponentiate log values, including log-probability ratios. It holds the detached capped importance weights for replay.

514 Divide by `weights.sum(-1, keepdims=True)` in `weights`, and store the result back in that same target. This updates the detached capped importance weights for replay.

515 Iterate `e` over `range(2)`. The following indented statements run once for each selected element.

516 Iterate `c` over `range(6)`. The following indented statements run once for each selected element.

517 Set `idx` by call self.filter_rng.choice with the shown positional and keyword arguments.

518 Set `p[e, c]` by bound the numeric result elementwise.

10.50. FleetMaintenanceEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 520–537.

```
520     def step(self, action):
521         if self._done:
522             raise RuntimeError('reset before step, including after episode completion')
523         a = np.asarray(action)
```

```
        if not np.issubdtype(a.dtype, np.integer) or not self.action_space.contains(a):
            raise ValueError('action outside MultiDiscrete space')
        cfg = self.config
        low = np.quantile(self.particles, 0.1, axis=-1)
        mandatory = low.min(axis=(1, 2)) <= cfg.mandatory_health
        costs = 0.0
        events = []
        rejected = []
        completed = []
        grounded = set(self.jobs)
        for aircraft in sorted(range(cfg.aircraft), key=lambda i: (-int(a[i, 7]), i)):
            if aircraft in self.jobs:
                continue
            reason = self.validate_row(aircraft, a[aircraft])
```

Line-by-line explanation.

520 Define `step(self, action)`. This step method is part of FleetMaintenanceEnv.step; its parameters and defaults are the interface used by the calls in this section.

521 Test `self._done`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

522 Raise `RuntimeError('reset before step, including after episode completion')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

523 Set `a` by call np.asarray with the shown positional and keyword arguments.

524 Test `not np.issubdtype(a.dtype, np.integer) or not self.action_space.contains(a)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

525 Raise `ValueError('action outside MultiDiscrete space')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

526 Set `cfg` by evaluate self.config.

527 Set `low` by call np.quantile with the shown positional and keyword arguments.

528 Set `mandatory` by evaluate the comparison low.min(axis=(1, 2)) <= cfg.mandatory_health.

529 Set `costs` by use the literal 0.0.

530 Set `events` by construct the shown list in its stated order.

531 Set `rejected` by construct the shown list in its stated order.

532 Set `completed` by construct the shown list in its stated order.

533 Set `grounded` by call set with the shown positional and keyword arguments.

534 Iterate `aircraft` over `sorted(range(cfg.aircraft), key=lambda i: (-int(a[i, 7]), i))`. The following indented statements run once for each selected element.

535 Test `aircraft in self.jobs`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

536 Skip the remainder of this loop iteration and begin the next iteration.

537 Set `reason` by call self.validate_row with the shown positional and keyword arguments.

Source lines 538–555.

```
            if reason is None and int(a[aircraft, 0]):
                reason, cost = self._start_job(aircraft, a[aircraft])
                costs += cost
            if reason:
                rejected.append((aircraft, reason))
                costs += 2.0
            if mandatory[aircraft] and aircraft not in self.jobs:
                grounded.add(aircraft)
                self.metrics['unsafe_attempts'] += float(not int(a[aircraft, 0]))
            if aircraft in self.jobs:
                grounded.add(aircraft)
        self.metrics['rejections'] += len(rejected)
        flights = 0
        failures = 0
        weather = self.weather_tape[self.day] < cfg.weather_probability
        for aircraft in range(cfg.aircraft):
            if aircraft in grounded:
                continue
```

Line-by-line explanation.

538 Test `reason is None and int(a[aircraft, 0])`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

539 Set `(reason, cost)` by call self._start_job with the shown positional and keyword arguments.

540 Add `cost` to `costs`, and store the result back in that same target. The change is visible to subsequent statements in this block.

541 Test `reason`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

542 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

543 Add `2.0` to `costs`, and store the result back in that same target. The change is visible to subsequent statements in this block.

544 Test `mandatory[aircraft] and aircraft not in self.jobs`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

545 Call grounded.add with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

546 Add `float(not int(a[aircraft, 0]))` to `self.metrics['unsafe_attempts']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

547 Test `aircraft in self.jobs`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

548 Call grounded.add with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

549 Add `len(rejected)` to `self.metrics['rejections']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

550 Set `flights` by use the literal 0.

551 Set `failures` by use the literal 0.

552 Set `weather` by evaluate the comparison self.weather_tape[self.day] < cfg.weather_probability.

553 Iterate `aircraft` over `range(cfg.aircraft)`. The following indented statements run once for each selected element.

554 Test `aircraft in grounded`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

555 Skip the remainder of this loop iteration and begin the next iteration.

Source lines 556–573.

```
556             flights += 1
557             self.health[aircraft] -= WEAR * self.exposure[aircraft] * cfg.wear_scale * np.maximum(0,
                ↪  self.wear_tape[self.day, aircraft]) * (3 if weather else 1)
558             if weather:
559                 self.metrics['weather_exposures'] += 1
560             for e in range(2):
561                 for kind, p, impact in [('bird', cfg.bird_probability, [1, 0.5, 0.15, 0, 0, 0]),
                    ↪  ('hail', cfg.hail_probability, [0.7, 0.4, 0.3, 0.1, 0.1, 0.1])]:
562                     k = int(kind == 'hail')
563                     if self.event_tape[self.day, aircraft, e, k] < p:
564                         severity = self.severity_tape[self.day, aircraft, e, k]
565                         self.health[aircraft, e] -= severity * np.array(impact)
566                         events.append({'aircraft': aircraft, 'aircraft_plate':
                            ↪  self.aircraft_plates[aircraft], 'engine': e, 'type': kind})
567                         self.metrics[kind + '_events'] += 1
568             np.clip(self.health[aircraft], 0, 1, out=self.health[aircraft])
569             failed = bool(np.any(self.health[aircraft] <= 0))
570             failures += failed
571             self.measurements[aircraft] = self._sensor_model(self.health[aircraft],
                ↪  self.exposure[aircraft])
572             self.measurements[aircraft] += self.noise_tape[self.day, aircraft] *
                ↪  np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C])
573             self._clip_measurements(self.measurements[aircraft])
```

Line-by-line explanation.

556 Add `1` to `flights`, and store the result back in that same target. The change is visible to subsequent statements in this block.

557 Subtract `WEAR * self.exposure[aircraft] * cfg.wear_scale * np.maximum(0, self.wear_tape[self.day, aircraft]) * (3 if weather else 1)` from `self.health[aircraft]`, and store the result back in that same target. This updates hidden physical component condition.

558 Test `weather`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

559 Add `1` to `self.metrics['weather_exposures']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

560 Iterate `e` over `range(2)`. The following indented statements run once for each selected element.

561 Iterate `(kind, p, impact)` over `[('bird', cfg.bird_probability, [1, 0.5, 0.15, 0, 0, 0]), ('hail', cfg.hail_probability, [0.7, 0.4, 0.3, 0.1, 0.1, 0.1])]`. The following indented statements run once for each selected element.

562 Set `k` by convert the result to a Python integer.

563 Test `self.event_tape[self.day, aircraft, e, k] < p`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

564 Set `severity` by select the indexed/sliced entries of self.severity_tape.

565 Subtract `severity * np.array(impact)` from `self.health[aircraft, e]`, and store the result back in that same target. This updates hidden physical component condition.

566 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

567 Add `1` to `self.metrics[kind + '_events']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

568 Bound the numeric result elementwise. The exact receiver and arguments are shown on this source line.

569 Set `failed` by convert the condition to a Python Boolean.

570 Add `failed` to `failures`, and store the result back in that same target. The change is visible to subsequent statements in this block.

571 Set `self.measurements[aircraft]` by call self._sensor_model with the shown positional and keyword arguments.

572 Add `self.noise_tape[self.day, aircraft] * np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C])` to `self.measurements[aircraft]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

573 Call self._clip_measurements with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

Source lines 574–591.

```
574             self.last_flight_day[aircraft] = self.day + 1
575             self._update_filter(aircraft, self.measurements[aircraft])
576         for aircraft, j in list(self.jobs.items()):
577             elapsed = j.total_duration - j.remaining + 1
578             for (e, c), task_day in zip(j.tasks, j.completion_days):
579                 if elapsed == task_day:
580                     self.health[aircraft, e, c] = 1.0
581                     self.particles[aircraft, e, c] = 1.0
582                     self.metrics['replacements'] += 1
583             j.remaining -= 1
584             if j.remaining == 0:
585                 completed.append(aircraft)
586                 del self.jobs[aircraft]
587         downtime = len(grounded)
588         costs += downtime * 2 + failures * 80
589         reward = (flights * 10 - costs) / (cfg.aircraft * 10)
590         self.metrics['return'] += reward
591         self.metrics['cost'] += costs
```

Line-by-line explanation.

574 Set `self.last_flight_day[aircraft]` by add self.day and 1.

575 Call self._update_filter with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

576 Iterate `(aircraft, j)` over `list(self.jobs.items())`. The following indented statements run once for each selected element.

577 Set `elapsed` by add j.total_duration - j.remaining and 1.

578 Iterate `((e, c), task_day)` over `zip(j.tasks, j.completion_days)`. The following indented statements run once for each selected element.

579 Test `elapsed == task_day`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

580 Set `self.health[aircraft, e, c]` by use the literal 1.0. It holds hidden physical component condition.

581 Set `self.particles[aircraft, e, c]` by use the literal 1.0. It holds samples representing uncertain component health.

582 Add `1` to `self.metrics['replacements']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

583 Subtract `1` from `j.remaining`, and store the result back in that same target. The change is visible to subsequent statements in this block.

584 Test `j.remaining == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

585 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

586 Execute the Python fragment `del self.jobs[aircraft]` in `FleetMaintenanceEnv.step`. Its indentation places it in the surrounding control block.

587 Set `downtime` by count entries.

588 Add `downtime * 2 + failures * 80` to `costs`, and store the result back in that same target. The change is visible to subsequent statements in this block.

589 Set `reward` by divide flights * 10 - costs and cfg.aircraft * 10.

590 Add `reward` to `self.metrics['return']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

591 Add `costs` to `self.metrics['cost']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Source lines 592–600.

```
592         self.metrics['flights'] += flights
593         self.metrics['flight_failures'] += failures
594         self.metrics['downtime_aircraft_days'] += downtime
595         self.day += 1
596         if self.day % cfg.replenish_days == 0:
597             self.stock = np.minimum(cfg.aircraft, self.stock + max(1, cfg.aircraft // 2))
598         self._done = self.day >= cfg.horizon
599         info = {'day': self.day, 'aircraft_plates': self.aircraft_plates, 'resources':
            ↪  self.resource_details(), 'events': events, 'rejected': rejected, 'completed': completed,
            ↪  'reward_terms': {'flight_revenue': flights * 10, 'cost': costs}, 'metrics':
            ↪  dict(self.metrics)}
600         return (self._observation(), float(reward), self._done, False, info)
```

Line-by-line explanation.

592 Add `flights` to `self.metrics['flights']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

593 Add `failures` to `self.metrics['flight_failures']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

594 Add `downtime` to `self.metrics['downtime_aircraft_days']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

595 Add `1` to `self.day`, and store the result back in that same target. The change is visible to subsequent statements in this block.

596 Test `self.day % cfg.replenish_days == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

597 Set `self.stock` by call np.minimum with the shown positional and keyword arguments. It holds spare inventory by component type.

598 Set `self._done` by evaluate the comparison self.day >= cfg.horizon.

599 Set `info` by construct the shown dict in its stated order.

600 Return `(self._observation(), float(reward), self._done, False, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.51. FleetMaintenanceEnv.render

This method implements FleetMaintenanceEnv.render in the class described above. Read its inputs, validity branches and returned objects together; persistent writes use the instance or simulator records.

Source lines 602–603.

```
602     def render(self):
603         return f'day={self.day} aircraft_plates={self.aircraft_plates} busy={[self.aircraft_plates[i]
        ↪  for i in sorted(self.jobs)]} crews/shops={self.resource_state()}
        ↪  stock={self.stock.tolist()}'
```

Line-by-line explanation.

602 Define `render(self)`. This render method is part of FleetMaintenanceEnv.render; its parameters and defaults are the interface used by the calls in this section.

603 Return `f'day={self.day} aircraft_plates={self.aircraft_plates} busy={[self.aircraft_plates[i] for i in sorted(self.jobs)]} crews/shops={self.resource_state()} stock={self.stock.tolist()}'` to the caller by evaluate f'day={self.day} aircraft_plates={self.aircraft_plates} busy={[self.aircraft_plates[i] for i in sorted(self.jobs)]} crews/shops={self.resource_state()} stock={self.stock.tolist()}'. No further statements in this call execute.

10.52. threshold_action

A fixed threshold controller for the original full-action interface. It selects observed-condition-driven repairs and feasible resources; it contains no trained neural parameters.

Source lines 605–622.

```
605 def threshold_action(env, observation, threshold=0.65, bundle_threshold=0.75):
606     """Feasible rule using only observations, not hidden health or event diagnostics."""
607     rows, public = env.split_observation(observation)
608     action = env.idle_action()
609     free_mask = sum(1 << i for i in range(env.config.crews) if public[10 + i] > 0.5)
610     shop_offset = 10 + 2 * env.config.crews
611     shops = sorted((i for i in range(env.config.workshops) if public[shop_offset + i] > 0.5),
        ↪  key=lambda i: env.workshop_time_factors[i])
```

```
    stock = np.rint(public[4:10] * env.config.aircraft).astype(int)
    scores = rows[:, 24:36].min(1)
    rank = np.argsort(scores)
    for priority, aircraft in enumerate(rank[::-1]):
        action[aircraft, 7] = priority
    for aircraft in rank:
        if rows[aircraft, 37] > 0.5 or scores[aircraft] > threshold or not free_mask or (not shops):
            continue
        health = rows[aircraft, :12].reshape(2, 6)
        selected = health <= bundle_threshold
        if not selected.any():
```

Line-by-line explanation.

605 Define `threshold_action(env, observation, threshold=0.65, bundle_threshold=0.75)`. A fixed threshold controller for the original full-action interface. It selects observed-condition-driven repairs and feasible resources; it contains no trained neural parameters.

606 Documentation text: Feasible rule using only observations, not hidden health or event diagnostics.. This states the contract or qualification for `threshold_action`; it does not update the simulator or model.

607 Set `(rows, public)` by call env.split_observation with the shown positional and keyword arguments.

608 Set `action` by call env.idle_action with the shown positional and keyword arguments. It holds the sampled or stored action whose probability is evaluated.

609 Set `free_mask` by add the terms of the supplied iterable.

610 Set `shop_offset` by add 10 and 2 * env.config.crews.

611 Set `shops` by call sorted with the shown positional and keyword arguments.

612 Set `stock` by call np.rint(public[4:10] * env.config.aircraft).astype with the shown positional and keyword arguments. It holds spare inventory by component type.

613 Set `scores` by call rows[:, 24:36].min with the shown positional and keyword arguments.

614 Set `rank` by call np.argsort with the shown positional and keyword arguments.

615 Iterate `(priority, aircraft)` over `enumerate(rank[::-1])`. The following indented statements run once for each selected element.

616 Set `action[aircraft, 7]` by evaluate priority. It holds the sampled or stored action whose probability is evaluated.

617 Iterate `aircraft` over `rank`. The following indented statements run once for each selected element.

618 Test `rows[aircraft, 37] > 0.5 or scores[aircraft] > threshold or (not free_mask) or (not shops)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

619 Skip the remainder of this loop iteration and begin the next iteration.

620 Set `health` by reinterpret the shape while retaining element order. It holds hidden physical component condition.

621 Set `selected` by evaluate the comparison health <= bundle_threshold.

622 Test `not selected.any()`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 623–636.

```
            selected.flat[rows[aircraft, 24:36].argmin()] = True
        bundles = [0, 0]
        for e in range(2):
            for c in range(6):
                if selected[e, c] and stock[c] > 0:
                    bundles[e] |= 1 << c
                    stock[c] -= 1
        mask = int(bundles[0] != 0) + 2 * int(bundles[1] != 0)
        if not mask:
            continue
        crew = min((i for i in range(env.config.crews) if free_mask & (1 << i)), key=lambda i:
        ↪  env.crew_time_factors[i])
        action[aircraft] = [1, mask, *bundles, 1 << crew, shops.pop(0), int(health.argmin()),
        ↪  action[aircraft, 7]]
        free_mask &= ~(1 << crew)
    return action
```

Line-by-line explanation.

623 Set `selected.flat[rows[aircraft, 24:36].argmin()]` by use the literal True.

624 Set `bundles` by construct the shown list in its stated order.

625 Iterate `e` over `range(2)`. The following indented statements run once for each selected element.

626 Iterate `c` over `range(6)`. The following indented statements run once for each selected element.

627 Test `selected[e, c] and stock[c] > 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

628 Bitwise-OR in `1 << c` in `bundles[e]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

629 Subtract `1` from `stock[c]`, and store the result back in that same target. This updates spare inventory by component type.

630 Set `mask` by add int(bundles[0] != 0) and 2 * int(bundles[1] != 0).

631 Test `not mask`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

632 Skip the remainder of this loop iteration and begin the next iteration.

633 Set `crew` by call min with the shown positional and keyword arguments.

634 Set `action[aircraft]` by construct the shown list in its stated order. It holds the sampled or stored action whose probability is evaluated.

635 Bitwise-AND with `~(1 << crew)` in `free_mask`, and store the result back in that same target. The change is visible to subsequent statements in this block.

636 Return `action` to the caller by evaluate action. No further statements in this call execute.

10.53. Module declarations and integration

Source lines 639–641.

```
# ======================================================================
# fleet_maintenance/adapters.py
# ======================================================================
```

Line-by-line explanation.

639 A visual section separator in the source. It has no executable effect.

640 Source comment: fleet_maintenance/adapters.py This line documents the following code and does not execute.

641 A visual section separator in the source. It has no executable effect.

10.54. FlattenFleetAction

A reversible action-shape adapter. It exposes the fleet matrix as a single MultiDiscrete vector for library interfaces; it is not a separate control algorithm.

Source lines 644–645.

```
644 class FlattenFleetAction(gym.ActionWrapper):
645     """Stable-Baselines3 expects a one-dimensional MultiDiscrete nvec."""
```

Line-by-line explanation.

644 Define `FlattenFleetAction` with the stated inheritance. A reversible action-shape adapter. It exposes the fleet matrix as a single MultiDiscrete vector for library interfaces; it is not a separate control algorithm.

645 Documentation text: Stable-Baselines3 expects a one-dimensional MultiDiscrete nvec.. This states the contract or qualification for `FlattenFleetAction`; it does not update the simulator or model.

10.55. FlattenFleetAction.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 647–650.

```
647     def __init__(self, env):
648         super().__init__(env)
649         self.action_shape = env.action_space.nvec.shape
650         self.action_space = spaces.MultiDiscrete(env.action_space.nvec.ravel())
```

Line-by-line explanation.

647 Define `__init__(self, env)`. This ___init___ method is part of FlattenFleetAction.___init___; its parameters and defaults are the interface used by the calls in this section.

648 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

649 Set `self.action_shape` by evaluate env.action_space.nvec.shape.

650 Set `self.action_space` by call spaces.MultiDiscrete with the shown positional and keyword arguments.

10.56. FlattenFleetAction.action

Reshape a flattened library action into the simulator's aircraft-by-field matrix.

Source lines 652–653.

```
652     def action(self, action):
653         return np.asarray(action, dtype=np.int64).reshape(self.action_shape)
```

Line-by-line explanation.

652 Define `action(self, action)`. Reshape a flattened library action into the simulator's aircraft-by-field matrix.

653 Return `np.asarray(action, dtype=np.int64).reshape(self.action_shape)` to the caller by reinterpret the shape while retaining element order. No further statements in this call execute.

10.57. FlattenFleetAction.reverse_action

Flatten a simulator fleet action back into the wrapper's one-dimensional representation.

Source lines 655–656.

```
655     def reverse_action(self, action):
656         return np.asarray(action, dtype=np.int64).ravel()
```

Line-by-line explanation.

655 Define `reverse_action(self, action)`. Flatten a simulator fleet action back into the wrapper's one-dimensional representation.

656 Return `np.asarray(action, dtype=np.int64).ravel()` to the caller by call np.asarray(action, dtype=np.int64).ravel with the shown positional and keyword arguments. No further statements in this call execute.

10.58. Module declarations and integration

Source lines 659–666.

```
659 # ======================================================================
660 # fleet_maintenance/policies.py
661 # ======================================================================
664 import torch
665 from torch import nn
666 from torch.distributions import Categorical
```

Line-by-line explanation.

659 A visual section separator in the source. It has no executable effect.

660 Source comment: fleet_maintenance/policies.py This line documents the following code and does not execute.

661 A visual section separator in the source. It has no executable effect.

664 Import `import torch`. Torch supplies tensors, automatic differentiation and model persistence.

665 Import `from torch import nn`. Torch supplies tensors, automatic differentiation and model persistence.

666 Import `from torch.distributions import Categorical`. Torch supplies tensors, automatic differentiation and model persistence.

10.59. _fleet_observation_tensor

Convert to float32, batch a single observation and enforce the declared global width without allocating per-aircraft actor context.

Source lines 668–675.

```
668 def _fleet_observation_tensor(observation, width, error_message):
669     """Validate global inputs without allocating per-aircraft actor features."""
670     obs = torch.as_tensor(observation, dtype=torch.float32)
671     if obs.ndim == 1:
672         obs = obs.unsqueeze(0)
673     if obs.ndim != 2 or obs.shape[1] != width:
674         raise ValueError(error_message)
675     return obs
```

Line-by-line explanation.

668 Validate and batch the global observation through the shared helper. This preserves the expected width and caller-specific error while avoiding actor-context allocation.

669 Documentation text: Validate global inputs without allocating per-aircraft actor features.. This states the contract or qualification for `_fleet_observation_tensor`; it does not update the simulator or model.

670 Set `obs` by convert inputs to a tensor with the requested dtype. It holds the observed belief vector, rather than hidden physical health.

671 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

672 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health.

673 Test `obs.ndim != 2 or obs.shape[1] != width`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

674 Raise `ValueError(error_message)` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

675 Return `obs` to the caller by evaluate obs. No further statements in this call execute.

10.60. FleetActorCritic

The full-action policy family. One actor is reused across aircraft; optional global encoding and previous-field conditioning distinguish flat, conditional and shared modes. A scalar global critic and a fleet-joint likelihood support the original PPO trainer.

Source lines 678–688.

```
678 class FleetActorCritic(nn.Module):
679     """Central critic + flat, conditional, or parameter-shared aircraft actors.
680     shared: local aircraft estimates + public resource counts. Parameters reused
681     across aircraft; a central deterministic resolver enforces resource capacity.
682     conditional: global actor context and masked dependent action distributions.
683     flat: shared global features with no learned conditioning on preceding choices.
684     All methods use the same logical masks; scarce-resource conflicts are resolved
685     by the simulator and reported separately from structural feasibility.
686     All act daily and obey pre-start commitment; this is not temporal HRL.
687     """
688
```

Line-by-line explanation.

678 Define `FleetActorCritic` with the stated inheritance. The full-action policy family. One actor is reused across aircraft; optional global encoding and previous-field conditioning distinguish flat, conditional and shared modes. A scalar global critic and a fleet-joint likelihood support the original PPO trainer.

679 Documentation text: Central critic + flat, conditional, or parameter-shared aircraft actors.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

681 Documentation text: shared: local aircraft estimates + public resource counts. Parameters reused. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

682 Documentation text: across aircraft; a central deterministic resolver enforces resource capacity.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

683 Documentation text: conditional: global actor context and masked dependent action distributions.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

684 Documentation text: flat: shared global features with no learned conditioning on preceding choices.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

685 Documentation text: All methods use the same logical masks; scarce-resource conflicts are resolved. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

686 Documentation text: by the simulator and reported separately from structural feasibility.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

687 Documentation text: All act daily and obey pre-start commitment; this is not temporal HRL.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

688 Close the documentation string for `FleetActorCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.61. FleetActorCritic.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 690–707.

```
690     def __init__(self, env: FleetMaintenanceEnv, mode='conditional', goal_size=0):
691         super().__init__()
692         if mode not in ('flat', 'conditional', 'shared'):
693             raise ValueError(mode)
694         self.mode = mode
695         self.n = env.config.aircraft
696         self.local = env.local_size
697         self.public = env.public_size
698         self.dims = env.row_dims.tolist()
699         self.mandatory = env.config.mandatory_health
700         self.goal_size = goal_size
701         size = env.observation_space.shape[0]
702         # Keep legacy registered layers/state keys even when this mode does not execute them.
703         self.global_encoder = nn.Sequential(nn.Linear(size, 64), nn.Tanh())
704         in_size = self.local + self.public + (0 if mode == 'shared' else 64) + goal_size
705         self.actor = nn.Sequential(nn.Linear(in_size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh())
706         self.heads = nn.ModuleList((nn.Linear(64, d) for d in self.dims))
707         self.condition = nn.ModuleList((nn.Linear(8, 64, bias=False) for _ in self.dims))
```

Line-by-line explanation.

690 Define `__init__(self, env: FleetMaintenanceEnv, mode='conditional', goal_size=0)`. This ___init___ method is part of FleetActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

691 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

692 Test `mode not in ('flat', 'conditional', 'shared')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

693 Raise `ValueError(mode)` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

694 Set `self.mode` by evaluate mode.

695 Set `self.n` by evaluate env.config.aircraft.

696 Set `self.local` by evaluate env.local_size.

697 Set `self.public` by evaluate env.public_size.

698 Set `self.dims` by call env.row_dims.tolist with the shown positional and keyword arguments.

699 Set `self.mandatory` by evaluate env.config.mandatory_health.

700 Set `self.goal_size` by evaluate goal_size.

701 Set `size` by select the indexed/sliced entries of env.observation_space.shape.

702 Source comment: Keep legacy registered layers/state keys even when this mode does not execute them. This line documents the following code and does not execute.

703 Set `self.global_encoder` by compose the listed layers in execution order.

704 Set `in_size` by add self.local + self.public + (0 if mode == 'shared' else 64) and goal_size.

705 Set `self.actor` by compose the listed layers in execution order.

706 Set `self.heads` by register the listed independent networks so parameters and checkpoints include them.

707 Set `self.condition` by register the listed independent networks so parameters and checkpoints include them.

Source lines 708–708.

```
708         self.critic = nn.Sequential(nn.Linear(size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh(),
            ↪  nn.Linear(64, 1))
```

Line-by-line explanation.

708 Set `self.critic` by compose the listed layers in execution order.

10.62. FleetActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 710–711.

```
710     def value(self, obs):
711         return self.critic(obs).squeeze(-1)
```

Line-by-line explanation.

710 Define `value(self, obs)`. This value method is part of FleetActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

711 Return `self.critic(obs).squeeze(-1)` to the caller by call self.critic(obs).squeeze with the shown positional and keyword arguments. No further statements in this call execute.

10.63. FleetActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 713–730.

```
713     def act(self, obs, actions=None, deterministic=False, goal=None):
714         if obs.ndim == 1:
715             obs = obs.unsqueeze(0)
716         batch = obs.shape[0]
717         local = obs[:, :-self.public].reshape(batch, self.n, self.local)
718         public = obs[:, -self.public:].unsqueeze(1).expand(-1, self.n, -1)
719         features = [local, public]
720         if self.goal_size:
721             if goal is None:
722                 raise ValueError('goal-conditioned worker requires a goal')
723             features.append(goal.unsqueeze(1).expand(-1, self.n, -1))
724         if self.mode != 'shared':
725             features.append(self.global_encoder(obs).unsqueeze(1).expand(-1, self.n, -1))
726         h = self.actor(torch.cat(features, -1))
727         previous = None if self.mode == 'flat' else torch.zeros(batch, self.n, 8, device=obs.device)
728         logprob = torch.zeros(batch, device=obs.device)
729         entropy = torch.zeros_like(logprob)
730         result = []
```

Line-by-line explanation.

713 Define `act(self, obs, actions=None, deterministic=False, goal=None)`. This act method is part of FleetActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health.

Set `batch` by select the indexed/sliced entries of obs.shape.

Set `local` by reinterpret the shape while retaining element order.

Set `public` by call obs[:, -self.public:].unsqueeze(1).expand with the shown positional and keyword arguments.

Set `features` by construct the shown list in its stated order.

Test `self.goal_size`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Test `goal is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Raise `ValueError('goal-conditioned worker requires a goal')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

Test `self.mode != 'shared'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

Set `h` by call self.actor with the shown positional and keyword arguments.

Allocate previous-choice conditioning only for modes that use it. Flat logits do not read this history; later feasibility masks still apply.

Set `logprob` by call torch.zeros with the shown positional and keyword arguments.

Set `entropy` by call torch.zeros_like with the shown positional and keyword arguments.

Set `result` by construct the shown list in its stated order.

Source lines 731–748.

```
        busy = local[:, :, 37] > 0.5
        if self.goal_size:
            busy = busy | (goal[:, 0] > 0.5).unsqueeze(-1)
        urgent = local[:, :, 24:36].amin(-1) <= self.mandatory
        for k, head in enumerate(self.heads):
            logits = head(h if self.mode == 'flat' else torch.tanh(h + self.condition[k](previous)))
            valid = torch.ones_like(logits, dtype=torch.bool)
            valid[busy] = False
            valid[:, :, 0] |= busy
            if k == 0:
                valid[:, :, 0] &= ~(urgent & ~busy)
            else:
                ground = result[0] == 1
                if k == 1:
                    valid[:, :, 0] &= ~ground
                if k in (2, 3):
                    engine_on = result[1] & 1 << k - 2 != 0
                    active = ground & engine_on & ~busy
```

Line-by-line explanation.

731 Set `busy` by evaluate the comparison local[:, :, 37] > 0.5.

732 Test `self.goal_size`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

733 Set `busy` by apply a bitwise union to busy and (goal[:, 0] > 0.5).unsqueeze(-1).

734 Set `urgent` by evaluate the comparison local[:, :, 24:36].amin(-1) <= self.mandatory.

735 Iterate `(k, head)` over `enumerate(self.heads)`. The following indented statements run once for each selected element.

736 Set `logits` by call head with the shown positional and keyword arguments.

737 Set `valid` by call torch.ones_like with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

738 Set `valid[busy]` by use the literal False. It holds the categorical structural-validity mask.

739 Bitwise-OR in `busy` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

740 Test `k == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

741 Bitwise-AND with `~(urgent & ~busy)` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

742 Execute this alternative only when the corresponding preceding condition did not select its branch.

743 Set `ground` by evaluate the comparison result[0] == 1.

744 Test `k == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

745 Bitwise-AND with `~ground` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

746 Test `k in (2, 3)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

747 Set `engine_on` by evaluate the comparison result[1] & 1 « k - 2 != 0.

748 Set `active` by apply a bitwise intersection to ground & engine_on and ~busy.

Source lines 749–766.

```
                valid[:, :, 0] &= ~active
                valid[:, :, 1:] &= active.unsqueeze(-1)
            elif k >= 4:
                valid[:, :, 1:] &= (ground & ~busy).unsqueeze(-1)
                if k == 4:
                    valid[:, :, 0] &= ~(ground & ~busy)
            if k == 1:
                valid[:, :, 1:] &= (ground & ~busy).unsqueeze(-1)
        dist = Categorical(logits=logits.masked_fill(~valid, -1000000000.0))
        choice = actions[:, :, k] if actions is not None else dist.probs.argmax(-1) if
        ↪  deterministic else dist.sample()
        result.append(choice)
        logprob += dist.log_prob(choice).sum(-1)
```

```
            entropy += dist.entropy().sum(-1)
            if self.mode != 'flat' and k < len(self.heads) - 1:
                # Earlier conditioning layers save their inputs for backward; do not mutate in place.
                addition = torch.zeros_like(previous)
                addition[:, :, k] = choice / float(max(1, self.dims[k] - 1))
                previous = previous + addition
```

Line-by-line explanation.

749 Bitwise-AND with `~active` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

750 Bitwise-AND with `active.unsqueeze(-1)` in `valid[:, :, 1:]`, and store the result back in that same target. This updates the categorical structural-validity mask.

751 Test `k >= 4`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

752 Bitwise-AND with `(ground & ~busy).unsqueeze(-1)` in `valid[:, :, 1:]`, and store the result back in that same target. This updates the categorical structural-validity mask.

753 Test `k == 4`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

754 Bitwise-AND with `~(ground & ~busy)` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

755 Test `k == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

756 Bitwise-AND with `(ground & ~busy).unsqueeze(-1)` in `valid[:, :, 1:]`, and store the result back in that same target. This updates the categorical structural-validity mask.

757 Set `dist` by construct a discrete distribution from the supplied logits/probabilities.

758 Set `choice` by select actions[:, :, k] when actions is not None, otherwise dist.probs.argmax(-1) if deterministic else dist.sample().

759 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

760 Add `dist.log_prob(choice).sum(-1)` to `logprob`, and store the result back in that same target. The change is visible to subsequent statements in this block.

761 Add `dist.entropy().sum(-1)` to `entropy`, and store the result back in that same target. The change is visible to subsequent statements in this block.

762 Update history only when the policy conditions on previous choices and another field remains. The last update would have no consumer.

763 Source comment: Earlier conditioning layers save their inputs for backward; do not mutate in place. This line documents the following code and does not execute.

764 Set `addition` by call torch.zeros_like with the shown positional and keyword arguments.

765 Set `addition[:, :, k]` by divide choice and float(max(1, self.dims[k] - 1)).

766 Set `previous` by add previous and addition.

Source lines 767–767.

```
767         return (torch.stack(result, -1), logprob, entropy, self.value(obs))
```

Line-by-line explanation.

767 Return `(torch.stack(result, -1), logprob, entropy, self.value(obs))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.64. OptionActorCritic

The original fleet-level learned manager. Its four actions invoke fixed threshold routines; the manager chooses an option rather than individual replacement/resource fields.

Source lines 769–770.

```
769 class OptionActorCritic(nn.Module):
770     """Learned manager selects one of four fixed fleet planning options."""
```

Line-by-line explanation.

769 Define `OptionActorCritic` with the stated inheritance. The original fleet-level learned manager. Its four actions invoke fixed threshold routines; the manager chooses an option rather than individual replacement/resource fields.

770 Documentation text: Learned manager selects one of four fixed fleet planning options.. This states the contract or qualification for `OptionActorCritic`; it does not update the simulator or model.

10.65. OptionActorCritic.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 772–777.

```
772     def __init__(self, env):
773         super().__init__()
774         size = env.observation_space.shape[0]
775         self.encoder = nn.Sequential(nn.Linear(size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh())
776         self.actor = nn.Linear(64, 4)
777         self.critic = nn.Linear(64, 1)
```

Line-by-line explanation.

772 Define `__init__(self, env)`. This ___init___ method is part of OptionActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

773 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

774 Set `size` by select the indexed/sliced entries of env.observation_space.shape.

775 Set `self.encoder` by compose the listed layers in execution order.

776 Set `self.actor` by create a trainable affine layer with the shown input/output widths.

777 Set `self.critic` by create a trainable affine layer with the shown input/output widths.

10.66. OptionActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 779–780.

```
779     def value(self, obs):
780         return self.critic(self.encoder(obs)).squeeze(-1)
```

Line-by-line explanation.

779 Define `value(self, obs)`. This value method is part of OptionActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

780 Return `self.critic(self.encoder(obs)).squeeze(-1)` to the caller by call self.critic(self.encoder(obs)).squeeze with the shown positional and keyword arguments. No further statements in this call execute.

10.67. OptionActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 782–788.

```
782     def act(self, obs, actions=None, deterministic=False):
783         if obs.ndim == 1:
784             obs = obs.unsqueeze(0)
785         h = self.encoder(obs)
786         dist = Categorical(logits=self.actor(h))
787         action = actions if actions is not None else dist.probs.argmax(-1) if deterministic else
        ↪  dist.sample()
788         return (action, dist.log_prob(action), dist.entropy(), self.critic(h).squeeze(-1))
```

Line-by-line explanation.

782 Define `act(self, obs, actions=None, deterministic=False)`. This act method is part of OptionActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

783 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

784 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health.

785 Set `h` by call self.encoder with the shown positional and keyword arguments.

786 Set `dist` by construct a discrete distribution from the supplied logits/probabilities.

787 Set `action` by select actions when actions is not None, otherwise dist.probs.argmax(-1) if deterministic else dist.sample(). It holds the sampled or stored action whose probability is evaluated.

788 Return `(action, dist.log_prob(action), dist.entropy(), self.critic(h).squeeze(-1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.68. LearnedOptionActorCritic

The original learned fleet manager plus option-conditioned full-action worker. A sampled goal is passed to the worker and their log probabilities are added. The manager value supplies the training baseline.

Source lines 790–797.

```
790 class LearnedOptionActorCritic(nn.Module):
791     """Jointly trained manager + goal-conditioned plan worker + fixed executor.
793     Macro 0 waits; macros 1..3 are latent planning contexts, not certified skills.
794     The worker selects engines, components and resources once before execution.
795     Both levels learn from fleet return. This is not MAXQ or option-critic and
796     option collapse/credit assignment require later ablations and diagnostics.
797     """
```

Line-by-line explanation.

790 Define `LearnedOptionActorCritic` with the stated inheritance. The original learned fleet manager plus option-conditioned full-action worker. A sampled goal is passed to the worker and their log probabilities are added. The manager value supplies the training baseline.

791 Documentation text: Jointly trained manager + goal-conditioned plan worker + fixed executor.. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

793 Documentation text: Macro 0 waits; macros 1..3 are latent planning contexts, not certified skills.. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

794 Documentation text: The worker selects engines, components and resources once before execution.. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

795 Documentation text: Both levels learn from fleet return. This is not MAXQ or option-critic and. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

796 Documentation text: option collapse/credit assignment require later ablations and diagnostics.. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

797 Close the documentation string for `LearnedOptionActorCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.69. LearnedOptionActorCritic.__init__

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 799–803.

```
799     def __init__(self, env):
800         super().__init__()
801         self.manager = OptionActorCritic(env)
802         self.worker = FleetActorCritic(env, 'conditional', goal_size=4)
803         self.n = env.config.aircraft
```

Line-by-line explanation.

799 Define `__init__(self, env)`. This ___init___ method is part of LearnedOptionActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

800 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

801 Set `self.manager` by call OptionActorCritic with the shown positional and keyword arguments.

802 Set `self.worker` by call FleetActorCritic with the shown positional and keyword arguments.

803 Set `self.n` by evaluate env.config.aircraft.

10.70. LearnedOptionActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 805–806.

```
805     def value(self, obs):
806         return self.manager.value(obs)
```

Line-by-line explanation.

805 Define `value(self, obs)`. This value method is part of LearnedOptionActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

806 Return `self.manager.value(obs)` to the caller by call self.manager.value with the shown positional and keyword arguments. No further statements in this call execute.

10.71. LearnedOptionActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 808–817.

```
808     def act(self, obs, actions=None, deterministic=False):
809         if obs.ndim == 1:
810             obs = obs.unsqueeze(0)
811         macro = None if actions is None else actions[:, 0]
812         plan = None if actions is None else actions[:, 1:].reshape(-1, self.n, 8)
```

```
        option, lp, entropy, value = self.manager.act(obs, macro, deterministic)
        goal = torch.nn.functional.one_hot(option, num_classes=4).float()
        plan, worker_lp, worker_entropy, _ = self.worker.act(obs, plan, deterministic, goal)
        composite = torch.cat([option.unsqueeze(-1), plan.flatten(1)], -1)
        return (composite, lp + worker_lp, entropy + worker_entropy, value)
```

Line-by-line explanation.

808 Define `act(self, obs, actions=None, deterministic=False)`. This act method is part of LearnedOptionActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

809 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

810 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health.

811 Set `macro` by select None when actions is None, otherwise actions[:, 0].

812 Set `plan` by select None when actions is None, otherwise actions[:, 1:].reshape(-1, self.n, 8).

813 Set `(option, lp, entropy, value)` by call self.manager.act with the shown positional and keyword arguments.

814 Set `goal` by call torch.nn.functional.one_hot(option, num_classes=4).float with the shown positional and keyword arguments.

815 Set `(plan, worker_lp, worker_entropy, _)` by call self.worker.act with the shown positional and keyword arguments.

816 Set `composite` by concatenate features along the specified existing axis.

817 Return `(composite, lp + worker_lp, entropy + worker_entropy, value)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.72. Module declarations and integration

Source lines 818–819.

```
# Multi-agent maintenance: explicit proposal ownership and a common resource broker.
MAPPO_METHODS = ('mappo_aircraft', 'mappo_component')
```

Line-by-line explanation.

818 Source comment: Multi-agent maintenance: explicit proposal ownership and a common resource broker. This line documents the following code and does not execute.

819 Set `MAPPO_METHODS` by construct the shown tuple in its stated order.

10.73. mappo_agent_inputs

Construct exactly the actor information views used by the two MAPPO assignments. Aircraft views contain one local row plus public state; component views collect their component-specific features across the fleet.

Source lines 822–839.

```
822 def mappo_agent_inputs(observation, aircraft, local_size, public_size, assignment):
823     """Build restricted actor inputs from observed beliefs, never hidden health."""
824     obs = _fleet_observation_tensor(observation, aircraft * local_size + public_size,
825                                     'incorrect fleet observation shape')
826     local = obs[:, :aircraft * local_size].reshape(-1, aircraft, local_size)
827     public = obs[:, -public_size:]
828     if assignment == 'aircraft':
829         # Each aircraft sees only its own row plus the common resource board.
830         return torch.cat([local, public.unsqueeze(1).expand(-1, aircraft, -1)], dim=-1)
831     if assignment != 'component':
832         raise ValueError('assignment must be aircraft or component')
833     # Ten own-component values: two engines x mean/std/lower/efficiency/temperature.
834     indices = [[c, c + 6, c + 12, c + 18, c + 24, c + 30,
835                 c + 45, c + 51, c + 57, c + 63, 36, 37, 38, 69] for c in range(6)]
836     own = torch.stack([local[:, :, ix] for ix in indices], dim=1)
837     # A broadcast safety-criticality report is an explicit cross-component message.
838     criticality = local[:, :, 24:36].amin(-1)
839     board = criticality[:, None, :, None].expand(-1, 6, -1, -1)
```

Line-by-line explanation.

822 Define `mappo_agent_inputs(observation, aircraft, local_size, public_size, assignment)`. Construct exactly the actor information views used by the two MAPPO assignments. Aircraft views contain one local row plus public state; component views collect their component-specific features across the fleet.

823 Documentation text: Build restricted actor inputs from observed beliefs, never hidden health.. This states the contract or qualification for `mappo_agent_inputs`; it does not update the simulator or model.

824 Validate and batch the global observation through the shared helper. This preserves the expected width and caller-specific error while avoiding actor-context allocation.

825 Continue the assignment begun at line 824 with `'incorrect fleet observation shape')`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

826 Set `local` by reinterpret the shape while retaining element order.

827 Set `public` by select the indexed/sliced entries of obs.

828 Test `assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

829 Source comment: Each aircraft sees only its own row plus the common resource board. This line documents the following code and does not execute.

830 Return `torch.cat([local, public.unsqueeze(1).expand(-1, aircraft, -1)], dim=-1)` to the caller by concatenate features along the specified existing axis. No further statements in this call execute.

831 Test `assignment != 'component'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

832 Raise `ValueError('assignment must be aircraft or component')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

833 Source comment: Ten own-component values: two engines x mean/std/lower/efficiency/temperature. This line documents the following code and does not execute.

834 Set `indices` by construct the collection by iterating the shown comprehension and applying its filters.

835 Continue the assignment begun at line 834 with `c + 45, c + 51, c + 57, c + 63, 36, 37, 38, 69] for c in range(6)]`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

836 Set `own` by join equal-shaped tensors along the specified new axis.

837 Source comment: A broadcast safety-criticality report is an explicit cross-component message. This line documents the following code and does not execute.

838 Set `criticality` by call local[:, :, 24:36].amin with the shown positional and keyword arguments.

839 Set `board` by call criticality[:, None, :, None].expand with the shown positional and keyword arguments.

Source lines 840–841.

```
840     features = torch.cat([own, board], dim=-1).flatten(2)
841     return torch.cat([features, public.unsqueeze(1).expand(-1, 6, -1)], dim=-1)
```

Line-by-line explanation.

840 Set `features` by call torch.cat([own, board], dim=-1).flatten with the shown positional and keyword arguments.

841 Return `torch.cat([features, public.unsqueeze(1).expand(-1, 6, -1)], dim=-1)` to the caller by concatenate features along the specified existing axis. No further statements in this call execute.

10.74. MultiAgentFleetEnv

A request adapter with aircraft or component-type ownership. It converts agent proposals into full fleet actions through a deterministic stock/resource broker. Shared team reward does not remove the broker or create learned communication.

Source lines 844–854.

```
844 class MultiAgentFleetEnv:
845     """Cooperative dictionary API around the same physical fleet simulator.
847     Aircraft agents propose two six-bit bundles; six component agents each
848     propose a two-engine mask for every aircraft. Both cover the same twelve
849     replacement bits per aircraft. They do not learn crew/shop/order choices.
850     A public request board and deterministic centralized dispatcher coordinate
851     execution. This is decentralized proposal generation with centralized
852     arbitration, not fully decentralized resource allocation or learned messaging.
853     This small adapter is not a PettingZoo/RLlib API implementation.
854     """
```

Line-by-line explanation.

844 Define `MultiAgentFleetEnv` with the stated inheritance. A request adapter with aircraft or component-type ownership. It converts agent proposals into full fleet actions through a deterministic stock/resource broker. Shared team reward does not remove the broker or create learned communication.

845 Documentation text: Cooperative dictionary API around the same physical fleet simulator.. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

847 Documentation text: Aircraft agents propose two six-bit bundles; six component agents each. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

848 Documentation text: propose a two-engine mask for every aircraft. Both cover the same twelve. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

849 Documentation text: replacement bits per aircraft. They do not learn crew/shop/order choices.. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

850 Documentation text: A public request board and deterministic centralized dispatcher coordinate. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

851 Documentation text: execution. This is decentralized proposal generation with centralized. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

852 Documentation text: arbitration, not fully decentralized resource allocation or learned messaging.. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

853 Documentation text: This small adapter is not a PettingZoo/RLlib API implementation.. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

854 Close the documentation string for `MultiAgentFleetEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.75. MultiAgentFleetEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 856–868.

```
856     def __init__(self, config=None, assignment='aircraft'):
857         if assignment not in ('aircraft', 'component'):
858             raise ValueError('assignment must be aircraft or component')
859         self.env = FleetMaintenanceEnv(config)
860         self.assignment = assignment
861         self.agent_ids = self.env.aircraft_plates if assignment == 'aircraft' else COMPONENTS
862         self.num_agents = len(self.agent_ids)
863         dims = [64, 64] if assignment == 'aircraft' else [4] * self.env.config.aircraft
864         self.action_spaces = {agent: spaces.MultiDiscrete(dims) for agent in self.agent_ids}
865         width = self.env.local_size + self.env.public_size if assignment == 'aircraft' else 15 *
            ↪  self.env.config.aircraft + self.env.public_size
866         self.observation_spaces = {agent: spaces.Box(0, 1, shape=(width,), dtype=np.float32) for agent
            ↪  in self.agent_ids}
867         self.central_observation = None
868         self.coordination_deferrals = 0
```

Line-by-line explanation.

856 Define `__init__(self, config=None, assignment='aircraft')`. This ___init___ method is part of MultiAgentFleetEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

857 Test `assignment not in ('aircraft', 'component')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

858 Raise `ValueError('assignment must be aircraft or component')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

859 Set `self.env` by call FleetMaintenanceEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

860 Set `self.assignment` by evaluate assignment.

861 Set `self.agent_ids` by select self.env.aircraft_plates when assignment == 'aircraft', otherwise COMPONENTS.

862 Set `self.num_agents` by count entries.

863 Set `dims` by select [64, 64] when assignment == 'aircraft', otherwise [4] * self.env.config.aircraft.

864 Set `self.action_spaces` by construct the collection by iterating the shown comprehension and applying its filters.

865 Set `width` by select self.env.local_size + self.env.public_size when assignment == 'aircraft', otherwise 15 * self.env.config.aircraft + self.env.public_size.

866 Set `self.observation_spaces` by construct the collection by iterating the shown comprehension and applying its filters.

867 Set `self.central_observation` by use the literal None.

868 Set `self.coordination_deferrals` by use the literal 0.

10.76. MultiAgentFleetEnv.observations

Build the agent-keyed observation dictionary using the restricted-view information contract for the selected ownership.

Source lines 870–873.

```
870     def observations(self):
871         views = mappo_agent_inputs(self.central_observation, self.env.config.aircraft,
872                                    self.env.local_size, self.env.public_size,
                                       ↪  self.assignment)[0].numpy()
873         return {agent: views[i].copy() for i, agent in enumerate(self.agent_ids)}
```

Line-by-line explanation.

870 Define `observations(self)`. Build the agent-keyed observation dictionary using the restricted-view information contract for the selected ownership.

871 Set `views` by expose this CPU tensor as a NumPy array.

872 Continue the assignment begun at line 871 with `self.env.local_size, self.env.public_size, self.assignment)[0].numpy()`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

873 Return `{agent: views[i].copy() for i, agent in enumerate(self.agent_ids)}` to the caller by construct the collection by iterating the shown comprehension and applying its filters. No further statements in this call execute.

10.77. MultiAgentFleetEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 875–879.

```
875     def reset(self, *, seed=None, options=None):
876         self.central_observation, info = self.env.reset(seed=seed, options=options)
877         self.coordination_deferrals = 0
878         info.update(agent_ids=self.agent_ids, central_observation=self.central_observation.copy())
879         return self.observations(), info
```

Line-by-line explanation.

875 Define `reset(self, *, seed=None, options=None)`. This reset method is part of MultiAgentFleetEnv.reset; its parameters and defaults are the interface used by the calls in this section.

876 Set `(self.central_observation, info)` by initialize a new episode using the stated seed/options.

877 Set `self.coordination_deferrals` by use the literal 0.

878 Call info.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

879 Return `(self.observations(), info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.78. MultiAgentFleetEnv.compose_action

Resolve agent replacement requests through the fixed broker, preserve active plans, prioritize urgency and allocate feasible stock, crew and workshop selections.

Source lines 881–898.

```
881     def compose_action(self, actions):
882         """Merge owned requests and allocate resources using observed data only."""
883         if self.central_observation is None:
884             raise RuntimeError('reset before composing agent proposals')
885         if not isinstance(actions, dict) or set(actions) != set(self.agent_ids):
886             raise ValueError('supply exactly one proposal for each agent ID')
887         for agent in self.agent_ids:
888             proposal = np.asarray(actions[agent])
889             if not np.issubdtype(proposal.dtype, np.integer) or not
                ↪ self.action_spaces[agent].contains(proposal):
890                 raise ValueError(f'invalid proposal for {agent}')
891         raw = np.stack([actions[agent] for agent in self.agent_ids])
892         if self.assignment == 'aircraft':
893             bundles = raw.copy()
894         else:
895             # Specialist c can set only component bit c, independently per engine.
896             bundles = np.zeros((self.env.config.aircraft, 2), dtype=np.int64)
897             for c in range(6):
898                 for engine in range(2):
```

Line-by-line explanation.

881 Define `compose_action(self, actions)`. Resolve agent replacement requests through the fixed broker, preserve active plans, prioritize urgency and allocate feasible stock, crew and workshop selections.

882 Documentation text: Merge owned requests and allocate resources using observed data only.. This states the contract or qualification for `MultiAgentFleetEnv.compose_action`; it does not update the simulator or model.

883 Test `self.central_observation is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

884 Raise `RuntimeError('reset before composing agent proposals')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

885 Test `not isinstance(actions, dict) or set(actions) != set(self.agent_ids)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

886 Raise `ValueError('supply exactly one proposal for each agent ID')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

887 Iterate `agent` over `self.agent_ids`. The following indented statements run once for each selected element.

888 Set `proposal` by call np.asarray with the shown positional and keyword arguments.

889 Test `not np.issubdtype(proposal.dtype, np.integer) or not self.action_spaces[agent].contains(proposal)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

890 Raise `ValueError(f'invalid proposal for {agent}')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

891 Set `raw` by call np.stack with the shown positional and keyword arguments.

892 Test `self.assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

893 Set `bundles` by make a separate array/dictionary copy.

894 Execute this alternative only when the corresponding preceding condition did not select its branch.

895 Source comment: Specialist c can set only component bit c, independently per engine. This line documents the following code and does not execute.

896 Set `bundles` by call np.zeros with the shown positional and keyword arguments.

897 Iterate `c` over `range(6)`. The following indented statements run once for each selected element.

898 Iterate `engine` over `range(2)`. The following indented statements run once for each selected element.

Source lines 899–916.

```
                bundles[:, engine] |= ((raw[c] >> engine) & 1) << c
        rows, public = self.env.split_observation(self.central_observation)
        free_mask = sum(1 << i for i in range(self.env.config.crews) if public[10 + i] > 0.5)
        offset = 10 + 2 * self.env.config.crews
        shops = [i for i in range(self.env.config.workshops) if public[offset + i] > 0.5]
        stock = np.rint(public[4:10] * self.env.config.aircraft).astype(int)
        scores = rows[:, 24:36].min(1)
        rank = np.argsort(scores, kind='stable')
        fleet_action = self.env.idle_action()
        deferred = []
        for position, aircraft in enumerate(rank):
            # The broker collects per-aircraft criticality reports; it is not a critic.
            fleet_action[aircraft, 7] = self.env.config.aircraft - position - 1
            if rows[aircraft, 37] > 0.5 or not bundles[aircraft].any():
                continue
            b0, b1 = map(int, bundles[aircraft])
            tasks = [(e, c) for e, b in enumerate((b0, b1)) for c in range(6) if b & (1 << c)]
            needed = np.bincount([c for _, c in tasks], minlength=6)
```

Line-by-line explanation.

899 Bitwise-OR in `(raw[c] » engine & 1) « c` in `bundles[:, engine]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

900 Set `(rows, public)` by call self.env.split_observation with the shown positional and keyword arguments.

901 Set `free_mask` by add the terms of the supplied iterable.

902 Set `offset` by add 10 and 2 * self.env.config.crews.

903 Set `shops` by construct the collection by iterating the shown comprehension and applying its filters.

904 Set `stock` by call np.rint(public[4:10] * self.env.config.aircraft).astype with the shown positional and keyword arguments. It holds spare inventory by component type.

905 Set `scores` by call rows[:, 24:36].min with the shown positional and keyword arguments.

906 Set `rank` by call np.argsort with the shown positional and keyword arguments.

907 Set `fleet_action` by call self.env.idle_action with the shown positional and keyword arguments.

908 Set `deferred` by construct the shown list in its stated order.

909 Iterate `(position, aircraft)` over `enumerate(rank)`. The following indented statements run once for each selected element.

910 Source comment: The broker collects per-aircraft criticality reports; it is not a critic. This line documents the following code and does not execute.

911 Set `fleet_action[aircraft, 7]` by subtract self.env.config.aircraft - position and 1.

912 Test `rows[aircraft, 37] > 0.5 or not bundles[aircraft].any()`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

913 Skip the remainder of this loop iteration and begin the next iteration.

914 Set `(b0, b1)` by call map with the shown positional and keyword arguments.

915 Set `tasks` by construct the collection by iterating the shown comprehension and applying its filters.

916 Set `needed` by call np.bincount with the shown positional and keyword arguments.

Source lines 917–931.

```
            if not free_mask or not shops:
                deferred.append((int(aircraft), 'resource_conflict'))
                continue
            if np.any(needed > stock):
                deferred.append((int(aircraft), 'spare_shortage'))
                continue
            crew = min((i for i in range(self.env.config.crews) if free_mask & (1 << i)), key=lambda
            ↪  i: self.env.crew_time_factors[i])
            shop = min(shops, key=lambda i: self.env.workshop_time_factors[i])
            first = min(tasks, key=lambda pair: rows[aircraft, pair[0] * 6 + pair[1]])
            engines = int(b0 != 0) + 2 * int(b1 != 0)
            fleet_action[aircraft, :7] = [1, engines, b0, b1, 1 << crew, shop, first[0] * 6 + first[1]]
            free_mask &= ~(1 << crew)
            shops.remove(shop)
            stock -= needed
        return fleet_action, deferred
```

Line-by-line explanation.

917 Test `not free_mask or not shops`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

918 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

919 Skip the remainder of this loop iteration and begin the next iteration.

920 Test `np.any(needed > stock)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

921 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

922 Skip the remainder of this loop iteration and begin the next iteration.

923 Set `crew` by call min with the shown positional and keyword arguments.

924 Set `shop` by call min with the shown positional and keyword arguments.

925 Set `first` by call min with the shown positional and keyword arguments.

926 Set `engines` by add int(b0 != 0) and 2 * int(b1 != 0).

927 Set `fleet_action[aircraft, :7]` by construct the shown list in its stated order.

928 Bitwise-AND with `~(1 << crew)` in `free_mask`, and store the result back in that same target. The change is visible to subsequent statements in this block.

929 Call shops.remove with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

930 Subtract `needed` from `stock`, and store the result back in that same target. This updates spare inventory by component type.

931 Return `(fleet_action, deferred)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.79. MultiAgentFleetEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 933–943.

```
933     def step(self, actions):
934         fleet_action, deferred = self.compose_action(actions)
935         self.central_observation, reward, terminated, truncated, info = self.env.step(fleet_action)
936         self.coordination_deferrals += len(deferred)
937         info['metrics']['coordination_deferrals'] = self.coordination_deferrals
938         info.update(agent_ids=self.agent_ids, agent_proposals={key: np.asarray(value).tolist() for
            ↪  key, value in actions.items()},
939                     fleet_action=fleet_action.tolist(), deferred_requests=deferred,
940                     central_observation=self.central_observation.copy())
941         # The cooperative reward is replicated, not divided by the number of agents.
942         rewards = {agent: reward for agent in self.agent_ids}
943         return self.observations(), rewards, terminated, truncated, info
```

Line-by-line explanation.

933 Define `step(self, actions)`. This step method is part of MultiAgentFleetEnv.step; its parameters and defaults are the interface used by the calls in this section.

934 Set `(fleet_action, deferred)` by call self.compose_action with the shown positional and keyword arguments.

935 Set `(self.central_observation, reward, terminated, truncated, info)` by execute this object's step with the shown arguments.

936 Add `len(deferred)` to `self.coordination_deferrals`, and store the result back in that same target. The change is visible to subsequent statements in this block.

937 Set `info['metrics']['coordination_deferrals']` by evaluate self.coordination_deferrals.

938 Call info.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

939 Continue the expression begun at line 938 with `fleet_action=fleet_action.tolist(), deferred_requests=deferred,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

940 Continue the expression begun at line 938 with `central_observation=self.central_observation.copy())`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

941 Source comment: The cooperative reward is replicated, not divided by the number of agents. This line documents the following code and does not execute.

942 Set `rewards` by construct the collection by iterating the shown comprehension and applying its filters. It holds the primitive reward sequence used for return accounting.

943 Return `(self.observations(), rewards, terminated, truncated, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.80. MultiAgentActorCritic

The local-view MAPPO controller. Aircraft actors share parameters; component actors have separate parameters. The centralized observed-belief critic returns one value per agent, while actor-only execution consumes restricted inputs.

Source lines 946–953.

```
946 class MultiAgentActorCritic(nn.Module):
947     """Feed-forward MAPPO prototype with explicit per-agent policy statistics.
949     Aircraft agents share actor weights; six specialist actors have separate
950     weights. The centralized critic has one value head per agent and receives
951     the full observed fleet belief, never the simulator's hidden health.
952     Actors can execute through policy(agent_observations) without the critic.
953     """
```

Line-by-line explanation.

946 Define `MultiAgentActorCritic` with the stated inheritance. The local-view MAPPO controller. Aircraft actors share parameters; component actors have separate parameters. The centralized observed-belief critic returns one value per agent, while actor-only execution consumes restricted inputs.

947 Documentation text: Feed-forward MAPPO prototype with explicit per-agent policy statistics.. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

949 Documentation text: Aircraft agents share actor weights; six specialist actors have separate. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

950 Documentation text: weights. The centralized critic has one value head per agent and receives. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

951 Documentation text: the full observed fleet belief, never the simulator's hidden health.. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

952 Documentation text: Actors can execute through policy(agent_observations) without the critic.. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

953 Close the documentation string for `MultiAgentActorCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.81. MultiAgentActorCritic.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 955–972.

```
955     def __init__(self, env, assignment='aircraft'):
956         super().__init__()
957         if assignment not in ('aircraft', 'component'):
958             raise ValueError('assignment must be aircraft or component')
959         self.assignment = assignment
```

```
960         self.n = env.config.aircraft
961         self.local = env.local_size
962         self.public = env.public_size
963         self.agent_ids = env.aircraft_plates if assignment == 'aircraft' else COMPONENTS
964         self.num_agents = len(self.agent_ids)
965         self.input_size = self.local + self.public if assignment == 'aircraft' else 15 * self.n +
            ↪  self.public
966         self.fields = 2 if assignment == 'aircraft' else self.n
967         self.categories = 64 if assignment == 'aircraft' else 4
968         count = 1 if assignment == 'aircraft' else 6
969         self.actors = nn.ModuleList([nn.Sequential(nn.Linear(self.input_size, 64), nn.Tanh(),
970                                                    nn.Linear(64, 64), nn.Tanh(),
971                                                    nn.Linear(64, self.fields * self.categories)) for _
                                                    ↪  in range(count)])
972         self.critic = nn.Sequential(nn.Linear(env.observation_space.shape[0], 64), nn.Tanh(),
```

Line-by-line explanation.

955 Define `__init__(self, env, assignment='aircraft')`. This ___init___ method is part of MultiAgentActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

956 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

957 Test `assignment not in ('aircraft', 'component')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

958 Raise `ValueError('assignment must be aircraft or component')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

959 Set `self.assignment` by evaluate assignment.

960 Set `self.n` by evaluate env.config.aircraft.

961 Set `self.local` by evaluate env.local_size.

962 Set `self.public` by evaluate env.public_size.

963 Set `self.agent_ids` by select env.aircraft_plates when assignment == 'aircraft', otherwise COMPONENTS.

964 Set `self.num_agents` by count entries.

965 Set `self.input_size` by select self.local + self.public when assignment == 'aircraft', otherwise 15 * self.n + self.public.

966 Set `self.fields` by select 2 when assignment == 'aircraft', otherwise self.n.

967 Set `self.categories` by select 64 when assignment == 'aircraft', otherwise 4.

968 Set `count` by select 1 when assignment == 'aircraft', otherwise 6.

969 Set `self.actors` by register the listed independent networks so parameters and checkpoints include them.

970 Continue the assignment begun at line 969 with `nn.Linear(64, 64), nn.Tanh(),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

971 Continue the assignment begun at line 969 with `nn.Linear(64, self.fields * self.categories)) for _ in range(count)])`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

972 Set `self.critic` by compose the listed layers in execution order.

Source lines 973–973.

```
973                                     nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, self.num_agents))
```

Line-by-line explanation.

973 Continue the assignment begun at line 972 with `nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, self.num_agents))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

10.82. MultiAgentActorCritic.agent_inputs

Prepare the actor inputs for this policy family. MAPPO restricts inputs by owner; the DCMAC override supplies the global observed fleet belief.

Source lines 975–976.

```
975     def agent_inputs(self, obs):
976         return mappo_agent_inputs(obs, self.n, self.local, self.public, self.assignment)
```

Line-by-line explanation.

975 Define `agent_inputs(self, obs)`. Prepare the actor inputs for this policy family. MAPPO restricts inputs by owner; the DCMAC override supplies the global observed fleet belief.

976 Return `mappo_agent_inputs(obs, self.n, self.local, self.public, self.assignment)` to the caller by call mappo_agent_inputs with the shown positional and keyword arguments. No further statements in this call execute.

10.83. MultiAgentActorCritic.policy

Run the actor path using its allowed information. The separate critic path need not be available during actor-only execution.

Source lines 978–995.

```
978     def policy(self, agent_obs, actions=None, deterministic=False):
979         """Execute only restricted actors; no central observation or critic needed."""
980         x = torch.as_tensor(agent_obs, dtype=torch.float32)
981         if x.ndim != 3 or x.shape[1:] != (self.num_agents, self.input_size):
982             raise ValueError('incorrect per-agent observation shape')
983         if self.assignment == 'aircraft':
984             logits = self.actors[0](x).reshape(-1, self.num_agents, self.fields, self.categories)
985             busy = (x[:, :, 37] > 0.5).unsqueeze(-1).expand(-1, -1, 2)
986         else:
987             logits = torch.stack([actor(x[:, i]) for i, actor in enumerate(self.actors)], dim=1)
988             logits = logits.reshape(-1, 6, self.n, 4)
989             busy = x[:, :, :15 * self.n].reshape(-1, 6, self.n, 15)[:, :, :, 11] > 0.5
990         valid = (~busy).unsqueeze(-1) | (torch.arange(self.categories, device=x.device) == 0)
991         distribution = Categorical(logits=logits.masked_fill(~valid, -1e9))
992         if actions is None:
993             selected = distribution.probs.argmax(-1) if deterministic else distribution.sample()
994         else:
995             selected = torch.as_tensor(actions, device=x.device)
```

Line-by-line explanation.

978 Define `policy(self, agent_obs, actions=None, deterministic=False)`. This policy method is part of MultiAgentActorCritic.policy; its parameters and defaults are the interface used by the calls in this section.

979 Documentation text: Execute only restricted actors; no central observation or critic needed.. This states the contract or qualification for `MultiAgentActorCritic.policy`; it does not update the simulator or model.

980 Set `x` by convert inputs to a tensor with the requested dtype.

981 Test `x.ndim != 3 or x.shape[1:] != (self.num_agents, self.input_size)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

982 Raise `ValueError('incorrect per-agent observation shape')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

983 Test `self.assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

984 Set `logits` by reinterpret the shape while retaining element order.

985 Set `busy` by call (x[:, :, 37] > 0.5).unsqueeze(-1).expand with the shown positional and keyword arguments.

986 Execute this alternative only when the corresponding preceding condition did not select its branch.

987 Set `logits` by join equal-shaped tensors along the specified new axis.

988 Set `logits` by reinterpret the shape while retaining element order.

989 Set `busy` by evaluate the comparison x[:, :, :15 * self.n].reshape(-1, 6, self.n, 15)[:, :, :, 11] > 0.5.

990 Set `valid` by apply a bitwise union to (~busy).unsqueeze(-1) and torch.arange(self.categories, device=x.device) == 0. It holds the categorical structural-validity mask.

991 Set `distribution` by construct a discrete distribution from the supplied logits/probabilities.

992 Test `actions is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

993 Set `selected` by select distribution.probs.argmax(-1) when deterministic, otherwise distribution.sample().

994 Execute this alternative only when the corresponding preceding condition did not select its branch.

995 Set `selected` by convert inputs to a tensor with the requested dtype.

Source lines 996–1001.

```
            if selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64):
                raise ValueError('incorrect stored agent action shape or dtype')
            if torch.any((selected < 0) | (selected >= self.categories)) or torch.any(busy & (selected
            ↪  != 0)):
                raise ValueError('stored agent action violates a category or busy mask')
        # Sum fields WITHIN each agent; preserve the agent axis for MAPPO clipping.
        return selected, distribution.log_prob(selected).sum(-1), distribution.entropy().sum(-1)
```

Line-by-line explanation.

996 Test `selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

997 Raise `ValueError('incorrect stored agent action shape or dtype')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

998 Test `torch.any((selected < 0) | (selected >= self.categories)) or torch.any(busy & (selected != 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

999 Raise `ValueError('stored agent action violates a category or busy mask')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1000 Source comment: Sum fields WITHIN each agent; preserve the agent axis for MAPPO clipping. This line documents the following code and does not execute.

1001 Return `(selected, distribution.log_prob(selected).sum(-1), distribution.entropy().sum(-1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.84. MultiAgentActorCritic.active_agents

Identify which decision roles have an actual free-aircraft action opportunity for normalization and actor updates.

Source lines 1003–1007.

```
1003     def active_agents(self, agent_obs):
1004         if self.assignment == 'aircraft':
1005             return (agent_obs[:, :, 37] <= 0.5).float()
1006         busy = agent_obs[:, :, :15 * self.n].reshape(-1, 6, self.n, 15)[:, :, :, 11] > 0.5
1007         return (~busy).any(-1).float()
```

Line-by-line explanation.

1003 Define `active_agents(self, agent_obs)`. Identify which decision roles have an actual free-aircraft action opportunity for normalization and actor updates.

1004 Test `self.assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1005 Return `(agent_obs[:, :, 37] <= 0.5).float()` to the caller by call (agent_obs[:, :, 37] <= 0.5).float with the shown positional and keyword arguments. No further statements in this call execute.

1006 Set `busy` by evaluate the comparison agent_obs[:, :, :15 * self.n].reshape(-1, 6, self.n, 15)[:, :, :, 11] > 0.5.

1007 Return `(~busy).any(-1).float()` to the caller by call (~busy).any(-1).float with the shown positional and keyword arguments. No further statements in this call execute.

10.85. MultiAgentActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1009–1011.

```
1009     def value(self, obs):
1010         x = torch.as_tensor(obs, dtype=torch.float32)
1011         return self.critic(x.unsqueeze(0) if x.ndim == 1 else x)
```

Line-by-line explanation.

`1009` Define `value(self, obs)`. This value method is part of MultiAgentActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

`1010` Set `x` by convert inputs to a tensor with the requested dtype.

`1011` Return `self.critic(x.unsqueeze(0) if x.ndim == 1 else x)` to the caller by call self.critic with the shown positional and keyword arguments. No further statements in this call execute.

10.86. MultiAgentActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 1013–1015.

```
1013     def act(self, obs, actions=None, deterministic=False):
1014         selected, logp, entropy = self.policy(self.agent_inputs(obs), actions, deterministic)
1015         return selected, logp, entropy, self.value(obs)
```

Line-by-line explanation.

`1013` Define `act(self, obs, actions=None, deterministic=False)`. This act method is part of MultiAgentActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

`1014` Set `(selected, logp, entropy)` by call self.policy with the shown positional and keyword arguments.

`1015` Return `(selected, logp, entropy, self.value(obs))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.87. MultiAgentActorCritic.action_dict

Convert batched sampled action tensors into the agent-keyed NumPy requests expected by the multi-agent adapter.

Source lines 1017–1021.

```
    def action_dict(self, selected):
        raw = torch.as_tensor(selected).detach().cpu().numpy()
        if raw.shape != (self.num_agents, self.fields):
            raise ValueError('incorrect unbatched agent action shape')
        return {agent: raw[i].copy() for i, agent in enumerate(self.agent_ids)}
```

Line-by-line explanation.

1017 Define `action_dict(self, selected)`. Convert batched sampled action tensors into the agent-keyed NumPy requests expected by the multi-agent adapter.

1018 Set `raw` by expose this CPU tensor as a NumPy array.

1019 Test `raw.shape != (self.num_agents, self.fields)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1020 Raise `ValueError('incorrect unbatched agent action shape')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1021 Return `{agent: raw[i].copy() for i, agent in enumerate(self.agent_ids)}` to the caller by construct the collection by iterating the shown comprehension and applying its filters. No further statements in this call execute.

10.88. Module declarations and integration

Source lines 1024–1025.

```
DCMAC_METHODS = ('dcmac_aircraft', 'ddmac_aircraft', 'dcmac_component', 'ddmac_component')
REQUEST_METHODS = (*MAPPO_METHODS, *DCMAC_METHODS)
```

Line-by-line explanation.

1024 Set `DCMAC_METHODS` by construct the shown tuple in its stated order.

1025 Set `REQUEST_METHODS` by construct the shown tuple in its stated order.

10.89. request_assignment

Translate a registered request-method suffix into aircraft versus component ownership and reject unsupported method names.

Source lines 1028–1032.

```
def request_assignment(method):
    """Resolve ownership for registered proposal methods; reject silent fallbacks."""
    if method not in REQUEST_METHODS:
        raise ValueError('unknown request policy method')
    return method.rsplit('_', 1)[1]
```

Line-by-line explanation.

1028 Define `request_assignment(method)`. Translate a registered request-method suffix into aircraft versus component ownership and reject unsupported method names.

1029 Documentation text: Resolve ownership for registered proposal methods; reject silent fallbacks.. This states the contract or qualification for `request_assignment`; it does not update the simulator or model.

1030 Test `method not in REQUEST_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1031 Raise `ValueError('unknown request policy method')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1032 Return `method.rsplit('_', 1)[1]` to the caller by select the indexed/sliced entries of method.rsplit('_', 1). No further statements in this call execute.

10.90. DCMACActorCritic

The global-view DCMAC/DDMAC adaptation. DCMAC has one joint network with factorized output blocks; DDMAC constructs one global-input actor per agent. Both use one scalar fleet critic and the replay-based trainer.

Source lines 1035–1044.

```
1035 class DCMACActorCritic(MultiAgentActorCritic):
1036     """Fleet adaptation of DCMAC/DDMAC; global observed beliefs enter all actors.
1038     DCMAC has one actor with factorized control-unit outputs. DDMAC has an
1039     independently parameterized actor per control unit. Both have one scalar
1040     centralized value network and use replay with joint importance weighting.
1041     The fixed dispatcher owns resources; paper-specific inspection actions,
1042     budget augmentation and primal-dual risk constraints are not reproduced.
1043     Inherits only the proposal-to-dictionary conversion from the MAPPO class.
1044     """
```

Line-by-line explanation.

1035 Define `DCMACActorCritic` with the stated inheritance. The global-view DCMAC/DDMAC adaptation. DCMAC has one joint network with factorized output blocks; DDMAC constructs one global-input actor per agent. Both use one scalar fleet critic and the replay-based trainer.

1036 Documentation text: Fleet adaptation of DCMAC/DDMAC; global observed beliefs enter all actors.. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1038 Documentation text: DCMAC has one actor with factorized control-unit outputs. DDMAC has an. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1039 Documentation text: independently parameterized actor per control unit. Both have one scalar. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1040 Documentation text: centralized value network and use replay with joint importance weighting.. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1041 Documentation text: The fixed dispatcher owns resources; paper-specific inspection actions,. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1042 Documentation text: budget augmentation and primal-dual risk constraints are not reproduced.. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1043 Documentation text: Inherits only the proposal-to-dictionary conversion from the MAPPO class.. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1044 Close the documentation string for `DCMACActorCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.91. DCMACActorCritic.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1046–1063.

```
1046     def __init__(self, env, assignment='aircraft', decentralized=False):
1047         nn.Module.__init__(self)
1048         if assignment not in ('aircraft', 'component'):
1049             raise ValueError('assignment must be aircraft or component')
1050         self.assignment = assignment
1051         self.decentralized = decentralized
1052         self.n = env.config.aircraft
1053         self.local = env.local_size
1054         self.public = env.public_size
1055         self.agent_ids = env.aircraft_plates if assignment == 'aircraft' else COMPONENTS
1056         self.num_agents = len(self.agent_ids)
1057         self.input_size = env.observation_space.shape[0]
1058         self.fields = 2 if assignment == 'aircraft' else self.n
1059         self.categories = 64 if assignment == 'aircraft' else 4
1060         count = self.num_agents if decentralized else 1
1061         output = self.fields * self.categories * (1 if decentralized else self.num_agents)
1062         self.actors = nn.ModuleList([
1063             nn.Sequential(nn.Linear(self.input_size, 64), nn.Tanh(),
```

Line-by-line explanation.

1046 Define `__init__(self, env, assignment='aircraft', decentralized=False)`. This ___init___ method is part of DCMACActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

1047 Call nn.module.___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1048 Test `assignment not in ('aircraft', 'component')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1049 Raise `ValueError('assignment must be aircraft or component')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1050 Set `self.assignment` by evaluate assignment.

1051 Set `self.decentralized` by evaluate decentralized.

1052 Set `self.n` by evaluate env.config.aircraft.

1053 Set `self.local` by evaluate env.local_size.

1054 Set `self.public` by evaluate env.public_size.

1055 Set `self.agent_ids` by select env.aircraft_plates when assignment == 'aircraft', otherwise COMPONENTS.

1056 Set `self.num_agents` by count entries.

1057 Set `self.input_size` by select the indexed/sliced entries of env.observation_space.shape.

1058 Set `self.fields` by select 2 when assignment == 'aircraft', otherwise self.n.

1059 Set `self.categories` by select 64 when assignment == 'aircraft', otherwise 4.

1060 Set `count` by select self.num_agents when decentralized, otherwise 1.

1061 Set `output` by multiply self.fields * self.categories and 1 if decentralized else self.num_agents.

1062 Set `self.actors` by register the listed independent networks so parameters and checkpoints include them.

1063 Continue the assignment begun at line 1062 with `nn.Sequential(nn.Linear(self.input_size, 64), nn.Tanh(),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

Source lines 1064–1068.

```
1064                         nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, output))
1065             for _ in range(count)
1066         ])
1067         self.critic = nn.Sequential(nn.Linear(self.input_size, 64), nn.Tanh(),
1068                                     nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, 1))
```

Line-by-line explanation.

1064 Continue the assignment begun at line 1062 with `nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, output))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1065 Continue the assignment begun at line 1062 with `for _ in range(count)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1066 Continue the assignment begun at line 1062 with `])`. Close the nested call/container at this indentation, completing that part of the same statement.

1067 Set `self.critic` by compose the listed layers in execution order.

1068 Continue the assignment begun at line 1067 with `nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, 1))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

10.92. DCMACActorCritic.agent_inputs

Prepare the actor inputs for this policy family. MAPPO restricts inputs by owner; the DCMAC override supplies the global observed fleet belief.

Source lines 1070–1073.

```
1070    def agent_inputs(self, obs):
1071        """Return global actor inputs; unlike MAPPO, execution needs fleet beliefs."""
1072        return _fleet_observation_tensor(obs, self.input_size,
1073                                         'incorrect global actor observation shape')
```

Line-by-line explanation.

1070 Define `agent_inputs(self, obs)`. Prepare the actor inputs for this policy family. MAPPO restricts inputs by owner; the DCMAC override supplies the global observed fleet belief.

1071 Documentation text: Return global actor inputs; unlike MAPPO, execution needs fleet beliefs.. This states the contract or qualification for `DCMACActorCritic.agent_inputs`; it does not update the simulator or model.

1072 Return `_fleet_observation_tensor(obs, self.input_size, 'incorrect global actor observation shape')` to the caller by call _fleet_observation_tensor with the shown positional and keyword arguments. No further statements in this call execute.

1073 Continue the expression begun at line 1072 with `'incorrect global actor observation shape')`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

10.93. DCMACActorCritic._busy_fields

Identify action fields whose owning aircraft is physically committed so their distributions can be forced to the no-operation category.

Source lines 1075–1079.

```
1075    def _busy_fields(self, obs):
1076        busy = obs[:, :self.n * self.local].reshape(-1, self.n, self.local)[:, :, 37] > 0.5
1077        if self.assignment == 'aircraft':
1078            return busy.unsqueeze(-1).expand(-1, -1, 2)
1079        return busy.unsqueeze(1).expand(-1, 6, -1)
```

Line-by-line explanation.

1075 Define `_busy_fields(self, obs)`. Identify action fields whose owning aircraft is physically committed so their distributions can be forced to the no-operation category.

1076 Set `busy` by evaluate the comparison obs[:, :self.n * self.local].reshape(-1, self.n, self.local)[:, :, 37] $>$ 0.5.

1077 Test `self.assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1078 Return `busy.unsqueeze(-1).expand(-1, -1, 2)` to the caller by call busy.unsqueeze(-1).expand with the shown positional and keyword arguments. No further statements in this call execute.

1079 Return `busy.unsqueeze(1).expand(-1, 6, -1)` to the caller by call busy.unsqueeze(1).expand with the shown positional and keyword arguments. No further statements in this call execute.

10.94. DCMACActorCritic.active_agents

Identify which decision roles have an actual free-aircraft action opportunity for normalization and actor updates.

Source lines 1081–1082.

```
1081     def active_agents(self, obs):
1082         return (~self._busy_fields(self.agent_inputs(obs))).any(-1).float()
```

Line-by-line explanation.

1081 Define `active_agents(self, obs)`. Identify which decision roles have an actual free-aircraft action opportunity for normalization and actor updates.

1082 Return `(~self._busy_fields(self.agent_inputs(obs))).any(-1).float()` to the caller by call (~self._busy_fields(self.agent_inputs(obs))).any(-1).float with the shown positional and keyword arguments. No further statements in this call execute.

10.95. DCMACActorCritic.policy

Run the actor path using its allowed information. The separate critic path need not be available during actor-only execution.

Source lines 1084–1101.

```
1084     def policy(self, obs, actions=None, deterministic=False):
1085         """Factorized categorical proposals; no hidden state or critic evaluation."""
1086         x = self.agent_inputs(obs)
1087         if self.decentralized:
1088             logits = torch.stack([actor(x) for actor in self.actors], dim=1)
1089         else:
1090             logits = self.actors[0](x)
1091         logits = logits.reshape(-1, self.num_agents, self.fields, self.categories)
1092         busy = self._busy_fields(x)
1093         valid = (~busy).unsqueeze(-1) | (torch.arange(self.categories, device=x.device) == 0)
1094         distribution = Categorical(logits=logits.masked_fill(~valid, -1e9))
1095         if actions is None:
1096             selected = distribution.probs.argmax(-1) if deterministic else distribution.sample()
1097         else:
1098             selected = torch.as_tensor(actions, device=x.device)
1099             if selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64):
1100                 raise ValueError('incorrect stored agent action shape or dtype')
1101             if torch.any((selected < 0) | (selected >= self.categories)) or torch.any(busy & (selected
                 ↪  != 0)):
```

Line-by-line explanation.

1084 Define `policy(self, obs, actions=None, deterministic=False)`. This policy method is part of DCMACActorCritic.policy; its parameters and defaults are the interface used by the calls in this section.

1085 Documentation text: Factorized categorical proposals; no hidden state or critic evaluation.. This states the contract or qualification for `DCMACActorCritic.policy`; it does not update the simulator or model.

1086 Set `x` by call self.agent_inputs with the shown positional and keyword arguments.

1087 Test `self.decentralized`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1088 Set `logits` by join equal-shaped tensors along the specified new axis.

1089 Execute this alternative only when the corresponding preceding condition did not select its branch.

1090 Set `logits` by call self.actors[0] with the shown positional and keyword arguments.

1091 Set `logits` by reinterpret the shape while retaining element order.

1092 Set `busy` by call self._busy_fields with the shown positional and keyword arguments.

1093 Set `valid` by apply a bitwise union to (~busy).unsqueeze(-1) and torch.arange(self.categories, device=x.device) == 0. It holds the categorical structural-validity mask.

1094 Set `distribution` by construct a discrete distribution from the supplied logits/probabilities.

1095 Test `actions is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1096 Set `selected` by select distribution.probs.argmax(-1) when deterministic, otherwise distribution.sample().

1097 Execute this alternative only when the corresponding preceding condition did not select its branch.

1098 Set `selected` by convert inputs to a tensor with the requested dtype.

1099 Test `selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1100 Raise `ValueError('incorrect stored agent action shape or dtype')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1101 Test `torch.any((selected < 0) | (selected >= self.categories)) or torch.any(busy & (selected != 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 1102–1103.

```
1102                 raise ValueError('stored agent action violates a category or busy mask')
1103         return selected, distribution.log_prob(selected).sum(-1), distribution.entropy().sum(-1)
```

Line-by-line explanation.

1102 Raise `ValueError('stored agent action violates a category or busy mask')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1103 Return `(selected, distribution.log_prob(selected).sum(-1), distribution.entropy().sum(-1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.96. DCMACActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1105–1106.

```
1105     def value(self, obs):
1106         return self.critic(self.agent_inputs(obs)).squeeze(-1)
```

Line-by-line explanation.

`1105` Define `value(self, obs)`. This value method is part of DCMACActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

`1106` Return `self.critic(self.agent_inputs(obs)).squeeze(-1)` to the caller by call self.critic(self.agent_inputs(obs)).squeeze with the shown positional and keyword arguments. No further statements in this call execute.

10.97. DCMACActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 1108–1110.

```
1108     def act(self, obs, actions=None, deterministic=False):
1109         selected, logp, entropy = self.policy(obs, actions, deterministic)
1110         return selected, logp, entropy, self.value(obs)
```

Line-by-line explanation.

`1108` Define `act(self, obs, actions=None, deterministic=False)`. This act method is part of DCMACActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

`1109` Set `(selected, logp, entropy)` by call self.policy with the shown positional and keyword arguments.

`1110` Return `(selected, logp, entropy, self.value(obs))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.98. dcmac_losses

Compute the joint importance ratio, capped detached replay weight, scalar TD residual, weighted policy loss and weighted value loss. Agent log probabilities are summed before importance weighting.

Source lines 1113–1131.

```
1113 def dcmac_losses(logp, behavior_logp, values, next_values, rewards, dones,
1114                  gamma=0.99, importance_clip=2.0):
1115     """One-step replay update: joint truncated ratio and detached TD advantage.
1117     The ratio is pi(joint proposal|belief)/mu(joint proposal|belief), not the
1118     product of separately truncated ratios. Truncation introduces bias but
1119     bounds the high-variance replay weights. No PPO clipping or GAE is used.
1120     """
1121     if logp.ndim != 2 or behavior_logp.shape != logp.shape:
1122         raise ValueError('proposal log probabilities must have batch x agent shape')
1123     if any(t.shape != (len(logp),) for t in (values, next_values, rewards, dones)):
```

```
        raise ValueError('one scalar value, reward and terminal flag per replay transition required')
    if not np.isfinite(importance_clip) or importance_clip <= 0 or not 0 <= gamma <= 1:
        raise ValueError('positive finite importance cap and gamma in [0,1] required')
    if not all(bool(torch.isfinite(t).all()) for t in (logp, behavior_logp, values, next_values,
    ↪  rewards, dones)):
        raise ValueError('replay tensors must be finite')
    joint_logp = logp.sum(-1)
    log_ratio = (joint_logp - behavior_logp.sum(-1)).detach()
    weights = torch.exp(log_ratio.clamp(max=float(np.log(importance_clip))))
```

Line-by-line explanation.

1113 Define `dcmac_losses(logp, behavior_logp, values, next_values, rewards, dones,`. Compute the joint importance ratio, capped detached replay weight, scalar TD residual, weighted policy loss and weighted value loss. Agent log probabilities are summed before importance weighting.

1114 Execute the Python fragment `gamma=0.99, importance_clip=2.0):` in `dcmac_losses`. Its indentation places it in the surrounding control block.

1115 Documentation text: One-step replay update: joint truncated ratio and detached TD advantage.. This states the contract or qualification for `dcmac_losses`; it does not update the simulator or model.

1117 Documentation text: The ratio is pi(joint proposal|belief)/mu(joint proposal|belief), not the. This states the contract or qualification for `dcmac_losses`; it does not update the simulator or model.

1118 Documentation text: product of separately truncated ratios. Truncation introduces bias but. This states the contract or qualification for `dcmac_losses`; it does not update the simulator or model.

1119 Documentation text: bounds the high-variance replay weights. No PPO clipping or GAE is used.. This states the contract or qualification for `dcmac_losses`; it does not update the simulator or model.

1120 Close the documentation string for `dcmac_losses`. This delimiter ends the descriptive text and performs no simulator or learning update.

1121 Test `logp.ndim != 2 or behavior_logp.shape != logp.shape`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1122 Raise `ValueError('proposal log probabilities must have batch x agent shape')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1123 Test `any((t.shape != (len(logp),) for t in (values, next_values, rewards, dones)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1124 Raise `ValueError('one scalar value, reward and terminal flag per replay transition required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1125 Test `not np.isfinite(importance_clip) or importance_clip <= 0 or (not 0 <= gamma <= 1)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1126 Raise `ValueError('positive finite importance cap and gamma in [0,1] required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1127 Test `not all((bool(torch.isfinite(t).all()) for t in (logp, behavior_logp, values, next_values, rewards, dones)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1128 Raise `ValueError('replay tensors must be finite')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1129 Set `joint_logp` by sum across the specified tensor/array axis.

1130 Set `log_ratio` by stop gradients through this result.

1131 Set `weights` by exponentiate log values, including log-probability ratios. It holds the detached capped importance weights for replay.

Source lines 1132–1136.

```
1132     target = rewards + gamma * (1 - dones.float()) * next_values.detach()
1133     delta = target - values
1134     actor_loss = -(weights * joint_logp * delta.detach()).mean()
1135     critic_loss = 0.5 * (weights * delta.square()).mean()
1136     return actor_loss, critic_loss, weights, target.detach()
```

Line-by-line explanation.

1132 Set `target` by add rewards and gamma * (1 - dones.float()) * next_values.detach(). It holds a bootstrapped learning target.

1133 Set `delta` by subtract target and values.

1134 Set `actor_loss` by evaluate -(weights * joint_logp * delta.detach()).mean(). It holds the objective differentiated through policy parameters.

1135 Set `critic_loss` by multiply 0.5 and (weights * delta.square()).mean(). It holds the prediction-error objective for the value estimator.

1136 Return `(actor_loss, critic_loss, weights, target.detach())` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.99. train_dcmac

Collect daily joint request transitions into bounded uniform replay, then update global-input actor(s) and the scalar critic with importance-weighted TD losses. This function does not apply PPO clipping or GAE.

Source lines 1139–1157.

```
1139 def train_dcmac(config, method='dcmac_aircraft', steps=4096, seed=0,
1140                 rollout_size=128, replay_capacity=4096, importance_clip=2.0):
1141     """Off-policy DCMAC/DDMAC adaptation with uniform bounded experience replay.
1143     Stochastic actor sampling is the behavior policy; its original proposal
1144     likelihood is stored even when the dispatcher defers that proposal.
1145     One replay minibatch is updated per day after two samples are available.
1146     rollout_size names minibatch size here, preserving the common train API.
1147     """
1148     if method not in DCMAC_METHODS or steps < 1 or rollout_size < 2 or replay_capacity < 2:
1149         raise ValueError('registered DCMAC/DDMAC method, positive steps and batch/capacity >= 2
         ↪  required')
1150     if not np.isfinite(importance_clip) or importance_clip <= 0:
1151         raise ValueError('importance_clip must be positive and finite')
```

```
1152      torch.set_num_threads(1)
1153      torch.manual_seed(seed)
1154      rng = np.random.default_rng(seed)
1155      assignment = request_assignment(method)
1156      env = MultiAgentFleetEnv(config, assignment)
1157      model = DCMACActorCritic(env.env, assignment, method.startswith('ddmac_'))
```

Line-by-line explanation.

1139 Define `train_dcmac(config, method='dcmac_aircraft', steps=4096, seed=0,`. Collect daily joint request transitions into bounded uniform replay, then update global-input actor(s) and the scalar critic with importance-weighted TD losses. This function does not apply PPO clipping or GAE.

1140 Execute the Python fragment `rollout_size=128, replay_capacity=4096, importance_clip=2.0):` in `train_dcmac`. Its indentation places it in the surrounding control block.

1141 Documentation text: Off-policy DCMAC/DDMAC adaptation with uniform bounded experience replay.. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

1143 Documentation text: Stochastic actor sampling is the behavior policy; its original proposal. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

1144 Documentation text: likelihood is stored even when the dispatcher defers that proposal.. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

1145 Documentation text: One replay minibatch is updated per day after two samples are available.. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

1146 Documentation text: rollout_size names minibatch size here, preserving the common train API.. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

1147 Close the documentation string for `train_dcmac`. This delimiter ends the descriptive text and performs no simulator or learning update.

1148 Test `method not in DCMAC_METHODS or steps < 1 or rollout_size < 2 or (replay_capacity < 2)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1149 Raise `ValueError('registered DCMAC/DDMAC method, positive steps and batch/capacity >= 2 required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1150 Test `not np.isfinite(importance_clip) or importance_clip <= 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1151 Raise `ValueError('importance_clip must be positive and finite')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1152 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

1153 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

1154 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

1155 Set `assignment` by call request_assignment with the shown positional and keyword arguments.

1156 Set `env` by call MultiAgentFleetEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

1157 Set `model` by call DCMACActorCritic with the shown positional and keyword arguments. It holds the chosen controller architecture.

Source lines 1158–1175.

```
    actor_optimizer = torch.optim.Adam(model.actors.parameters(), lr=3e-4)
    critic_optimizer = torch.optim.Adam(model.critic.parameters(), lr=3e-4)
    replay, logs = [], []
    _, info = env.reset(seed=seed)
    obs = info['central_observation']
    episode = 0
    for day in range(1, steps + 1):
        with torch.no_grad():
            action, behavior_logp, _ = model.policy(obs)
        _, rewards, done, truncated, info = env.step(model.action_dict(action[0]))
        next_obs = info['central_observation']
        terminal = done or truncated
        transition = (obs.copy(), action[0].numpy().copy(), behavior_logp[0].numpy().copy(),
                      rewards[model.agent_ids[0]], next_obs.copy(), terminal, day)
        if len(replay) < replay_capacity:
            replay.append(transition)
        else:
            replay[(day - 1) % replay_capacity] = transition
```

Line-by-line explanation.

1158 Set `actor_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer restricted to actor parameters.

1159 Set `critic_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer for the centralized value estimator.

1160 Set `(replay, logs)` by construct the shown tuple in its stated order.

1161 Set `(_, info)` by initialize a new episode using the stated seed/options.

1162 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health.

1163 Set `episode` by use the literal 0.

1164 Iterate `day` over `range(1, steps + 1)`. The following indented statements run once for each selected element.

1165 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1166 Set `(action, behavior_logp, _)` by call model.policy with the shown positional and keyword arguments.

1167 Set `(_, rewards, done, truncated, info)` by execute this object's step with the shown arguments.

1168 Set `next_obs` by select the indexed/sliced entries of info.

1169 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

1170 Set `transition` by construct the shown tuple in its stated order.

1171 Continue the assignment begun at line 1170 with `rewards[model.agent_ids[0]], next_obs.copy(), terminal, day)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1172 Test `len(replay) < replay_capacity`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1173 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1174 Execute this alternative only when the corresponding preceding condition did not select its branch.

1175 Set `replay[(day - 1) % replay_capacity]` by evaluate transition. It holds the bounded off-policy transition buffer.

Source lines 1176–1193.

```
        if len(replay) >= 2:
            indices = rng.choice(len(replay), size=min(rollout_size, len(replay)), replace=False)
            batch = [replay[int(i)] for i in indices]
            states, actions, behavior, reward, following, terminal_flags, sampled_days = [
                torch.as_tensor(np.asarray(items)) for items in zip(*batch)
            ]
            _, logp, entropy = model.policy(states, actions)
            values = model.value(states)
            with torch.no_grad():
                next_values = model.value(following)
            actor_loss, value_loss, weights, _ = dcmac_losses(
                logp, behavior, values, next_values, reward.float(), terminal_flags,
                importance_clip=importance_clip
            )
            active = model.active_agents(states)
            entropy_bonus = (entropy * active).sum() / active.sum().clamp_min(1)
            policy_loss = actor_loss - 0.001 * entropy_bonus
            if not bool(torch.isfinite(policy_loss + value_loss)):
```

Line-by-line explanation.

1176 Test `len(replay) >= 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1177 Set `indices` by call rng.choice with the shown positional and keyword arguments.

1178 Set `batch` by construct the collection by iterating the shown comprehension and applying its filters.

1179 Set `(states, actions, behavior, reward, following, terminal_flags, sampled_days)` by construct the collection by iterating the shown comprehension and applying its filters.

1180 Continue the assignment begun at line 1179 with `torch.as_tensor(np.asarray(items)) for items in zip(*batch)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1181 Continue the assignment begun at line 1179 with `]`. Close the nested call/container at this indentation, completing that part of the same statement.

1182 Set `(_, logp, entropy)` by call model.policy with the shown positional and keyword arguments.

1183 Set `values` by call model.value with the shown positional and keyword arguments. It holds critic predictions used for baselines or TD residuals.

1184 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1185 Set `next_values` by call model.value with the shown positional and keyword arguments. It holds the bootstrap value estimate at the following observation.

1186 Set `(actor_loss, value_loss, weights, _)` by compute capped joint replay-weighted actor and scalar-critic losses.

1187 Continue the assignment begun at line 1186 with `logp, behavior, values, next_values, reward.float(), terminal_flags,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1188 Continue the assignment begun at line 1186 with `importance_clip=importance_clip`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1189 Continue the assignment begun at line 1186 with `)`. Close the nested call/container at this indentation, completing that part of the same statement.

1190 Set `active` by call model.active_agents with the shown positional and keyword arguments.

1191 Set `entropy_bonus` by divide (entropy * active).sum() and active.sum().clamp_min(1).

1192 Set `policy_loss` by subtract actor_loss and 0.001 * entropy_bonus.

1193 Test `not bool(torch.isfinite(policy_loss + value_loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 1194–1211.

```
                raise RuntimeError('nonfinite DCMAC/DDMAC loss')
            actor_optimizer.zero_grad()
            policy_loss.backward()
            torch.nn.utils.clip_grad_norm_(model.actors.parameters(), 0.5)
            if bool(active.any()):
                actor_optimizer.step()
            critic_optimizer.zero_grad()
            value_loss.backward()
            torch.nn.utils.clip_grad_norm_(model.critic.parameters(), 0.5)
            critic_optimizer.step()
            logs.append({'record_type': 'update', 'method': method, 'simulated_days': day,
                         'policy_loss': float(actor_loss.detach()), 'value_loss':
                         ↪  float(value_loss.detach()),
                         'importance_mean': float(weights.mean()), 'importance_max':
                         ↪  float(weights.max()),
                         'replay_size': len(replay), 'batch_size': len(batch),
                         'sampled_transition_days': sampled_days.tolist(),
                         'num_agents': model.num_agents})
        obs = next_obs
        if terminal:
```

Line-by-line explanation.

1194 Raise `RuntimeError('nonfinite DCMAC/DDMAC loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1195 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1196 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

1197 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1198 Test `bool(active.any())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1199 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1200 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1201 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

1202 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1203 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1204 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1205 Continue the expression begun at line 1204 with `'policy_loss': float(actor_loss.detach()), 'value_loss': float(value_loss.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1206 Continue the expression begun at line 1204 with `'importance_mean': float(weights.mean()), 'importance_max': float(weights.max()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1207 Continue the expression begun at line 1204 with `'replay_size': len(replay), 'batch_size': len(batch),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1208 Continue the expression begun at line 1204 with `'sampled_transition_days': sampled_days.tolist(),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1209 Continue the expression begun at line 1204 with `'num_agents': model.num_agents})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1210 Set `obs` by evaluate next_obs. It holds the observed belief vector, rather than hidden physical health.

1211 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 1212–1216.

```
            logs.append({'record_type': 'episode', 'episode': episode, 'simulated_days': day,
            ↪  **info['metrics']})
            episode += 1
            _, info = env.reset(seed=seed + episode)
            obs = info['central_observation']
    return model, logs, steps
```

Line-by-line explanation.

1212 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1213 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1214 Set `(_, info)` by initialize a new episode using the stated seed/options.

1215 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health.

1216 Return `(model, logs, steps)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.100. mappo_gae

Calculate bootstrapped generalized advantage estimates while retaining a separate agent axis and masking terminal continuations.

Source lines 1219–1234.

```
def mappo_gae(rewards, values, next_values, dones, gamma=0.99, lam=0.95):
    """Daily generalized advantages with a separate value baseline per agent."""
    values = np.asarray(values, dtype=np.float32)
    next_values = np.asarray(next_values, dtype=np.float32)
    rewards = np.asarray(rewards, dtype=np.float32)
    dones = np.asarray(dones, dtype=np.float32)
    if values.ndim != 2 or next_values.shape != values.shape or rewards.shape != (len(values),) or
    ↪  dones.shape != rewards.shape:
        raise ValueError('MAPPO rollout shapes must be time x agent, with one team reward and done per
        ↪  time')
    advantages = np.zeros_like(values)
    carry = np.zeros(values.shape[1], dtype=np.float32)
    for t in range(len(rewards) - 1, -1, -1):
        live = 1 - dones[t]
        delta = rewards[t] + gamma * next_values[t] * live - values[t]
        carry = delta + gamma * lam * live * carry
        advantages[t] = carry
    return advantages, advantages + values
```

Line-by-line explanation.

1219 Define `mappo_gae(rewards, values, next_values, dones, gamma=0.99, lam=0.95)`. Calculate bootstrapped generalized advantage estimates while retaining a separate agent axis and masking terminal continuations.

1220 Documentation text: Daily generalized advantages with a separate value baseline per agent.. This states the contract or qualification for `mappo_gae`; it does not update the simulator or model.

1221 Set `values` by call np.asarray with the shown positional and keyword arguments. It holds critic predictions used for baselines or TD residuals.

1222 Set `next_values` by call np.asarray with the shown positional and keyword arguments. It holds the bootstrap value estimate at the following observation.

1223 Set `rewards` by call np.asarray with the shown positional and keyword arguments. It holds the primitive reward sequence used for return accounting.

1224 Set `dones` by call np.asarray with the shown positional and keyword arguments.

1225 Test `values.ndim != 2 or next_values.shape != values.shape or rewards.shape != (len(values),) or (dones.shape != rewards.shape)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1226 Raise `ValueError('MAPPO rollout shapes must be time x agent, with one team reward and done per time')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1227 Set `advantages` by call np.zeros_like with the shown positional and keyword arguments. It holds the policy credit signal with its retained aircraft/agent axis.

1228 Set `carry` by call np.zeros with the shown positional and keyword arguments.

1229 Iterate `t` over `range(len(rewards) - 1, -1, -1)`. The following indented statements run once for each selected element.

1230 Set `live` by subtract 1 and dones[t].

1231 Set `delta` by subtract rewards[t] + gamma * next_values[t] * live and values[t].

1232 Set `carry` by add delta and gamma * lam * live * carry.

1233 Set `advantages[t]` by evaluate carry. It holds the policy credit signal with its retained aircraft/agent axis.

1234 Return `(advantages, advantages + values)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.101. mappo_policy_loss

Form individual-agent probability ratios and apply PPO clipping before averaging over active decisions; this differs from clipping a fleet-joint ratio.

Source lines 1237–1243.

```
1237 def mappo_policy_loss(new_logp, old_logp, advantages, active, clip=0.2):
1238     """Clip each agent ratio BEFORE averaging; never form a fleet joint ratio."""
1239     if new_logp.ndim != 2 or not (new_logp.shape == old_logp.shape == advantages.shape ==
     ↪  active.shape):
1240         raise ValueError('MAPPO policy tensors must have identical batch x agent shapes')
1241     ratio = torch.exp(new_logp - old_logp)
1242     surrogate = torch.minimum(ratio * advantages, ratio.clamp(1 - clip, 1 + clip) * advantages)
1243     return -(surrogate * active).sum() / active.sum().clamp_min(1)
```

Line-by-line explanation.

1237 Define `mappo_policy_loss(new_logp, old_logp, advantages, active, clip=0.2)`. Form individual-agent probability ratios and apply PPO clipping before averaging over active decisions; this differs from clipping a fleet-joint ratio.

1238 Documentation text: Clip each agent ratio BEFORE averaging; never form a fleet joint ratio.. This states the contract or qualification for `mappo_policy_loss`; it does not update the simulator or model.

1239 Test `new_logp.ndim != 2 or not new_logp.shape == old_logp.shape == advantages.shape == active.shape`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1240 Raise `ValueError('MAPPO policy tensors must have identical batch x agent shapes')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1241 Set `ratio` by exponentiate log values, including log-probability ratios. It holds the current-to-behavior probability ratio.

1242 Set `surrogate` by call torch.minimum with the shown positional and keyword arguments.

1243 Return `-(surrogate * active).sum() / active.sum().clamp_min(1)` to the caller by divide -(surrogate * active).sum() and active.sum().clamp_min(1). No further statements in this call execute.

10.102. train_mappo

Collect daily restricted-view agent requests and shared rewards, estimate per-agent advantages, and run clipped policy/value updates with separate actor and critic optimizers.

Source lines 1246–1264.

```
1246 def train_mappo(config, method='mappo_aircraft', steps=4096, seed=0, rollout_size=128):
1247     """Train decentralized proposals with centralized observed-belief value heads.
1249     Educational adaptation of MAPPO, not an exact benchmark reproduction:
1250     feed-forward actors, four full-rollout epochs, clipped values, no recurrence,
1251     reward/value normalization, learned communication, or hierarchical options.
1252     """
1253     if method not in MAPPO_METHODS or steps < 1 or rollout_size < 2:
1254         raise ValueError('valid MAPPO method, positive steps, and rollout_size >= 2 required')
1255     torch.set_num_threads(1)
1256     torch.manual_seed(seed)
1257     assignment = 'aircraft' if method == 'mappo_aircraft' else 'component'
1258     env = MultiAgentFleetEnv(config, assignment)
1259     model = MultiAgentActorCritic(env.env, assignment)
1260     actor_optimizer = torch.optim.Adam(model.actors.parameters(), lr=3e-4)
1261     critic_optimizer = torch.optim.Adam(model.critic.parameters(), lr=3e-4)
1262     _, info = env.reset(seed=seed)
1263     obs = info['central_observation']
1264     days, episode = 0, 0
```

Line-by-line explanation.

1246 Define `train_mappo(config, method='mappo_aircraft', steps=4096, seed=0, rollout_size=128)`. Collect daily restricted-view agent requests and shared rewards, estimate per-agent advantages, and run clipped policy/value updates with separate actor and critic optimizers.

1247 Documentation text: Train decentralized proposals with centralized observed-belief value heads.. This states the contract or qualification for `train_mappo`; it does not update the simulator or model.

1249 Documentation text: Educational adaptation of MAPPO, not an exact benchmark reproduction:. This states the contract or qualification for `train_mappo`; it does not update the simulator or model.

1250 Documentation text: feed-forward actors, four full-rollout epochs, clipped values, no recurrence,. This states the contract or qualification for `train_mappo`; it does not update the simulator or model.

Documentation text: reward/value normalization, learned communication, or hierarchical options.. This states the contract or qualification for `train_mappo`; it does not update the simulator or model.

Close the documentation string for `train_mappo`. This delimiter ends the descriptive text and performs no simulator or learning update.

Test `method not in MAPPO_METHODS or steps < 1 or rollout_size < 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Raise `ValueError('valid MAPPO method, positive steps, and rollout_size >= 2 required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

Set `assignment` by select 'aircraft' when method == 'mappo_aircraft', otherwise 'component'.

Set `env` by call MultiAgentFleetEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

Set `model` by call MultiAgentActorCritic with the shown positional and keyword arguments. It holds the chosen controller architecture.

Set `actor_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer restricted to actor parameters.

Set `critic_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer for the centralized value estimator.

Set `(_, info)` by initialize a new episode using the stated seed/options.

Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health.

Set `(days, episode)` by construct the shown tuple in its stated order.

Source lines 1265–1282.

```
    logs = []
    while days < steps:
        data = {key: [] for key in ('obs', 'action', 'logp', 'value', 'next_value', 'reward', 'done',
        ↪  'active')}
        while len(data['obs']) < rollout_size and days < steps:
            with torch.no_grad():
                action, logp, _, values = model.act(torch.tensor(obs))
                active = model.active_agents(model.agent_inputs(obs))[0]
            _, rewards, done, truncated, info = env.step(model.action_dict(action[0]))
            next_obs = info['central_observation']
            with torch.no_grad():
                next_value = model.value(next_obs)[0].numpy()
            team_reward = rewards[model.agent_ids[0]]
            terminal = done or truncated
            items = (obs.copy(), action[0].numpy(), logp[0].numpy(), values[0].numpy(), next_value,
            ↪  team_reward, terminal, active.numpy())
            for key, item in zip(data, items):
                data[key].append(item)
            days += 1
            obs = next_obs
```

Line-by-line explanation.

1265 Set `logs` by construct the shown list in its stated order. It holds episode/update diagnostics for later inspection.

1266 Repeat the indented block while `days < steps` remains true; this condition controls the simulation/collection loop here.

1267 Set `data` by construct the collection by iterating the shown comprehension and applying its filters.

1268 Repeat the indented block while `len(data['obs']) < rollout_size and days < steps` remains true; this condition controls the simulation/collection loop here.

1269 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1270 Set `(action, logp, _, values)` by call model.act with the shown positional and keyword arguments.

1271 Set `active` by select the indexed/sliced entries of model.active_agents(model.agent_inputs(obs)).

1272 Set `(_, rewards, done, truncated, info)` by execute this object's step with the shown arguments.

1273 Set `next_obs` by select the indexed/sliced entries of info.

1274 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1275 Set `next_value` by expose this CPU tensor as a NumPy array.

1276 Set `team_reward` by select the indexed/sliced entries of rewards.

1277 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

1278 Set `items` by construct the shown tuple in its stated order.

1279 Iterate `(key, item)` over `zip(data, items)`. The following indented statements run once for each selected element.

1280 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1281 Add `1` to `days`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1282 Set `obs` by evaluate next_obs. It holds the observed belief vector, rather than hidden physical health.

Source lines 1283–1300.

```
1283             if terminal:
1284                 logs.append({'record_type': 'episode', 'episode': episode, 'simulated_days': days,
                     ↪  **info['metrics']})
1285                 episode += 1
1286                 _, info = env.reset(seed=seed + episode)
1287                 obs = info['central_observation']
1288         advantages, targets = mappo_gae(data['reward'], data['value'], data['next_value'],
             ↪  data['done'])
1289         tensors = {key: torch.as_tensor(np.asarray(value)) for key, value in data.items() if key in
             ↪  ('obs', 'action', 'logp', 'value', 'active')}
```

```
        advantages = torch.tensor(advantages)
        targets = torch.tensor(targets)
        active = tensors['active']
        counts = active.sum(0).clamp_min(1)
        mean = (advantages * active).sum(0) / counts
        variance = ((advantages - mean).square() * active).sum(0) / counts
        advantages = (advantages - mean) / torch.sqrt(variance + 1e-8) * active
        for _ in range(4):
            _, newlog, entropy, values = model.act(tensors['obs'], tensors['action'])
            policy_loss = mappo_policy_loss(newlog, tensors['logp'], advantages, active)
            entropy_bonus = (entropy * active).sum() / active.sum().clamp_min(1)
```

Line-by-line explanation.

1283 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1284 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1285 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1286 Set `(_, info)` by initialize a new episode using the stated seed/options.

1287 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health.

1288 Set `(advantages, targets)` by compute advantages with the agent axis retained.

1289 Set `tensors` by construct the collection by iterating the shown comprehension and applying its filters.

1290 Set `advantages` by call torch.tensor with the shown positional and keyword arguments. It holds the policy credit signal with its retained aircraft/agent axis.

1291 Set `targets` by call torch.tensor with the shown positional and keyword arguments.

1292 Set `active` by select the indexed/sliced entries of tensors.

1293 Set `counts` by call active.sum(0).clamp_min with the shown positional and keyword arguments.

1294 Set `mean` by divide (advantages * active).sum(0) and counts.

1295 Set `variance` by divide ((advantages - mean).square() * active).sum(0) and counts.

1296 Set `advantages` by multiply (advantages - mean) / torch.sqrt(variance + 1e-08) and active. It holds the policy credit signal with its retained aircraft/agent axis.

1297 Iterate `_` over `range(4)`. The following indented statements run once for each selected element.

1298 Set `(_, newlog, entropy, values)` by call model.act with the shown positional and keyword arguments.

1299 Set `policy_loss` by clip per-agent PPO ratios before the active-agent average.

1300 Set `entropy_bonus` by divide (entropy * active).sum() and active.sum().clamp_min(1).

Source lines 1301–1318.

```
            clipped_values = tensors['value'] + (values - tensors['value']).clamp(-0.2, 0.2)
            value_loss = torch.maximum((values - targets).square(), (clipped_values -
            ↪  targets).square()).mean()
            actor_loss = policy_loss - 0.001 * entropy_bonus
            if not torch.isfinite(actor_loss + value_loss):
                raise RuntimeError('nonfinite MAPPO loss')
            actor_optimizer.zero_grad()
            actor_loss.backward()
            torch.nn.utils.clip_grad_norm_(model.actors.parameters(), 0.5)
            if bool(active.any()):
                actor_optimizer.step()
            critic_optimizer.zero_grad()
            (0.5 * value_loss).backward()
            torch.nn.utils.clip_grad_norm_(model.critic.parameters(), 0.5)
            critic_optimizer.step()
        logs.append({'record_type': 'update', 'simulated_days': days, 'policy_loss':
        ↪  float(policy_loss.detach()),
                     'value_loss': float(value_loss.detach()), 'entropy':
                     ↪  float(entropy_bonus.detach()),
                     'num_agents': model.num_agents, 'method': method})
    return model, logs, days
```

Line-by-line explanation.

1301 Set `clipped_values` by add tensors['value'] and (values - tensors['value']).clamp(-0.2, 0.2).

1302 Set `value_loss` by average the selected entries.

1303 Set `actor_loss` by subtract policy_loss and 0.001 * entropy_bonus. It holds the objective differentiated through policy parameters.

1304 Test `not torch.isfinite(actor_loss + value_loss)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1305 Raise `RuntimeError('nonfinite MAPPO loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1306 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1307 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

1308 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1309 Test `bool(active.any())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1310 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1311 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1312 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

1313 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1314 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1315 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1316 Continue the expression begun at line 1315 with `'value_loss': float(value_loss.detach()), 'entropy': float(entropy_bonus.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1317 Continue the expression begun at line 1315 with `'num_agents': model.num_agents, 'method': method})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1318 Return `(model, logs, days)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.103. Module declarations and integration

Source lines 1320–1328.

```
OPTION_NAMES = ('wait_one_day', 'targeted_maintenance', 'opportunistic_bundle', 'preventive_bundle')
HRL_METHODS = ('concurrent_options', 'planning_hierarchy', 'aggregate_targets',
               'maxq', 'option_critic', 'hierarchical_vdn', 'hrl_flat')
HRL_PPO_METHODS = ('concurrent_options', 'planning_hierarchy', 'aggregate_targets', 'hrl_flat')
HRL_BUNDLES = (1, 2, 4, 8, 16, 32, 63)
HRL_CATALOG = ((0, 0),) + tuple((b if e & 1 else 0, b if e & 2 else 0)
                                for e in (1, 2, 3) for b in HRL_BUNDLES)
HRL_BUDGETS = (0, 1, 2, 12)
```

Line-by-line explanation.

1320 Set `OPTION_NAMES` by construct the shown tuple in its stated order.

1322 Set `HRL_METHODS` by construct the shown tuple in its stated order.

1323 Continue the assignment begun at line 1322 with `'maxq', 'option_critic', 'hierarchical_vdn', 'hrl_flat')`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1324 Set `HRL_PPO_METHODS` by construct the shown tuple in its stated order.

1325 Set `HRL_BUNDLES` by construct the shown tuple in its stated order.

1326 Set `HRL_CATALOG` by add ((0, 0),) and tuple(((b if e & 1 else 0, b if e & 2 else 0) for e in (1, 2, 3) for b in HRL_BUNDLES)).

1327 Continue the assignment begun at line 1326 with `for e in (1, 2, 3) for b in HRL_BUNDLES)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1328 Set `HRL_BUDGETS` by construct the shown tuple in its stated order.

10.104. hrl_mlp

Construct the common two-hidden-layer neural block used by the new hierarchy implementations.

Source lines 1331–1333.

```
def hrl_mlp(width, output):
    return nn.Sequential(nn.Linear(width, 64), nn.Tanh(), nn.Linear(64, 64),
                         nn.Tanh(), nn.Linear(64, output))
```

Line-by-line explanation.

1331 Define `hrl_mlp(width, output)`. Construct the common two-hidden-layer neural block used by the new hierarchy implementations.

1332 Return `nn.Sequential(nn.Linear(width, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, output))` to the caller by compose the listed layers in execution order. No further statements in this call execute.

1333 Continue the expression begun at line 1332 with `nn.Tanh(), nn.Linear(64, output))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

10.105. hrl_catalog_mask

Build structural feasibility masks for the 22-entry catalog using busy status, observed mandatory grounding and optional replacement-count budgets. Joint stock/resource feasibility remains the broker responsibility.

Source lines 1336–1353.

```
1336 def hrl_catalog_mask(observation, aircraft, local_size=70, mandatory=0.08, goals=None):
1337     """Structural masks only; the shared broker resolves stock/resource competition."""
1338     obs = torch.as_tensor(observation, dtype=torch.float32)
1339     if obs.ndim == 1:
1340         obs = obs.unsqueeze(0)
1341     rows = obs[:, :aircraft * local_size].reshape(-1, aircraft, local_size)
1342     busy = rows[:, :, 37] > 0.5
1343     urgent = rows[:, :, 24:36].amin(-1) <= mandatory
1344     valid = torch.ones((*busy.shape, len(HRL_CATALOG)), dtype=torch.bool, device=obs.device)
1345     valid[:, :, 0] = ~urgent | busy
1346     valid[:, :, 1:] &= ~busy.unsqueeze(-1)
1347     if goals is not None:
1348         goals = torch.as_tensor(goals, device=obs.device, dtype=torch.long)
1349         if goals.shape != busy.shape or torch.any((goals < 0) | (goals >= len(HRL_BUDGETS))):
1350             raise ValueError('one valid budget goal per aircraft required')
1351         counts = torch.tensor([sum(int(b).bit_count() for b in pair) for pair in HRL_CATALOG],
         ↪  device=obs.device)
1352         budgets = torch.tensor(HRL_BUDGETS, device=obs.device)[goals]
1353         valid &= counts <= budgets.unsqueeze(-1)
```

Line-by-line explanation.

1336 Define `hrl_catalog_mask(observation, aircraft, local_size=70, mandatory=0.08, goals=None)`. Build structural feasibility masks for the 22-entry catalog using busy status, observed mandatory grounding and optional replacement-count budgets. Joint stock/resource feasibility remains the broker responsibility.

1337 Documentation text: Structural masks only; the shared broker resolves stock/resource competition.. This states the contract or qualification for `hrl_catalog_mask`; it does not update the simulator or model.

1338 Set `obs` by convert inputs to a tensor with the requested dtype. It holds the observed belief vector, rather than hidden physical health.

1339 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1340 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health.

1341 Set `rows` by reinterpret the shape while retaining element order.

1342 Set `busy` by evaluate the comparison rows[:, :, 37] > 0.5.

1343 Set `urgent` by evaluate the comparison rows[:, :, 24:36].amin(-1) <= mandatory.

1344 Set `valid` by call torch.ones with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

1345 Set `valid[:, :, 0]` by apply a bitwise union to ~urgent and busy. It holds the categorical structural-validity mask.

1346 Bitwise-AND with `~busy.unsqueeze(-1)` in `valid[:, :, 1:]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1347 Test `goals is not None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1348 Set `goals` by convert inputs to a tensor with the requested dtype. It holds per-aircraft upper-level replacement budgets.

1349 Test `goals.shape != busy.shape or torch.any((goals < 0) | (goals >= len(HRL_BUDGETS)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1350 Raise `ValueError('one valid budget goal per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1351 Set `counts` by call torch.tensor with the shown positional and keyword arguments.

1352 Set `budgets` by select the indexed/sliced entries of torch.tensor(HRL_BUDGETS, device=obs.device).

1353 Bitwise-AND with `counts <= budgets.unsqueeze(-1)` in `valid`, and store the result back in that same target. This updates the categorical structural-validity mask.

Source lines 1354–1355.

```
1354            valid[:, :, 0] |= busy
1355        return valid
```

Line-by-line explanation.

1354 Bitwise-OR in `busy` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1355 Return `valid` to the caller by evaluate valid. No further statements in this call execute.

10.106. hrl_categorical

Sample or reconstruct a masked categorical choice. Stored choices are validated so training evaluates the same action under the current parameters.

Source lines 1358–1372.

```
1358 def hrl_categorical(logits, valid, selected=None, deterministic=False):
1359     if logits.shape != valid.shape or not bool(valid.any(-1).all()):
1360         raise ValueError('categorical choices require matching masks with a feasible option')
1361     dist = Categorical(logits=logits.masked_fill(~valid, -1e9))
1362     if selected is None:
1363         selected = dist.probs.argmax(-1) if deterministic else dist.sample()
1364     else:
1365         selected = torch.as_tensor(selected, device=logits.device)
1366         if selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64):
1367             raise ValueError('incorrect hierarchical stored choice shape/dtype')
1368         if torch.any((selected < 0) | (selected >= logits.shape[-1])):
1369             raise ValueError('hierarchical choice outside action categories')
1370         if not bool(valid.gather(-1, selected.unsqueeze(-1)).all()):
1371             raise ValueError('hierarchical choice violates a feasibility mask')
1372     return selected, dist.log_prob(selected), dist.entropy()
```

Line-by-line explanation.

1358 Define `hrl_categorical(logits, valid, selected=None, deterministic=False)`. Sample or reconstruct a masked categorical choice. Stored choices are validated so training evaluates the same action under the current parameters.

1359 Test `logits.shape != valid.shape or not bool(valid.any(-1).all())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1360 Raise `ValueError('categorical choices require matching masks with a feasible option')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1361 Set `dist` by construct a discrete distribution from the supplied logits/probabilities.

1362 Test `selected is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1363 Set `selected` by select dist.probs.argmax(-1) when deterministic, otherwise dist.sample().

1364 Execute this alternative only when the corresponding preceding condition did not select its branch.

1365 Set `selected` by convert inputs to a tensor with the requested dtype.

1366 Test `selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1367 Raise `ValueError('incorrect hierarchical stored choice shape/dtype')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1368 Test `torch.any((selected < 0) | (selected >= logits.shape[-1]))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1369 Raise `ValueError('hierarchical choice outside action categories')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1370 Test `not bool(valid.gather(-1, selected.unsqueeze(-1)).all())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1371 Raise `ValueError('hierarchical choice violates a feasibility mask')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1372 Return `(selected, dist.log_prob(selected), dist.entropy())` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.107. AircraftOptionEnv

One-day concurrent-option wrapper. It keeps individual aircraft jobs committed, records start/completion events, and exposes decision opportunities for free aircraft without waiting for all aircraft to become free.

Source lines 1375–1383.

```
1375 class AircraftOptionEnv:
1376     """Daily fleet interface with independently committed aircraft repair options.
1378     Catalog 0 operates; other entries request one engine, the other, or both,
1379     with one component type or a full six-component bundle. Both engines use
1380     the same bundle if selected. This is a strict subset of the older action
1381     space. Every new method shares this catalog and the same fixed broker.
1382     A new decision for a free aircraft never waits for another aircraft's job.
1383     """
```

Line-by-line explanation.

1375 Define `AircraftOptionEnv` with the stated inheritance. One-day concurrent-option wrapper. It keeps individual aircraft jobs committed, records start/completion events, and exposes decision opportunities for free aircraft without waiting for all aircraft to become free.

1376 Documentation text: Daily fleet interface with independently committed aircraft repair options.. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

1378 Documentation text: Catalog 0 operates; other entries request one engine, the other, or both,. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

1379 Documentation text: with one component type or a full six-component bundle. Both engines use. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

1380 Documentation text: the same bundle if selected. This is a strict subset of the older action. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

1381 Documentation text: space. Every new method shares this catalog and the same fixed broker.. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

1382 Documentation text: A new decision for a free aircraft never waits for another aircraft's job.. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

1383 Close the documentation string for `AircraftOptionEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.108. AircraftOptionEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1385–1391.

```
1385    def __init__(self, config=None):
1386        self.adapter = MultiAgentFleetEnv(config, 'aircraft')
1387        self.env = self.adapter.env
1388        self.n = self.env.config.aircraft
1389        self.active_options = {}
1390        self.option_history = []
1391        self.central_observation = None
```

Line-by-line explanation.

1385 Define `__init__(self, config=None)`. This ___init___ method is part of AircraftOptionEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

1386 Set `self.adapter` by call MultiAgentFleetEnv with the shown positional and keyword arguments.

1387 Set `self.env` by evaluate self.adapter.env. It holds the environment or wrapper that owns physical transitions.

1388 Set `self.n` by evaluate self.env.config.aircraft.

1389 Set `self.active_options` by construct the shown dict in its stated order.

1390 Set `self.option_history` by construct the shown list in its stated order.

1391 Set `self.central_observation` by use the literal None.

10.109. AircraftOptionEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 1393–1399.

```
1393    def reset(self, *, seed=None, options=None):
1394        _, info = self.adapter.reset(seed=seed, options=options)
1395        self.central_observation = info['central_observation']
1396        self.active_options.clear()
1397        self.option_history.clear()
1398        self.last_info = info
1399        return self.central_observation.copy(), info
```

Line-by-line explanation.

1393 Define `reset(self, *, seed=None, options=None)`. This reset method is part of AircraftOptionEnv.reset; its parameters and defaults are the interface used by the calls in this section.

1394 Set `(_, info)` by initialize a new episode using the stated seed/options.

1395 Set `self.central_observation` by select the indexed/sliced entries of info.

1396 Call self.active_options.clear with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1397 Call self.option_history.clear with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1398 Set `self.last_info` by evaluate info.

1399 Return `(self.central_observation.copy(), info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.110. AircraftOptionEnv.decision_mask

Return one Boolean per aircraft indicating that it is free to receive a new plan. This permission does not certify joint resource feasibility.

Source lines 1401–1405.

```
1401     def decision_mask(self):
1402         if self.central_observation is None:
1403             raise RuntimeError('reset before reading aircraft options')
1404         rows, _ = self.env.split_observation(self.central_observation)
1405         return rows[:, 37] <= 0.5
```

Line-by-line explanation.

1401 Define `decision_mask(self)`. Return one Boolean per aircraft indicating that it is free to receive a new plan. This permission does not certify joint resource feasibility.

1402 Test `self.central_observation is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1403 Raise `RuntimeError('reset before reading aircraft options')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1404 Set `(rows, _)` by call self.env.split_observation with the shown positional and keyword arguments.

1405 Return `rows[:, 37] <= 0.5` to the caller by evaluate the comparison rows[:, 37] <= 0.5. No further statements in this call execute.

10.111. AircraftOptionEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 1407–1424.

```
1407     def step(self, choices):
1408         choices = np.asarray(choices)
1409         if choices.shape != (self.n,) or not np.issubdtype(choices.dtype, np.integer):
1410             raise ValueError('one integer catalog action per aircraft required')
1411         if np.any((choices < 0) | (choices >= len(HRL_CATALOG))):
```

```
            raise ValueError('unknown maintenance catalog action')
        free = self.decision_mask()
        if np.any((~free) & (choices != 0)):
            raise ValueError('committed aircraft options accept no new planning request')
        previous = {i: job.plan for i, job in self.env.jobs.items()}
        day = self.env.day
        proposals = {plate: np.asarray(HRL_CATALOG[int(choices[i])], dtype=np.int64)
                     for i, plate in enumerate(self.env.aircraft_plates)}
        _, rewards, done, truncated, info = self.adapter.step(proposals)
        for i, plan in previous.items():
            if i in self.env.jobs and self.env.jobs[i].plan != plan:
                raise RuntimeError('a committed aircraft plan was changed')
        started, finished = [], []
```

Line-by-line explanation.

1407 Define `step(self, choices)`. This step method is part of AircraftOptionEnv.step; its parameters and defaults are the interface used by the calls in this section.

1408 Set `choices` by call np.asarray with the shown positional and keyword arguments.

1409 Test `choices.shape != (self.n,) or not np.issubdtype(choices.dtype, np.integer)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1410 Raise `ValueError('one integer catalog action per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1411 Test `np.any((choices < 0) | (choices >= len(HRL_CATALOG)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1412 Raise `ValueError('unknown maintenance catalog action')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1413 Set `free` by call self.decision_mask with the shown positional and keyword arguments. It holds the current per-aircraft permission to issue a new plan.

1414 Test `np.any(~free & (choices != 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1415 Raise `ValueError('committed aircraft options accept no new planning request')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1416 Set `previous` by construct the collection by iterating the shown comprehension and applying its filters.

1417 Set `day` by evaluate self.env.day.

1418 Set `proposals` by construct the collection by iterating the shown comprehension and applying its filters.

1419 Continue the assignment begun at line 1418 with `for i, plate in enumerate(self.env.aircraft_plates)}`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1420 Set `(_, rewards, done, truncated, info)` by execute this object's step with the shown arguments.

1421 Iterate `(i, plan)` over `previous.items()`. The following indented statements run once for each selected element.

1422 Test `i in self.env.jobs and self.env.jobs[i].plan != plan`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1423 Raise `RuntimeError('a committed aircraft plan was changed')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1424 Set `(started, finished)` by construct the shown tuple in its stated order.

Source lines 1425–1442.

```
1425         for i, job in self.env.jobs.items():
1426             if i not in previous:
1427                 record = {'aircraft': i, 'aircraft_plate': self.env.aircraft_plates[i],
1428                           'start_day': day, 'catalog_action': int(choices[i]),
1429                           'planned_duration': job.total_duration}
1430                 self.active_options[i] = record
1431                 started.append(dict(record))
1432         for i in info['completed']:
1433             record = dict(self.active_options.pop(i))
1434             record.update(end_day=self.env.day, duration=self.env.day - record['start_day'])
1435             self.option_history.append(record)
1436             finished.append(record)
1437         self.central_observation = info['central_observation']
1438         info.update(aircraft_option_started=started, aircraft_option_completed=finished,
1439                     decision_mask_before=free.tolist(),
                        ↪  active_aircraft_options=sorted(self.active_options))
1440         info['metrics'].update(option_starts=len(self.option_history) + len(self.active_options),
1441                                option_completions=len(self.option_history))
1442         self.last_info = info
```

Line-by-line explanation.

1425 Iterate `(i, job)` over `self.env.jobs.items()`. The following indented statements run once for each selected element.

1426 Test `i not in previous`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1427 Set `record` by construct the shown dict in its stated order.

1428 Continue the assignment begun at line 1427 with `'start_day': day, 'catalog_action': int(choices[i]),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1429 Continue the assignment begun at line 1427 with `'planned_duration': job.total_duration}`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1430 Set `self.active_options[i]` by evaluate record.

1431 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1432 Iterate `i` over `info['completed']`. The following indented statements run once for each selected element.

1433 Set `record` by call dict with the shown positional and keyword arguments.

1434 Call record.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1435 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1436 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1437 Set `self.central_observation` by select the indexed/sliced entries of info.

1438 Call info.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1439 Continue the expression begun at line 1438 with `decision_mask_before=free.tolist(), active_aircraft_options=sorted(self.active_options))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1440 Call info['metrics'].update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1441 Continue the expression begun at line 1440 with `option_completions=len(self.option_history))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1442 Set `self.last_info` by evaluate info.

Source lines 1443–1443.

```
1443         return self.central_observation.copy(), rewards[self.env.aircraft_plates[0]], done, truncated,
             ↪  info
```

Line-by-line explanation.

1443 Return `(self.central_observation.copy(), rewards[self.env.aircraft_plates[0]], done, truncated, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.112. AircraftPlanningHierarchy

Shared stage policies for concurrent options, three-stage planning, aggregate targets and the catalog flat comparator. It produces per-aircraft decision likelihoods and a centralized value vector. The catalog request is still executed by the fixed broker.

Source lines 1446–1452.

```
1446 class AircraftPlanningHierarchy(nn.Module):
1447     """Independent aircraft options with learned planning stages or explicit targets.
1449     Ground/engine/bundle selections are computational planning stages, not
1450     three elapsed physical epochs. Temporal abstraction is the committed job.
1451     All actors use observed global context; this is not local-information CTDE.
1452     """
```

Line-by-line explanation.

1446 Define `AircraftPlanningHierarchy` with the stated inheritance. Shared stage policies for concurrent options, three-stage planning, aggregate targets and the catalog flat comparator. It produces per-aircraft decision likelihoods and a centralized value vector. The catalog request is still executed by the fixed broker.

1447 Documentation text: Independent aircraft options with learned planning stages or explicit targets.. This states the contract or qualification for `AircraftPlanningHierarchy`; it does not update the simulator or model.

1449 Documentation text: Ground/engine/bundle selections are computational planning stages, not. This states the contract or qualification for `AircraftPlanningHierarchy`; it does not update the simulator or model.

1450 Documentation text: three elapsed physical epochs. Temporal abstraction is the committed job.. This states the contract or qualification for `AircraftPlanningHierarchy`; it does not update the simulator or model.

1451 Documentation text: All actors use observed global context; this is not local-information CTDE.. This states the contract or qualification for `AircraftPlanningHierarchy`; it does not update the simulator or model.

1452 Close the documentation string for `AircraftPlanningHierarchy`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.113. AircraftPlanningHierarchy.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1454–1471.

```
1454     def __init__(self, env, mode='planning_hierarchy'):
1455         super().__init__()
1456         if mode not in HRL_PPO_METHODS:
1457             raise ValueError('unknown aircraft planning hierarchy')
1458         self.mode = mode
1459         self.n = env.config.aircraft
1460         self.local = env.local_size
1461         self.width = env.observation_space.shape[0]
1462         self.mandatory = env.config.mandatory_health
1463         features = self.width + self.local
1464         if mode == 'hrl_flat':
1465             self.flat_policy = hrl_mlp(features, len(HRL_CATALOG))
1466         else:
1467             self.manager = hrl_mlp(features, 4 if mode == 'aggregate_targets' else 2)
1468             if mode == 'planning_hierarchy':
1469                 self.engine_policy = hrl_mlp(features + 2, 3)
1470                 self.component_policy = hrl_mlp(features + 5, 7)
1471             if mode == 'aggregate_targets':
```

Line-by-line explanation.

1454 Define `__init__(self, env, mode='planning_hierarchy')`. This ___init___ method is part of AircraftPlanningHierarchy.___init___; its parameters and defaults are the interface used by the calls in this section.

1455 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1456 Test `mode not in HRL_PPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1457 Raise `ValueError('unknown aircraft planning hierarchy')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1458 Set `self.mode` by evaluate mode.

1459 Set `self.n` by evaluate env.config.aircraft.

1460 Set `self.local` by evaluate env.local_size.

1461 Set `self.width` by select the indexed/sliced entries of env.observation_space.shape.

1462 Set `self.mandatory` by evaluate env.config.mandatory_health.

1463 Set `features` by add self.width and self.local.

1464 Test `mode == 'hrl_flat'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1465 Set `self.flat_policy` by call hrl_mlp with the shown positional and keyword arguments.

1466 Execute this alternative only when the corresponding preceding condition did not select its branch.

1467 Set `self.manager` by call hrl_mlp with the shown positional and keyword arguments.

1468 Test `mode == 'planning_hierarchy'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1469 Set `self.engine_policy` by call hrl_mlp with the shown positional and keyword arguments.

1470 Set `self.component_policy` by call hrl_mlp with the shown positional and keyword arguments.

1471 Test `mode == 'aggregate_targets'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 1472–1473.

```
1472                 self.worker = hrl_mlp(features + 4, len(HRL_CATALOG))
1473         self.critic = hrl_mlp(self.width, self.n)
```

Line-by-line explanation.

1472 Set `self.worker` by call hrl_mlp with the shown positional and keyword arguments.

1473 Set `self.critic` by call hrl_mlp with the shown positional and keyword arguments.

10.114. AircraftPlanningHierarchy.features

Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

Source lines 1475–1479.

```
1475     def features(self, observation):
1476         obs = _fleet_observation_tensor(observation, self.width,
1477                                         'incorrect hierarchical fleet observation')
1478         local = obs[:, :self.n * self.local].reshape(-1, self.n, self.local)
1479         return obs, local, torch.cat([obs[:, None].expand(-1, self.n, -1), local], -1)
```

Line-by-line explanation.

1475 Define `features(self, observation)`. Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

1476 Validate and batch the global observation through the shared helper. This preserves the expected width and caller-specific error while avoiding actor-context allocation.

1477 Continue the assignment begun at line 1476 with `'incorrect hierarchical fleet observation')`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1478 Set `local` by reinterpret the shape while retaining element order.

1479 Return `(obs, local, torch.cat([obs[:, None].expand(-1, self.n, -1), local], -1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.115. AircraftPlanningHierarchy.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1481–1485.

```
1481     def value(self, observation):
1482         # A critic needs global inputs, not the repeated actor-context tensor.
1483         obs = _fleet_observation_tensor(observation, self.width,
1484                                         'incorrect hierarchical fleet observation')
1485         return self.critic(obs)
```

Line-by-line explanation.

1481 Define `value(self, observation)`. This value method is part of AircraftPlanningHierarchy.value; its parameters and defaults are the interface used by the calls in this section.

1482 Source comment: A critic needs global inputs, not the repeated actor-context tensor. This line documents the following code and does not execute.

1483 Validate and batch the global observation through the shared helper. This preserves the expected width and caller-specific error while avoiding actor-context allocation.

1484 Continue the assignment begun at line 1483 with `'incorrect hierarchical fleet observation')`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1485 Evaluate the existing centralized value network directly from the validated observation. Actor features are unnecessary for this call.

10.116. AircraftPlanningHierarchy.actor_parameters

Expose only the parameters updated by the actor optimizer, excluding the separately optimized value estimator.

Source lines 1487–1488.

```
1487     def actor_parameters(self):
1488         return [p for name, p in self.named_parameters() if not name.startswith('critic.')]
```

Line-by-line explanation.

1487 Define `actor_parameters(self)`. This actor_parameters method is part of AircraftPlanningHierarchy.actor_parameters; its parameters and defaults are the interface used by the calls in this section.

1488 Return `[p for name, p in self.named_parameters() if not name.startswith('critic.')]` to the caller by construct the collection by iterating the shown comprehension and applying its filters. No further statements in this call execute.

10.117. AircraftPlanningHierarchy.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 1490–1507.

```
1490     def act(self, observation, actions=None, deterministic=False):
1491         obs, local, features = self.features(observation)
1492         if actions is not None:
1493             actions = torch.as_tensor(actions)
1494             if actions.shape != (len(obs), self.n, 3) or actions.dtype not in (torch.int32,
                 ↪  torch.int64):
1495                 raise ValueError('hierarchical actions must be integer batch x aircraft x 3')
1496         mask = hrl_catalog_mask(obs, self.n, self.local, self.mandatory)
1497         busy = local[:, :, 37] > 0.5
1498         urgent = local[:, :, 24:36].amin(-1) <= self.mandatory
1499         zero = torch.zeros_like(busy, dtype=torch.long)
1500         if self.mode == 'hrl_flat':
1501             choice, logp, entropy = hrl_categorical(self.flat_policy(features), mask,
1502                 None if actions is None else actions[:, :, 0], deterministic)
1503             stored = torch.stack([choice, zero, zero], -1)
1504         else:
1505             count = 4 if self.mode == 'aggregate_targets' else 2
1506             valid = (~busy).unsqueeze(-1).expand(-1, -1, count).clone()
1507             valid[:, :, 0] = ~urgent | busy
```

Line-by-line explanation.

1490 Define `act(self, observation, actions=None, deterministic=False)`. This act method is part of AircraftPlanningHierarchy.act; its parameters and defaults are the interface used by the calls in this section.

1491 Set `(obs, local, features)` by call self.features with the shown positional and keyword arguments.

1492 Test `actions is not None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1493 Set `actions` by convert inputs to a tensor with the requested dtype.

1494 Test `actions.shape != (len(obs), self.n, 3) or actions.dtype not in (torch.int32, torch.int64)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1495 Raise `ValueError('hierarchical actions must be integer batch x aircraft x 3')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1496 Set `mask` by compute permitted catalog entries under commitment and optional budget constraints.

1497 Set `busy` by evaluate the comparison local[:, :, 37] > 0.5.

1498 Set `urgent` by evaluate the comparison local[:, :, 24:36].amin(-1) <= self.mandatory.

1499 Set `zero` by call torch.zeros_like with the shown positional and keyword arguments.

1500 Test `self.mode == 'hrl_flat'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1501 Set `(choice, logp, entropy)` by sample or score a structurally masked categorical decision.

1502 Continue the assignment begun at line 1501 with `None if actions is None else actions[:, :, 0], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1503 Set `stored` by join equal-shaped tensors along the specified new axis.

1504 Execute this alternative only when the corresponding preceding condition did not select its branch.

1505 Set `count` by select 4 when self.mode == 'aggregate_targets', otherwise 2.

1506 Set `valid` by call (~busy).unsqueeze(-1).expand(-1, -1, count).clone with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

1507 Set `valid[:, :, 0]` by apply a bitwise union to ~urgent and busy. It holds the categorical structural-validity mask.

Source lines 1508–1525.

```
            goal, logp, entropy = hrl_categorical(self.manager(features), valid,
                None if actions is None else actions[:, :, 0], deterministic)
            if self.mode == 'planning_hierarchy':
                active = (goal == 1) & ~busy
                engine_valid = active.unsqueeze(-1).expand(-1, -1, 3).clone()
                engine_valid[:, :, 0] |= ~active
                engine, lp, ent = hrl_categorical(self.engine_policy(torch.cat([
                    features, nn.functional.one_hot(goal, 2).float()], -1)), engine_valid,
                    None if actions is None else actions[:, :, 1], deterministic)
                logp, entropy = logp + lp, entropy + ent
                bundle_valid = active.unsqueeze(-1).expand(-1, -1, 7).clone()
                bundle_valid[:, :, 0] |= ~active
                bundle, lp, ent = hrl_categorical(self.component_policy(torch.cat([
                    features, nn.functional.one_hot(goal, 2).float(),
                    nn.functional.one_hot(engine, 3).float()], -1)), bundle_valid,
                    None if actions is None else actions[:, :, 2], deterministic)
                logp, entropy = logp + lp, entropy + ent
                stored = torch.stack([goal, engine, bundle], -1)
```

Line-by-line explanation.

`1508` Set `(goal, logp, entropy)` by sample or score a structurally masked categorical decision.

`1509` Continue the assignment begun at line 1508 with `None if actions is None else actions[:, :, 0], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

`1510` Test `self.mode == 'planning_hierarchy'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`1511` Set `active` by apply a bitwise intersection to goal == 1 and ~busy.

`1512` Set `engine_valid` by call active.unsqueeze(-1).expand(-1, -1, 3).clone with the shown positional and keyword arguments.

`1513` Bitwise-OR in `~active` in `engine_valid[:, :, 0]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

`1514` Set `(engine, lp, ent)` by sample or score a structurally masked categorical decision.

`1515` Continue the assignment begun at line 1514 with `features, nn.functional.one_hot(goal, 2).float()], -1)), engine_valid,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

`1516` Continue the assignment begun at line 1514 with `None if actions is None else actions[:, :, 1], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

`1517` Set `(logp, entropy)` by construct the shown tuple in its stated order.

`1518` Set `bundle_valid` by call active.unsqueeze(-1).expand(-1, -1, 7).clone with the shown positional and keyword arguments.

`1519` Bitwise-OR in `~active` in `bundle_valid[:, :, 0]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

`1520` Set `(bundle, lp, ent)` by sample or score a structurally masked categorical decision.

`1521` Continue the assignment begun at line 1520 with `features, nn.functional.one_hot(goal, 2).float(),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

`1522` Continue the assignment begun at line 1520 with `nn.functional.one_hot(engine, 3).float()], -1)), bundle_valid,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

`1523` Continue the assignment begun at line 1520 with `None if actions is None else actions[:, :, 2], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

`1524` Set `(logp, entropy)` by construct the shown tuple in its stated order.

`1525` Set `stored` by join equal-shaped tensors along the specified new axis.

Source lines 1526–1539.

```
            elif self.mode == 'aggregate_targets':
                # A positive goal is a maximum replacement count, not a guaranteed improvement.
                worker_valid = hrl_catalog_mask(obs, self.n, self.local, self.mandatory, goal)
                worker_valid[:, :, 0] = (goal == 0) | busy
                choice, lp, ent = hrl_categorical(self.worker(torch.cat([
                    features, nn.functional.one_hot(goal, 4).float()], -1)), worker_valid,
                    None if actions is None else actions[:, :, 1], deterministic)
                logp, entropy = logp + lp, entropy + ent
                stored = torch.stack([goal, choice, zero], -1)
            else:
                stored = torch.stack([goal, zero, zero], -1)
        if actions is not None and not torch.equal(stored, actions):
            raise ValueError('unused hierarchical action fields must be zero')
        return stored, logp, entropy, self.value(obs)
```

Line-by-line explanation.

1526 Test `self.mode == 'aggregate_targets'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1527 Source comment: A positive goal is a maximum replacement count, not a guaranteed improvement. This line documents the following code and does not execute.

1528 Set `worker_valid` by compute permitted catalog entries under commitment and optional budget constraints.

1529 Set `worker_valid[:, :, 0]` by apply a bitwise union to goal == 0 and busy.

1530 Set `(choice, lp, ent)` by sample or score a structurally masked categorical decision.

1531 Continue the assignment begun at line 1530 with `features, nn.functional.one_hot(goal, 4).float()], -1)), worker_valid,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1532 Continue the assignment begun at line 1530 with `None if actions is None else actions[:, :, 1], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1533 Set `(logp, entropy)` by construct the shown tuple in its stated order.

1534 Set `stored` by join equal-shaped tensors along the specified new axis.

1535 Execute this alternative only when the corresponding preceding condition did not select its branch.

1536 Set `stored` by join equal-shaped tensors along the specified new axis.

1537 Test `actions is not None and (not torch.equal(stored, actions))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1538 Raise `ValueError('unused hierarchical action fields must be zero')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1539 Return `(stored, logp, entropy, self.value(obs))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.118. AircraftPlanningHierarchy.catalog_actions

Convert the stored hierarchical planning choices into the catalog indices consumed by the common option wrapper.

Source lines 1541–1558.

```
1541    def catalog_actions(self, observation, actions):
1542        actions = torch.as_tensor(actions)
1543        if actions.ndim == 3:
1544            if actions.shape[0] != 1:
1545                raise ValueError('decode one fleet action at a time')
1546            actions = actions[0]
1547        if actions.shape != (self.n, 3):
1548            raise ValueError('incorrect hierarchical action for decoding')
1549        if self.mode == 'hrl_flat':
1550            return actions[:, 0].numpy().copy()
1551        if self.mode == 'aggregate_targets':
1552            return actions[:, 1].numpy().copy()
1553        if self.mode == 'planning_hierarchy':
1554            return torch.where(actions[:, 0] != 0, 1 + actions[:, 1] * 7 + actions[:, 2], 0).numpy()
1555        _, local, _ = self.features(observation)
1556        means = local[0, :, :12].reshape(self.n, 2, 6)
1557        engine = means.amin(-1).argmin(-1)
1558        selected = means[torch.arange(self.n), engine]
```

Line-by-line explanation.

1541 Define `catalog_actions(self, observation, actions)`. This catalog_actions method is part of AircraftPlanningHierarchy.catalog_actions; its parameters and defaults are the interface used by the calls in this section.

1542 Set `actions` by convert inputs to a tensor with the requested dtype.

1543 Test `actions.ndim == 3`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1544 Test `actions.shape[0] != 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1545 Raise `ValueError('decode one fleet action at a time')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1546 Set `actions` by select the indexed/sliced entries of actions.

1547 Test `actions.shape != (self.n, 3)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1548 Raise `ValueError('incorrect hierarchical action for decoding')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1549 Test `self.mode == 'hrl_flat'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1550 Return `actions[:, 0].numpy().copy()` to the caller by make a separate array/dictionary copy. No further statements in this call execute.

1551 Test `self.mode == 'aggregate_targets'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1552 Return `actions[:, 1].numpy().copy()` to the caller by make a separate array/dictionary copy. No further statements in this call execute.

1553 Test `self.mode == 'planning_hierarchy'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1554 Return `torch.where(actions[:, 0] != 0, 1 + actions[:, 1] * 7 + actions[:, 2], 0).numpy()` to the caller by expose this CPU tensor as a NumPy array. No further statements in this call execute.

1555 Set `(_, local, _)` by call self.features with the shown positional and keyword arguments.

1556 Set `means` by reinterpret the shape while retaining element order.

1557 Set `engine` by call means.amin(-1).argmin with the shown positional and keyword arguments.

1558 Set `selected` by select the indexed/sliced entries of means.

Source lines 1559–1561.

```
        component = selected.argmin(-1)
        bundle = torch.where((selected < .75).sum(-1) >= 3, 6, component)
        return torch.where(actions[:, 0] != 0, 1 + engine * 7 + bundle, 0).numpy()
```

Line-by-line explanation.

1559 Set `component` by call selected.argmin with the shown positional and keyword arguments.

1560 Set `bundle` by select values elementwise using the Boolean condition.

1561 Return `torch.where(actions[:, 0] != 0, 1 + engine * 7 + bundle, 0).numpy()` to the caller by expose this CPU tensor as a NumPy array. No further statements in this call execute.

10.119. hrl_option_target

Compute the discounted sum of primitive daily rewards and a bootstrap discounted by the actual option duration; a genuine terminal state has no bootstrap.

Source lines 1564–1567.

```
def hrl_option_target(rewards, next_value, terminal, gamma=.99):
    """Discount each primitive day once, including bootstrap over the actual duration."""
    return sum(gamma ** t * float(r) for t, r in enumerate(rewards)) + (
        0. if terminal else gamma ** len(rewards) * float(next_value))
```

Line-by-line explanation.

1564 Define `hrl_option_target(rewards, next_value, terminal, gamma=.99)`. Compute the discounted sum of primitive daily rewards and a bootstrap discounted by the actual option duration; a genuine terminal state has no bootstrap.

1565 Documentation text: Discount each primitive day once, including bootstrap over the actual duration.. This states the contract or qualification for `hrl_option_target`; it does not update the simulator or model.

1566 Return `sum((gamma ** t * float(r) for t, r in enumerate(rewards))) + (0.0 if terminal else gamma ** len(rewards) * float(next_value))` to the caller by add sum((gamma ** t * float(r) for t, r in enumerate(rewards))) and 0.0 if terminal else gamma ** len(rewards) * float(next_value). No further statements in this call execute.

1567 Continue the expression begun at line 1566 with `0. if terminal else gamma ** len(rewards) * float(next_value))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

10.120. train_aircraft_hrl

Keep a pending decision record per aircraft, append daily team rewards until that decision completes, and update stage actors from duration-aware decision targets. The critic also learns from daily targets. A budget cutoff bootstraps unfinished work without canceling it.

Source lines 1570–1587.

```
1570 def train_aircraft_hrl(config, method='planning_hierarchy', steps=4096, seed=0, rollout_size=128):
1571     if method not in HRL_PPO_METHODS or steps < 1 or rollout_size < 2:
1572         raise ValueError('registered hierarchy, positive steps, rollout_size >= 2 required')
1573     torch.set_num_threads(1)
1574     torch.manual_seed(seed)
1575     env = AircraftOptionEnv(config)
1576     model = AircraftPlanningHierarchy(env.env, method)
1577     actors = model.actor_parameters()
1578     actor_optimizer = torch.optim.Adam(actors, lr=3e-4)
1579     critic_optimizer = torch.optim.Adam(model.critic.parameters(), lr=3e-4)
1580     obs, _ = env.reset(seed=seed)
1581     pending, ready, daily, logs = {}, [], [], []
1582     episode = 0
1583     for day in range(1, steps + 1):
1584         with torch.no_grad():
1585             action, logp, _, values = model.act(obs)
1586         free = env.decision_mask()
1587         for i in np.flatnonzero(free):
```

Line-by-line explanation.

1570 Define `train_aircraft_hrl(config, method='planning_hierarchy', steps=4096, seed=0, rollout_size=128)`. Keep a pending decision record per aircraft, append daily team rewards until that decision completes, and update stage actors from duration-aware decision targets. The critic also learns from daily targets. A budget cutoff bootstraps unfinished work without canceling it.

1571 Test `method not in HRL_PPO_METHODS or steps < 1 or rollout_size < 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1572 Raise `ValueError('registered hierarchy, positive steps, rollout_size >= 2 required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1573 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

1574 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

1575 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

1576 Set `model` by call AircraftPlanningHierarchy with the shown positional and keyword arguments. It holds the chosen controller architecture.

1577 Set `actors` by call model.actor_parameters with the shown positional and keyword arguments.

1578 Set `actor_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer restricted to actor parameters.

1579 Set `critic_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer for the centralized value estimator.

1580 Set `(obs, _)` by initialize a new episode using the stated seed/options.

1581 Set `(pending, ready, daily, logs)` by construct the shown tuple in its stated order.

1582 Set `episode` by use the literal 0.

1583 Iterate `day` over `range(1, steps + 1)`. The following indented statements run once for each selected element.

1584 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1585 Set `(action, logp, _, values)` by call model.act with the shown positional and keyword arguments.

1586 Set `free` by call env.decision_mask with the shown positional and keyword arguments. It holds the current per-aircraft permission to issue a new plan.

1587 Iterate `i` over `np.flatnonzero(free)`. The following indented statements run once for each selected element.

Source lines 1588–1605.

```
            pending[int(i)] = {'obs': obs.copy(), 'action': action[0].numpy().copy(),
                'logp': float(logp[0, i]), 'value': float(values[0, i]), 'aircraft': int(i),
                ↪  'rewards': []}
        following, reward, done, truncated, info = env.step(model.catalog_actions(obs, action))
        terminal = done or truncated
        with torch.no_grad():
            next_values = model.value(following)[0].numpy()
        daily.append((obs.copy(), reward + .99 * (not terminal) * next_values))
        for i in list(pending):
            pending[i]['rewards'].append(reward)
            completed = terminal or bool(env.decision_mask()[i])
            # A budget cutoff bootstraps an unfinished option; it does not cancel its job.
            if completed or day == steps:
                item = pending.pop(i)
                item.update(target=hrl_option_target(item['rewards'], next_values[i], terminal),
                            duration=len(item['rewards']), option_complete=completed)
                ready.append(item)
        obs = following
        if terminal:
```

Line-by-line explanation.

1588 Set `pending[int(i)]` by construct the shown dict in its stated order. It holds unfinished per-aircraft decision records spanning primitive days.

1589 Continue the assignment begun at line 1588 with `'logp': float(logp[0, i]), 'value': float(values[0, i]), 'aircraft': int(i), 'rewards': []}`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1590 Set `(following, reward, done, truncated, info)` by execute this object's step with the shown arguments.

1591 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

1592 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1593 Set `next_values` by expose this CPU tensor as a NumPy array. It holds the bootstrap value estimate at the following observation.

1594 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1595 Iterate `i` over `list(pending)`. The following indented statements run once for each selected element.

1596 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1597 Set `completed` by evaluate terminal or bool(env.decision_mask()[i]).

1598 Source comment: A budget cutoff bootstraps an unfinished option; it does not cancel its job. This line documents the following code and does not execute.

1599 Test `completed or day == steps`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1600 Set `item` by remove and return the selected stored record.

1601 Call item.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1602 Continue the expression begun at line 1601 with `duration=len(item['rewards']), option_complete=completed)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1603 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1604 Set `obs` by evaluate following. It holds the observed belief vector, rather than hidden physical health.

1605 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 1606–1623.

```
            logs.append({'record_type': 'episode', 'simulated_days': day, **info['metrics']})
            episode += 1
            obs, _ = env.reset(seed=seed + episode)
        if day % rollout_size == 0 or day == steps:
            if ready:
                states = torch.tensor(np.stack([r['obs'] for r in ready]))
                actions = torch.tensor(np.stack([r['action'] for r in ready]))
```

```
                ids = torch.tensor([r['aircraft'] for r in ready])
                batch_ids = torch.arange(len(ready))
                old_logp = torch.tensor([r['logp'] for r in ready])
                advantages = torch.tensor([r['target'] - r['value'] for r in ready])
                if len(ready) > 1:
                    advantages = (advantages - advantages.mean()) /
                    ↪  advantages.std(unbiased=False).clamp_min(1e-6)
                for _ in range(4):
                    _, new_logp, entropy, _ = model.act(states, actions)
                    ratio = torch.exp(new_logp[batch_ids, ids] - old_logp)
                    loss = -torch.minimum(ratio * advantages, ratio.clamp(.8, 1.2) * advantages).mean()
                    loss -= .001 * entropy[batch_ids, ids].mean()
```

Line-by-line explanation.

1606 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1607 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1608 Set `(obs, _)` by initialize a new episode using the stated seed/options.

1609 Test `day % rollout_size == 0 or day == steps`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1610 Test `ready`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1611 Set `states` by call torch.tensor with the shown positional and keyword arguments.

1612 Set `actions` by call torch.tensor with the shown positional and keyword arguments.

1613 Set `ids` by call torch.tensor with the shown positional and keyword arguments.

1614 Set `batch_ids` by call torch.arange with the shown positional and keyword arguments.

1615 Set `old_logp` by call torch.tensor with the shown positional and keyword arguments.

1616 Set `advantages` by call torch.tensor with the shown positional and keyword arguments. It holds the policy credit signal with its retained aircraft/agent axis.

1617 Test `len(ready) > 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1618 Set `advantages` by divide advantages - advantages.mean() and advantages.std(unbiased=False).clamp_min(1e-06). It holds the policy credit signal with its retained aircraft/agent axis.

1619 Iterate `_` over `range(4)`. The following indented statements run once for each selected element.

1620 Set `(_, new_logp, entropy, _)` by call model.act with the shown positional and keyword arguments.

1621 Set `ratio` by exponentiate log values, including log-probability ratios. It holds the current-to-behavior probability ratio.

1622 Set `loss` by evaluate -torch.minimum(ratio * advantages, ratio.clamp(0.8, 1.2) * advantages).mean().

1623 Subtract `0.001 * entropy[batch_ids, ids].mean()` from `loss`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Source lines 1624–1641.

```
                    if not bool(torch.isfinite(loss)):
                        raise RuntimeError('nonfinite hierarchical policy loss')
                    actor_optimizer.zero_grad()
                    loss.backward()
                    torch.nn.utils.clip_grad_norm_(actors, .5)
                    actor_optimizer.step()
                logs.append({'record_type': 'option_update', 'method': method, 'simulated_days': day,
                    'decisions': len(ready), 'durations': [r['duration'] for r in ready],
                    'complete': [r['option_complete'] for r in ready], 'policy_loss':
                    ↪  float(loss.detach())})
                ready.clear()
            states = torch.tensor(np.stack([row[0] for row in daily]))
            targets = torch.tensor(np.stack([row[1] for row in daily]))
            for _ in range(4):
                value_loss = .5 * (model.value(states) - targets).square().mean()
                if not bool(torch.isfinite(value_loss)):
                    raise RuntimeError('nonfinite hierarchical value loss')
                critic_optimizer.zero_grad()
                value_loss.backward()
```

Line-by-line explanation.

1624 Test `not bool(torch.isfinite(loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1625 Raise `RuntimeError('nonfinite hierarchical policy loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1626 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1627 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

1628 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1629 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1630 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1631 Continue the expression begun at line 1630 with `'decisions': len(ready), 'durations': [r['duration'] for r in ready],`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1632 Continue the expression begun at line 1630 with `'complete': [r['option_complete'] for r in ready], 'policy_loss': float(loss.detach())})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1633 Call ready.clear with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1634 Set `states` by call torch.tensor with the shown positional and keyword arguments.

1635 Set `targets` by call torch.tensor with the shown positional and keyword arguments.

1636 Iterate `_` over `range(4)`. The following indented statements run once for each selected element.

1637 Set `value_loss` by multiply 0.5 and (model.value(states) - targets).square().mean().

1638 Test `not bool(torch.isfinite(value_loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1639 Raise `RuntimeError('nonfinite hierarchical value loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1640 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1641 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

Source lines 1642–1645.

```
1642                 torch.nn.utils.clip_grad_norm_(model.critic.parameters(), .5)
1643                 critic_optimizer.step()
1644             daily.clear()
1645     return model, logs, steps
```

Line-by-line explanation.

1642 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1643 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1644 Call daily.clear with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1645 Return `(model, logs, steps)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.121. MAXQMaintenance

Single-aircraft tabular MAXQ-0 adaptation. Primitive reward estimates V and completion estimates C are distinct tables. Recursive execution uses actual elapsed days and exports tables through PyTorch extra state.

Source lines 1648–1656.

```
1648 class MAXQMaintenance(nn.Module):
1649     """Tabular MAXQ-0 baseline on one aircraft and the common repair catalog.
1651     Root chooses operate or Repair; Repair chooses an Engine task; each Engine
1652     chooses a component bundle then waits through its committed work. Primitive
1653     V values and composite completion C values are stored separately. No pseudo
1654     rewards or MAXQ-Q projected completion function is used. Beliefs are binned,
1655     so the original exact-state convergence guarantees are not claimed.
1656     """
```

Line-by-line explanation.

1648 Define `MAXQMaintenance` with the stated inheritance. Single-aircraft tabular MAXQ-0 adaptation. Primitive reward estimates V and completion estimates C are distinct tables. Recursive execution uses actual elapsed days and exports tables through PyTorch extra state.

1649 Documentation text: Tabular MAXQ-0 baseline on one aircraft and the common repair catalog.. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

1651 Documentation text: Root chooses operate or Repair; Repair chooses an Engine task; each Engine. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

1652 Documentation text: chooses a component bundle then waits through its committed work. Primitive. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

1653 Documentation text: V values and composite completion C values are stored separately. No pseudo. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

1654 Documentation text: rewards or MAXQ-Q projected completion function is used. Beliefs are binned,. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

1655 Documentation text: so the original exact-state convergence guarantees are not claimed.. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

1656 Close the documentation string for `MAXQMaintenance`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.122. MAXQMaintenance.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1658–1666.

```
1658     def __init__(self, env):
1659         super().__init__()
1660         if env.config.aircraft != 1:
1661             raise ValueError('MAXQ baseline currently requires Config(aircraft=1)')
1662         self.n = 1
1663         self.local = env.local_size
1664         self.mandatory = env.config.mandatory_health
1665         self.leaf_values = {}
1666         self.completion = {}
```

Line-by-line explanation.

1658 Define `__init__(self, env)`. This ___init___ method is part of MAXQMaintenance.___init___; its parameters and defaults are the interface used by the calls in this section.

1659 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1660 Test `env.config.aircraft != 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1661 Raise `ValueError('MAXQ baseline currently requires Config(aircraft=1)')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1662 Set `self.n` by use the literal 1.

1663 Set `self.local` by evaluate env.local_size.

1664 Set `self.mandatory` by evaluate env.config.mandatory_health.

1665 Set `self.leaf_values` by construct the shown dict in its stated order.

1666 Set `self.completion` by construct the shown dict in its stated order.

10.123. MAXQMaintenance.state

Quantize the entire observed vector into a hashable tabular key. This aliases continuous belief states and never reads hidden health.

Source lines 1668–1670.

```
1668     def state(self, observation):
1669         # Explicit approximation: quantized observed beliefs, never true health.
1670         return tuple(np.rint(np.asarray(observation) * 8).astype(int).tolist())
```

Line-by-line explanation.

1668 Define `state(self, observation)`. Quantize the entire observed vector into a hashable tabular key. This aliases continuous belief states and never reads hidden health.

1669 Source comment: Explicit approximation: quantized observed beliefs, never true health. This line documents the following code and does not execute.

1670 Return `tuple(np.rint(np.asarray(observation) * 8).astype(int).tolist())` to the caller by call tuple with the shown positional and keyword arguments. No further statements in this call execute.

10.124. MAXQMaintenance.task_children

Return the allowed child tasks of the MAXQ parent. The dedicated name avoids colliding with PyTorch Module.children.

Source lines 1672–1686.

```
1672     def task_children(self, task, observation):
1673         busy = observation[37] > .5
1674         if task == 'Root':
1675             if busy:
1676                 return ('Wait',)
1677             urgent = np.min(observation[24:36]) <= self.mandatory
1678             return ('Repair',) if urgent else ('Operate', 'Repair')
1679         if task == 'Repair':
1680             return ('Engine0', 'Engine1', 'EngineBoth')
1681         if task.startswith('Engine'):
1682             if busy:
1683                 return ('Wait',)
1684             e = ('Engine0', 'Engine1', 'EngineBoth').index(task)
1685             return tuple(f'Submit{1 + 7 * e + c}' for c in range(7))
1686         return ()
```

Line-by-line explanation.

1672 Define `task_children(self, task, observation)`. Return the allowed child tasks of the MAXQ parent. The dedicated name avoids colliding with PyTorch Module.children.

1673 Set `busy` by evaluate the comparison observation[37] $>$ 0.5.

1674 Test `task == 'Root'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1675 Test `busy`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1676 Return `('Wait',)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

1677 Set `urgent` by evaluate the comparison np.min(observation[24:36]) <= self.mandatory.

1678 Return `('Repair',) if urgent else ('Operate', 'Repair')` to the caller by select ('Repair',) when urgent, otherwise ('Operate', 'Repair'). No further statements in this call execute.

1679 Test `task == 'Repair'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1680 Return `('Engine0', 'Engine1', 'EngineBoth')` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

1681 Test `task.startswith('Engine')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1682 Test `busy`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1683 Return `('Wait',)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

1684 Set `e` by call ('Engine0', 'Engine1', 'EngineBoth').index with the shown positional and keyword arguments.

1685 Return `tuple((f'Submit{1 + 7 * e + c}' for c in range(7)))` to the caller by call tuple with the shown positional and keyword arguments. No further statements in this call execute.

1686 Return `()` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.125. MAXQMaintenance.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1688–1693.

```
1688     def value(self, task, observation):
1689         children = self.task_children(task, observation)
1690         state = self.state(observation)
1691         if not children:
1692             return self.leaf_values.get((task, state), 0.)
1693         return max(self.q(task, observation, child) for child in children)
```

Line-by-line explanation.

1688 Define `value(self, task, observation)`. This value method is part of MAXQ-Maintenance.value; its parameters and defaults are the interface used by the calls in this section.

1689 Set `children` by call self.task_children with the shown positional and keyword arguments.

1690 Set `state` by call self.state with the shown positional and keyword arguments.

1691 Test `not children`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1692 Return `self.leaf_values.get((task, state), 0.0)` to the caller by call self.leaf_values.get with the shown positional and keyword arguments. No further statements in this call execute.

1693 Return `max((self.q(task, observation, child) for child in children))` to the caller by call max with the shown positional and keyword arguments. No further statements in this call execute.

10.126. MAXQMaintenance.q

Add the MAXQ child value and parent completion value for the given observed state and candidate child.

Source lines 1695–1696.

```
1695     def q(self, task, observation, child):
1696         return self.value(child, observation) + self.completion.get((task, self.state(observation),
         ↪  child), 0.)
```

Line-by-line explanation.

1695 Define `q(self, task, observation, child)`. Add the MAXQ child value and parent completion value for the given observed state and candidate child.

1696 Return `self.value(child, observation) + self.completion.get((task, self.state(observation), child), 0.0)` to the caller by add self.value(child, observation) and self.completion.get((task, self.state(observation), child), 0.0). No further statements in this call execute.

10.127. MAXQMaintenance.update_completion

Update C(parent,state,child) toward the appropriately discounted parent continuation after the child completes, using its actual primitive duration.

Source lines 1698–1701.

```
1698     def update_completion(self, task, observation, child, next_value, duration, alpha=.15, gamma=.99):
1699         key = (task, self.state(observation), child)
1700         old = self.completion.get(key, 0.)
1701         self.completion[key] = old + alpha * (gamma ** duration * next_value - old)
```

Line-by-line explanation.

1698 Define `update_completion(self, task, observation, child, next_value, duration, alpha=.15, gamma=.99)`. Update C(parent,state,child) toward the appropriately discounted parent continuation after the child completes, using its actual primitive duration.

1699 Set `key` by construct the shown tuple in its stated order.

1700 Set `old` by call self.completion.get with the shown positional and keyword arguments.

1701 Set `self.completion[key]` by add old and alpha * (gamma ** duration * next_value - old).

10.128. MAXQMaintenance.execute

Recursively execute the MAXQ task hierarchy, learn primitive/completion values when training, and respect both episode termination and the outer day budget.

Source lines 1703–1720.

```
1703     def execute(self, task, env, observation, done, budget, rng, learn=False, epsilon=.2):
1704         if budget < 1 or done:
1705             return observation, done, [], True
1706         children = self.task_children(task, observation)
1707         if not children:
1708             action = int(task[6:]) if task.startswith('Submit') else 0
1709             following, reward, done, truncated, info = env.step(np.array([action], dtype=np.int64))
1710             if learn:
1711                 key = (task, self.state(observation))
1712                 old = self.leaf_values.get(key, 0.)
1713                 self.leaf_values[key] = old + .15 * (reward - old)
1714             return following, done or truncated, [(observation.copy(), reward)], True
1715         trajectory = []
1716         invoked = False
1717         while len(trajectory) < budget and not done:
1718             if invoked and task != 'Root' and observation[37] <= .5:
1719                 break
1720             children = self.task_children(task, observation)
```

Line-by-line explanation.

1703 Define `execute(self, task, env, observation, done, budget, rng, learn=False, epsilon=.2)`. Recursively execute the MAXQ task hierarchy, learn primitive/completion values when training, and respect both episode termination and the outer day budget.

1704 Test `budget < 1 or done`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1705 Return `(observation, done, [], True)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

1706 Set `children` by call self.task_children with the shown positional and keyword arguments.

1707 Test `not children`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1708 Set `action` by select int(task[6:]) when task.startswith('Submit'), otherwise 0. It holds the sampled or stored action whose probability is evaluated.

1709 Set `(following, reward, done, truncated, info)` by execute this object's step with the shown arguments.

1710 Test `learn`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1711 Set `key` by construct the shown tuple in its stated order.

1712 Set `old` by call self.leaf_values.get with the shown positional and keyword arguments.

1713 Set `self.leaf_values[key]` by add old and 0.15 * (reward - old).

1714 Return `(following, done or truncated, [(observation.copy(), reward)], True)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

1715 Set `trajectory` by construct the shown list in its stated order.

1716 Set `invoked` by use the literal False.

1717 Repeat the indented block while `len(trajectory) < budget and (not done)` remains true; this condition controls the simulation/collection loop here.

1718 Test `invoked and task != 'Root' and (observation[37] <= 0.5)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1719 Exit the nearest loop immediately; execution resumes after that loop.

1720 Set `children` by call self.task_children with the shown positional and keyword arguments.

Source lines 1721–1738.

```
1721            scores = np.array([self.q(task, observation, child) for child in children])
1722            if learn and rng.random() < epsilon:
1723                selected = int(rng.integers(len(children)))
1724            else:
1725                selected = int(np.argmax(scores))
1726            child = children[selected]
1727            following, ended, trace, child_complete = self.execute(
1728                child, env, observation, done, budget - len(trajectory), rng, learn, epsilon)
1729            parent_terminal = ended or (task != 'Root' and following[37] <= .5)
1730            if learn and child_complete:
1731                next_value = 0. if parent_terminal else self.value(task, following)
1732                for offset, (state_obs, _) in enumerate(trace):
1733                    self.update_completion(task, state_obs, child, next_value, len(trace) - offset)
1734            trajectory.extend(trace)
1735            observation, done, invoked = following, ended, True
1736            if task == 'Repair' and child_complete:
1737                break
1738            if not trace:
```

Line-by-line explanation.

1721 Set `scores` by call np.array with the shown positional and keyword arguments.

1722 Test `learn and rng.random() < epsilon`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1723 Set `selected` by convert the result to a Python integer.

1724 Execute this alternative only when the corresponding preceding condition did not select its branch.

1725 Set `selected` by convert the result to a Python integer.

1726 Set `child` by select the indexed/sliced entries of children.

1727 Set `(following, ended, trace, child_complete)` by call self.execute with the shown positional and keyword arguments.

1728 Continue the assignment begun at line 1727 with `child, env, observation, done, budget - len(trajectory), rng, learn, epsilon)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1729 Set `parent_terminal` by evaluate ended or (task != 'Root' and following[37] <= 0.5).

1730 Test `learn and child_complete`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1731 Set `next_value` by select 0.0 when parent_terminal, otherwise self.value(task, following).

1732 Iterate `(offset, (state_obs, _))` over `enumerate(trace)`. The following indented statements run once for each selected element.

1733 Call self.update_completion with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1734 Call trajectory.extend with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1735 Set `(observation, done, invoked)` by construct the shown tuple in its stated order.

1736 Test `task == 'Repair' and child_complete`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1737 Exit the nearest loop immediately; execution resumes after that loop.

1738 Test `not trace`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 1739–1741.

```
1739                 raise RuntimeError('MAXQ child made no physical progress')
1740         complete = done or (task != 'Root' and invoked and observation[37] <= .5)
1741         return observation, done, trajectory, complete
```

Line-by-line explanation.

1739 Raise `RuntimeError('MAXQ child made no physical progress')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1740 Set `complete` by evaluate done or (task != 'Root' and invoked and (observation[37] <= 0.5)).

1741 Return `(observation, done, trajectory, complete)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.129. MAXQMaintenance.get_extra_state

Serialize MAXQ primitive and completion tables as model extra state so a checkpoint contains learned tabular values.

Source lines 1743–1744.

```
1743     def get_extra_state(self):
1744         return {'leaf_values': self.leaf_values, 'completion': self.completion}
```

Line-by-line explanation.

1743 Define `get_extra_state(self)`. Serialize MAXQ primitive and completion tables as model extra state so a checkpoint contains learned tabular values.

1744 Return `{'leaf_values': self.leaf_values, 'completion': self.completion}` to the caller by construct the shown dict in its stated order. No further statements in this call execute.

10.130. MAXQMaintenance.set_extra_state

Restore the saved MAXQ tables during checkpoint loading.

Source lines 1746–1748.

```
1746     def set_extra_state(self, state):
1747         self.leaf_values = dict(state['leaf_values'])
1748         self.completion = dict(state['completion'])
```

Line-by-line explanation.

1746 Define `set_extra_state(self, state)`. Restore the saved MAXQ tables during checkpoint loading.

1747 Set `self.leaf_values` by call dict with the shown positional and keyword arguments.

1748 Set `self.completion` by call dict with the shown positional and keyword arguments.

10.131. train_maxq

Train recursive MAXQ-0 episodes under the explicit one-aircraft restriction and the overall primitive-day budget.

Source lines 1751–1766.

```
1751 def train_maxq(config, steps=4096, seed=0):
1752     if steps < 1:
1753         raise ValueError('positive MAXQ simulation budget required')
1754     env = AircraftOptionEnv(config)
1755     model = MAXQMaintenance(env.env)
1756     rng = np.random.default_rng(seed)
1757     days, episode, logs = 0, 0, []
1758     while days < steps:
1759         obs, _ = env.reset(seed=seed + episode)
1760         _, terminal, trace, _ = model.execute('Root', env, obs, False, steps - days, rng, learn=True)
```

```
        days += len(trace)
        logs.append({'record_type': 'episode' if terminal else 'budget_cutoff', 'simulated_days': days,
                     'return': sum(r for _, r in trace), 'primitive_values': len(model.leaf_values),
                     'completion_values': len(model.completion)})
        episode += 1
    return model, logs, days
```

Line-by-line explanation.

1751 Define `train_maxq(config, steps=4096, seed=0)`. Train recursive MAXQ-0 episodes under the explicit one-aircraft restriction and the overall primitive-day budget.

1752 Test `steps < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1753 Raise `ValueError('positive MAXQ simulation budget required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1754 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

1755 Set `model` by call MAXQMaintenance with the shown positional and keyword arguments. It holds the chosen controller architecture.

1756 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

1757 Set `(days, episode, logs)` by construct the shown tuple in its stated order.

1758 Repeat the indented block while `days < steps` remains true; this condition controls the simulation/collection loop here.

1759 Set `(obs, _)` by initialize a new episode using the stated seed/options.

1760 Set `(_, terminal, trace, _)` by call model.execute with the shown positional and keyword arguments.

1761 Add `len(trace)` to `days`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1762 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1763 Continue the expression begun at line 1762 with `'return': sum(r for _, r in trace), 'primitive_values': len(model.leaf_values),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1764 Continue the expression begun at line 1762 with `'completion_values': len(model.completion)})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1765 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1766 Return `(model, logs, days)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.132. FleetOptionCritic

Three learned planning modes with option-conditioned catalog policies, option values and a separate termination network. Termination is forced to zero during committed repair; learned mode changes cannot cancel physical work.

Source lines 1769–1775.

```
1769 class FleetOptionCritic(nn.Module):
1770     """Option-critic adaptation with safe learned planning-mode termination.
1772     Options condition learned maintenance policies. Beta is exactly zero while
1773     a repair is committed. Changing planning mode never interrupts physical work.
1774     The greedy/epsilon-greedy policy over options is derived from Q_Omega.
1775     """
```

Line-by-line explanation.

1769 Define `FleetOptionCritic` with the stated inheritance. Three learned planning modes with option-conditioned catalog policies, option values and a separate termination network. Termination is forced to zero during committed repair; learned mode changes cannot cancel physical work.

1770 Documentation text: Option-critic adaptation with safe learned planning-mode termination.. This states the contract or qualification for `FleetOptionCritic`; it does not update the simulator or model.

1772 Documentation text: Options condition learned maintenance policies. Beta is exactly zero while. This states the contract or qualification for `FleetOptionCritic`; it does not update the simulator or model.

1773 Documentation text: a repair is committed. Changing planning mode never interrupts physical work.. This states the contract or qualification for `FleetOptionCritic`; it does not update the simulator or model.

1774 Documentation text: The greedy/epsilon-greedy policy over options is derived from Q_Omega.. This states the contract or qualification for `FleetOptionCritic`; it does not update the simulator or model.

1775 Close the documentation string for `FleetOptionCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.133. FleetOptionCritic.__init__

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1777–1788.

```
1777     def __init__(self, env, options=3):
1778         super().__init__()
1779         if options < 2:
1780             raise ValueError('at least two planning options required')
1781         self.n = env.config.aircraft
1782         self.local = env.local_size
1783         self.width = env.observation_space.shape[0]
1784         self.mandatory = env.config.mandatory_health
1785         self.options = options
1786         self.worker = hrl_mlp(self.width + self.local + options, len(HRL_CATALOG))
1787         self.q_network = hrl_mlp(self.width, self.n * options)
1788         self.termination = hrl_mlp(self.width + self.local, options)
```

Line-by-line explanation.

1777 Define `__init__(self, env, options=3)`. This ___init___ method is part of FleetOptionCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

1778 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1779 Test `options < 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1780 Raise `ValueError('at least two planning options required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1781 Set `self.n` by evaluate env.config.aircraft.

1782 Set `self.local` by evaluate env.local_size.

1783 Set `self.width` by select the indexed/sliced entries of env.observation_space.shape.

1784 Set `self.mandatory` by evaluate env.config.mandatory_health.

1785 Set `self.options` by evaluate options.

1786 Set `self.worker` by call hrl_mlp with the shown positional and keyword arguments.

1787 Set `self.q_network` by call hrl_mlp with the shown positional and keyword arguments.

1788 Set `self.termination` by call hrl_mlp with the shown positional and keyword arguments.

10.134. FleetOptionCritic.features

Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

Source lines 1790–1794.

```
1790     def features(self, observation):
1791         obs = _fleet_observation_tensor(observation, self.width,
1792                                         'incorrect option-critic observation')
1793         local = obs[:, :self.n * self.local].reshape(-1, self.n, self.local)
1794         return obs, local, torch.cat([obs[:, None].expand(-1, self.n, -1), local], -1)
```

Line-by-line explanation.

1790 Define `features(self, observation)`. Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

1791 Validate and batch the global observation through the shared helper. This preserves the expected width and caller-specific error while avoiding actor-context allocation.

1792 Continue the assignment begun at line 1791 with `'incorrect option-critic observation')`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1793 Set `local` by reinterpret the shape while retaining element order.

1794 Return `(obs, local, torch.cat([obs[:, None].expand(-1, self.n, -1), local], -1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.135. FleetOptionCritic.q_values

Evaluate the centralized option-value outputs and reshape them into aircraft by planning-mode values.

Source lines 1796–1799.

```
1796     def q_values(self, observation):
1797         obs = _fleet_observation_tensor(observation, self.width,
1798                                         'incorrect option-critic observation')
1799         return self.q_network(obs).reshape(-1, self.n, self.options)
```

Line-by-line explanation.

1796 Define `q_values(self, observation)`. Evaluate the centralized option-value outputs and reshape them into aircraft by planning-mode values.

1797 Validate and batch the global observation through the shared helper. This preserves the expected width and caller-specific error while avoiding actor-context allocation.

1798 Continue the assignment begun at line 1797 with `'incorrect option-critic observation')`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1799 Return `self.q_network(obs).reshape(-1, self.n, self.options)` to the caller by reinterpret the shape while retaining element order. No further statements in this call execute.

10.136. FleetOptionCritic.beta

Evaluate learned planning-mode termination probabilities and force them to zero for busy aircraft.

Source lines 1801–1807.

```
1801     def beta(self, observation, option_ids):
1802         _, local, features = self.features(observation)
1803         ids = torch.as_tensor(option_ids, dtype=torch.long)
1804         if ids.shape != local.shape[:2] or torch.any((ids < 0) | (ids >= self.options)):
1805             raise ValueError('one valid option per aircraft required')
1806         raw = torch.sigmoid(self.termination(features)).gather(-1, ids.unsqueeze(-1)).squeeze(-1)
1807         return raw * (local[:, :, 37] <= .5)
```

Line-by-line explanation.

1801 Define `beta(self, observation, option_ids)`. Evaluate learned planning-mode termination probabilities and force them to zero for busy aircraft.

1802 Set `(_, local, features)` by call self.features with the shown positional and keyword arguments.

1803 Set `ids` by convert inputs to a tensor with the requested dtype.

1804 Test `ids.shape != local.shape[:2] or torch.any((ids < 0) | (ids >= self.options))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1805 Raise `ValueError('one valid option per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1806 Set `raw` by call torch.sigmoid(self.termination(features)).gather(-1, ids.unsqueeze(-1)).squeeze with the shown positional and keyword arguments.

1807 Return `raw * (local[:, :, 37] <= 0.5)` to the caller by multiply raw and local[:, :, 37] <= 0.5. No further statements in this call execute.

10.137. FleetOptionCritic.policy

Run the actor path using its allowed information. The separate critic path need not be available during actor-only execution.

Source lines 1809–1816.

```
1809    def policy(self, observation, option_ids, actions=None, deterministic=False):
1810        obs, local, features = self.features(observation)
1811        ids = torch.as_tensor(option_ids, dtype=torch.long)
1812        if ids.shape != local.shape[:2] or torch.any((ids < 0) | (ids >= self.options)):
1813            raise ValueError('one valid option per aircraft required')
1814        logits = self.worker(torch.cat([features, nn.functional.one_hot(ids, self.options).float()],
        ↪  -1))
1815        valid = hrl_catalog_mask(obs, self.n, self.local, self.mandatory)
1816        return hrl_categorical(logits, valid, actions, deterministic)
```

Line-by-line explanation.

1809 Define `policy(self, observation, option_ids, actions=None, deterministic=False)`. This policy method is part of FleetOptionCritic.policy; its parameters and defaults are the interface used by the calls in this section.

1810 Set `(obs, local, features)` by call self.features with the shown positional and keyword arguments.

1811 Set `ids` by convert inputs to a tensor with the requested dtype.

1812 Test `ids.shape != local.shape[:2] or torch.any((ids < 0) | (ids >= self.options))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1813 Raise `ValueError('one valid option per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1814 Set `logits` by call self.worker with the shown positional and keyword arguments.

1815 Set `valid` by compute permitted catalog entries under commitment and optional budget constraints. It holds the categorical structural-validity mask.

1816 Return `hrl_categorical(logits, valid, actions, deterministic)` to the caller by sample or score a structurally masked categorical decision. No further statements in this call execute.

10.138. option_critic_target

Construct the option continuation mixture using safe termination probability and the maximum next option value.

Source lines 1819–1822.

```
1819 def option_critic_target(reward, following_q, option_ids, beta, terminal, gamma=.99):
1820     current = following_q.gather(-1, option_ids.unsqueeze(-1)).squeeze(-1)
1821     continuation = (1 - beta) * current + beta * following_q.max(-1).values
1822     return reward + gamma * (1 - terminal.float()) * continuation
```

Line-by-line explanation.

1819 Define `option_critic_target(reward, following_q, option_ids, beta, terminal, gamma=.99)`. Construct the option continuation mixture using safe termination probability and the maximum next option value.

1820 Set `current` by call following_q.gather(-1, option_ids.unsqueeze(-1)).squeeze with the shown positional and keyword arguments.

1821 Set `continuation` by add (1 - beta) * current and beta * following_q.max(-1).values.

1822 Return `reward + gamma * (1 - terminal.float()) * continuation` to the caller by add reward and gamma * (1 - terminal.float()) * continuation. No further statements in this call execute.

10.139. hrl_epsilon_choice

Choose a feasible action greedily or through epsilon exploration; invalid choices never enter the exploration set.

Source lines 1825–1834.

```
1825 def hrl_epsilon_choice(values, valid, rng, epsilon=0.):
1826     values = np.asarray(values)
1827     valid = np.asarray(valid, dtype=bool)
1828     if values.ndim != 2 or values.shape != valid.shape or not valid.any(-1).all():
1829         raise ValueError('one nonempty choice mask per control unit required')
1830     choices = np.argmax(np.where(valid, values, -np.inf), axis=-1)
1831     for i in range(len(choices)):
1832         if rng.random() < epsilon:
1833             choices[i] = rng.choice(np.flatnonzero(valid[i]))
1834     return choices.astype(np.int64)
```

Line-by-line explanation.

1825 Define `hrl_epsilon_choice(values, valid, rng, epsilon=0.)`. Choose a feasible action greedily or through epsilon exploration; invalid choices never enter the exploration set.

1826 Set `values` by call np.asarray with the shown positional and keyword arguments. It holds critic predictions used for baselines or TD residuals.

1827 Set `valid` by call np.asarray with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

1828 Test `values.ndim != 2 or values.shape != valid.shape or (not valid.any(-1).all())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1829 Raise `ValueError('one nonempty choice mask per control unit required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1830 Set `choices` by call np.argmax with the shown positional and keyword arguments.

1831 Iterate `i` over `range(len(choices))`. The following indented statements run once for each selected element.

1832 Test `rng.random() < epsilon`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1833 Set `choices[i]` by call rng.choice with the shown positional and keyword arguments.

1834 Return `choices.astype(np.int64)` to the caller by call choices.astype with the shown positional and keyword arguments. No further statements in this call execute.

10.140. train_option_critic

Perform online internal-policy, option-value and termination updates with separate optimizers. Committed jobs mask action/termination learning where no decision is permitted.

Source lines 1837–1854.

```
def train_option_critic(config, steps=4096, seed=0):
    if steps < 1:
        raise ValueError('positive option-critic simulation budget required')
    torch.set_num_threads(1)
    torch.manual_seed(seed)
    rng = np.random.default_rng(seed)
    env = AircraftOptionEnv(config)
    model = FleetOptionCritic(env.env)
    actor_opt = torch.optim.Adam(model.worker.parameters(), lr=3e-4)
    critic_opt = torch.optim.Adam(model.q_network.parameters(), lr=3e-4)
    beta_opt = torch.optim.Adam(model.termination.parameters(), lr=3e-4)
    obs, _ = env.reset(seed=seed)
    def select(observation):
        with torch.no_grad():
            q = model.q_values(observation)[0].numpy()
        return hrl_epsilon_choice(q, np.ones_like(q, dtype=bool), rng, .1)
    ids = select(obs)
    episode, logs = 0, []
```

Line-by-line explanation.

1837 Define `train_option_critic(config, steps=4096, seed=0)`. Perform online internal-policy, option-value and termination updates with separate optimizers. Committed jobs mask action/termination learning where no decision is permitted.

1838 Test `steps < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1839 Raise `ValueError('positive option-critic simulation budget required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1840 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

1841 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

1842 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

1843 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

1844 Set `model` by call FleetOptionCritic with the shown positional and keyword arguments. It holds the chosen controller architecture.

1845 Set `actor_opt` by create an Adam optimizer for the specified parameter collection.

1846 Set `critic_opt` by create an Adam optimizer for the specified parameter collection.

1847 Set `beta_opt` by create an Adam optimizer for the specified parameter collection.

1848 Set `(obs, _)` by initialize a new episode using the stated seed/options.

1849 Define `select(observation)`. This select method is part of train_option_critic; its parameters and defaults are the interface used by the calls in this section.

1850 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1851 Set `q` by expose this CPU tensor as a NumPy array. It holds an action or option value estimate.

1852 Return `hrl_epsilon_choice(q, np.ones_like(q, dtype=bool), rng, 0.1)` to the caller by call hrl_epsilon_choice with the shown positional and keyword arguments. No further statements in this call execute.

1853 Set `ids` by call select with the shown positional and keyword arguments.

1854 Set `(episode, logs)` by construct the shown tuple in its stated order.

Source lines 1855–1872.

```
    for day in range(1, steps + 1):
        option_ids = torch.tensor(ids).unsqueeze(0)
        actions, logp, entropy = model.policy(obs, option_ids)
        active = torch.tensor(env.decision_mask()).unsqueeze(0).float()
        following, reward, done, truncated, info = env.step(actions[0].detach().numpy())
        terminal = done or truncated
        with torch.no_grad():
            following_q = model.q_values(following)
        beta = model.beta(following, option_ids)
        target = option_critic_target(reward, following_q, option_ids, beta.detach(),
        ↪  torch.tensor(terminal))
        q = model.q_values(obs).gather(-1, option_ids.unsqueeze(-1)).squeeze(-1)
        advantage = (target - q).detach()
        actor_loss = -((logp * advantage + .001 * entropy) * active).sum() / active.sum().clamp_min(1)
        critic_loss = .5 * (q - target.detach()).square().mean()
        safe = torch.tensor(env.decision_mask()).unsqueeze(0).float() * (not terminal)
        termination_advantage = (following_q.gather(-1, option_ids.unsqueeze(-1)).squeeze(-1)
                                 - following_q.max(-1).values + .01).detach()
        termination_loss = (beta * termination_advantage * safe).sum() / safe.sum().clamp_min(1)
```

Line-by-line explanation.

1855 Iterate `day` over `range(1, steps + 1)`. The following indented statements run once for each selected element.

1856 Set `option_ids` by call torch.tensor(ids).unsqueeze with the shown positional and keyword arguments. It holds the current learned planning mode for each aircraft.

1857 Set `(actions, logp, entropy)` by call model.policy with the shown positional and keyword arguments.

1858 Set `active` by call torch.tensor(env.decision_mask()).unsqueeze(0).float with the shown positional and keyword arguments.

1859 Set `(following, reward, done, truncated, info)` by execute this object's step with the shown arguments.

1860 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

1861 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1862 Set `following_q` by call model.q_values with the shown positional and keyword arguments.

1863 Set `beta` by call model.beta with the shown positional and keyword arguments.

1864 Set `target` by mix continuation and reselection option values using safe termination. It holds a bootstrapped learning target.

1865 Set `q` by call model.q_values(obs).gather(-1, option_ids.unsqueeze(-1)).squeeze with the shown positional and keyword arguments. It holds an action or option value estimate.

1866 Set `advantage` by stop gradients through this result.

1867 Set `actor_loss` by divide -((logp * advantage + 0.001 * entropy) * active).sum() and active.sum().clamp_min(1). It holds the objective differentiated through policy parameters.

1868 Set `critic_loss` by multiply 0.5 and (q - target.detach()).square().mean(). It holds the prediction-error objective for the value estimator.

1869 Set `safe` by multiply torch.tensor(env.decision_mask()).unsqueeze(0).float() and not terminal.

1870 Set `termination_advantage` by stop gradients through this result.

1871 Continue the assignment begun at line 1870 with `- following_q.max(-1).values + .01).detach()`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1872 Set `termination_loss` by divide (beta * termination_advantage * safe).sum() and safe.sum().clamp_min(1).

Source lines 1873–1890.

```
        if not bool(torch.isfinite(actor_loss + critic_loss + termination_loss)):
            raise RuntimeError('nonfinite option-critic loss')
        actor_opt.zero_grad()
        actor_loss.backward()
```

```
        torch.nn.utils.clip_grad_norm_(model.worker.parameters(), .5)
        if bool(active.any()):
            actor_opt.step()
        critic_opt.zero_grad()
        critic_loss.backward()
        torch.nn.utils.clip_grad_norm_(model.q_network.parameters(), .5)
        critic_opt.step()
        beta_opt.zero_grad()
        termination_loss.backward()
        torch.nn.utils.clip_grad_norm_(model.termination.parameters(), .5)
        if bool(safe.any()):
            beta_opt.step()
        terminate = (rng.random(config.aircraft) < beta.detach()[0].numpy()) & env.decision_mask()
        next_ids = select(following)
```

Line-by-line explanation.

1873 Test `not bool(torch.isfinite(actor_loss + critic_loss + termination_loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1874 Raise `RuntimeError('nonfinite option-critic loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1875 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1876 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

1877 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1878 Test `bool(active.any())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1879 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1880 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1881 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

1882 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1883 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1884 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

1885 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

1886 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

1887 Test `bool(safe.any())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1888 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

1889 Set `terminate` by apply a bitwise intersection to rng.random(config.aircraft) < beta.detach()[0].numpy() and env.decision_mask().

1890 Set `next_ids` by call select with the shown positional and keyword arguments.

Source lines 1891–1902.

```
        ids = np.where(terminate, next_ids, ids)
        logs.append({'record_type': 'update', 'simulated_days': day,
                     'policy_loss': float(actor_loss.detach()), 'value_loss':
                     ↪  float(critic_loss.detach()),
                     'termination_loss': float(termination_loss.detach()),
                     'safe_termination_samples': int(safe.sum()), 'option_terminations':
                     ↪  int(terminate.sum())})
        obs = following
        if terminal:
            logs.append({'record_type': 'episode', 'simulated_days': day, **info['metrics']})
            episode += 1
            obs, _ = env.reset(seed=seed + episode)
            ids = select(obs)
    return model, logs, steps
```

Line-by-line explanation.

1891 Set `ids` by call np.where with the shown positional and keyword arguments.

1892 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1893 Continue the expression begun at line 1892 with `'policy_loss': float(actor_loss.detach()), 'value_loss': float(critic_loss.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1894 Continue the expression begun at line 1892 with `'termination_loss': float(termination_loss.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1895 Continue the expression begun at line 1892 with `'safe_termination_samples': int(safe.sum()), 'option_terminations': int(terminate.sum())})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1896 Set `obs` by evaluate following. It holds the observed belief vector, rather than hidden physical health.

1897 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1898 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1899 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1900 Set `(obs, _)` by initialize a new episode using the stated seed/options.

1901 Set `ids` by call select with the shown positional and keyword arguments.

1902 Return `(model, logs, steps)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.141. HierarchicalVDN

Two levels of independent aircraft Q networks with additive value decomposition. The upper level sets budgets on a macro clock and the lower level requests daily catalog actions, with a detached high-level advantage shaping its reward.

Source lines 1905–1912.

```
class HierarchicalVDN(nn.Module):
    """Two-level value-decomposition adaptation, not an exact HAVEN reproduction.
    Per-aircraft high policies select replacement budgets; local low policies
    choose catalog requests. Both levels sum agent Q values. The low reward
    receives a detached share of the macro TD advantage (coefficient 0.1).
    Macro epochs are three days; busy aircraft retain goals and committed plans.
    """
```

Line-by-line explanation.

1905 Define `HierarchicalVDN` with the stated inheritance. Two levels of independent aircraft Q networks with additive value decomposition. The upper level sets budgets on a macro clock and the lower level requests daily catalog actions, with a detached high-level advantage shaping its reward.

1906 Documentation text: Two-level value-decomposition adaptation, not an exact HAVEN reproduction.. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

1908 Documentation text: Per-aircraft high policies select replacement budgets; local low policies. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

1909 Documentation text: choose catalog requests. Both levels sum agent Q values. The low reward. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

1910 Documentation text: receives a detached share of the macro TD advantage (coefficient 0.1).. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

1911 Documentation text: Macro epochs are three days; busy aircraft retain goals and committed plans.. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

1912 Close the documentation string for `HierarchicalVDN`. This delimiter ends the descriptive text and performs no simulator or learning update.

10.142. HierarchicalVDN.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1914–1921.

```
    def __init__(self, env):
        super().__init__()
        self.n = env.config.aircraft
        self.local = env.local_size
        self.public = env.public_size
```

```
1919            self.mandatory = env.config.mandatory_health
1920            self.high_agents = nn.ModuleList([hrl_mlp(self.local + self.public, 4) for _ in range(self.n)])
1921            self.low_agents = nn.ModuleList([hrl_mlp(self.local + self.public + 4, len(HRL_CATALOG)) for _
                ↪  in range(self.n)])
```

Line-by-line explanation.

1914 Define `__init__(self, env)`. This ___init___ method is part of HierarchicalVDN.___init___; its parameters and defaults are the interface used by the calls in this section.

1915 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1916 Set `self.n` by evaluate env.config.aircraft.

1917 Set `self.local` by evaluate env.local_size.

1918 Set `self.public` by evaluate env.public_size.

1919 Set `self.mandatory` by evaluate env.config.mandatory_health.

1920 Set `self.high_agents` by register the listed independent networks so parameters and checkpoints include them.

1921 Set `self.low_agents` by register the listed independent networks so parameters and checkpoints include them.

10.143. HierarchicalVDN.features

Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

Source lines 1923–1924.

```
1923        def features(self, observation):
1924            return mappo_agent_inputs(observation, self.n, self.local, self.public, 'aircraft')
```

Line-by-line explanation.

1923 Define `features(self, observation)`. Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

1924 Return `mappo_agent_inputs(observation, self.n, self.local, self.public, 'aircraft')` to the caller by call mappo_agent_inputs with the shown positional and keyword arguments. No further statements in this call execute.

10.144. HierarchicalVDN.high_values

Evaluate the independent aircraft upper-level budget Q networks from local/public features.

Source lines 1926–1928.

```
1926        def high_values(self, observation):
1927            x = self.features(observation)
1928            return torch.stack([agent(x[:, i]) for i, agent in enumerate(self.high_agents)], 1)
```

Line-by-line explanation.

1926 Define `high_values(self, observation)`. Evaluate the independent aircraft upper-level budget Q networks from local/public features.

1927 Set `x` by call self.features with the shown positional and keyword arguments.

1928 Return `torch.stack([agent(x[:, i]) for i, agent in enumerate(self.high_agents)], 1)` to the caller by join equal-shaped tensors along the specified new axis. No further statements in this call execute.

10.145. HierarchicalVDN.low_values

Evaluate independent aircraft catalog Q networks conditioned on local/public features and the retained budget goal.

Source lines 1930–1936.

```
1930     def low_values(self, observation, goals):
1931         x = self.features(observation)
1932         goals = torch.as_tensor(goals, dtype=torch.long)
1933         if goals.shape != x.shape[:2] or torch.any((goals < 0) | (goals >= 4)):
1934             raise ValueError('one valid hierarchy goal per aircraft required')
1935         x = torch.cat([x, nn.functional.one_hot(goals, 4).float()], -1)
1936         return torch.stack([agent(x[:, i]) for i, agent in enumerate(self.low_agents)], 1)
```

Line-by-line explanation.

1930 Define `low_values(self, observation, goals)`. Evaluate independent aircraft catalog Q networks conditioned on local/public features and the retained budget goal.

1931 Set `x` by call self.features with the shown positional and keyword arguments.

1932 Set `goals` by convert inputs to a tensor with the requested dtype. It holds per-aircraft upper-level replacement budgets.

1933 Test `goals.shape != x.shape[:2] or torch.any((goals < 0) | (goals >= 4))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1934 Raise `ValueError('one valid hierarchy goal per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1935 Set `x` by concatenate features along the specified existing axis.

1936 Return `torch.stack([agent(x[:, i]) for i, agent in enumerate(self.low_agents)], 1)` to the caller by join equal-shaped tensors along the specified new axis. No further statements in this call execute.

10.146. HierarchicalVDN.mix

Add the aircraft Q values to obtain the joint value-decomposition estimate. There is no learned nonlinear mixer.

Source lines 1938–1940.

```
    @staticmethod
    def mix(values):
        return values.sum(-1)
```

Line-by-line explanation.

`1938` Apply the decorator `staticmethod` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

`1939` Define `mix(values)`. Add the aircraft Q values to obtain the joint value-decomposition estimate. There is no learned nonlinear mixer.

`1940` Return `values.sum(-1)` to the caller by sum across the specified tensor/array axis. No further statements in this call execute.

10.147. HierarchicalVDN.choose_goals

Choose structurally permitted upper-level budget goals with epsilon exploration during learning or greedily during evaluation.

Source lines 1942–1954.

```
    def choose_goals(self, observation, previous, rng, epsilon=0.):
        local = self.features(observation)[0]
        busy = local[:, 37].numpy() > .5
        urgent = local[:, 24:36].amin(-1).numpy() <= self.mandatory
        valid = np.ones((self.n, 4), dtype=bool)
        valid[urgent & ~busy, 0] = False
        with torch.no_grad():
            choices = hrl_epsilon_choice(self.high_values(observation)[0].numpy(), valid, rng, epsilon)
        if previous is not None:
            choices[busy] = np.asarray(previous)[busy]
        else:
            choices[busy] = 0
        return choices
```

Line-by-line explanation.

`1942` Define `choose_goals(self, observation, previous, rng, epsilon=0.)`. Choose structurally permitted upper-level budget goals with epsilon exploration during learning or greedily during evaluation.

`1943` Set `local` by select the indexed/sliced entries of self.features(observation).

`1944` Set `busy` by evaluate the comparison local[:, 37].numpy() > 0.5.

`1945` Set `urgent` by evaluate the comparison local[:, 24:36].amin(-1).numpy() <= self.mandatory.

`1946` Set `valid` by call np.ones with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

`1947` Set `valid[urgent & ~busy, 0]` by use the literal False. It holds the categorical structural-validity mask.

`1948` Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1949 Set `choices` by call hrl_epsilon_choice with the shown positional and keyword arguments.

1950 Test `previous is not None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1951 Set `choices[busy]` by select the indexed/sliced entries of np.asarray(previous).

1952 Execute this alternative only when the corresponding preceding condition did not select its branch.

1953 Set `choices[busy]` by use the literal 0.

1954 Return `choices` to the caller by evaluate choices. No further statements in this call execute.

10.148. HierarchicalVDN.choose_actions

Choose structurally permitted budget-conditioned catalog requests and retain the action values required by the low-level update.

Source lines 1956–1963.

```
1956     def choose_actions(self, observation, goals, rng, epsilon=0.):
1957         goals = torch.tensor(np.asarray(goals)).unsqueeze(0)
1958         valid = hrl_catalog_mask(observation, self.n, self.local, self.mandatory, goals)
1959         # An urgent aircraft may outgrow an old zero-budget goal before the next macro epoch.
1960         if not bool(valid.any(-1).all()):
1961             raise ValueError('a stale hierarchy goal must be refreshed after an urgency alarm')
1962         with torch.no_grad():
1963             return hrl_epsilon_choice(self.low_values(observation, goals)[0].numpy(),
             ↪  valid[0].numpy(), rng, epsilon)
```

Line-by-line explanation.

1956 Define `choose_actions(self, observation, goals, rng, epsilon=0.)`. Choose structurally permitted budget-conditioned catalog requests and retain the action values required by the low-level update.

1957 Set `goals` by call torch.tensor(np.asarray(goals)).unsqueeze with the shown positional and keyword arguments. It holds per-aircraft upper-level replacement budgets.

1958 Set `valid` by compute permitted catalog entries under commitment and optional budget constraints. It holds the categorical structural-validity mask.

1959 Source comment: An urgent aircraft may outgrow an old zero-budget goal before the next macro epoch. This line documents the following code and does not execute.

1960 Test `not bool(valid.any(-1).all())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1961 Raise `ValueError('a stale hierarchy goal must be refreshed after an urgency alarm')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1962 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1963 Return `hrl_epsilon_choice(self.low_values(observation, goals)[0].numpy(), valid[0].numpy(), rng, epsilon)` to the caller by call hrl_epsilon_choice with the shown positional and keyword arguments. No further statements in this call execute.

10.149. train_hierarchical_vdn

Collect macro budget choices and daily catalog requests, update additive high-level TD targets, and train the low level with the disclosed advantage-shaped reward.

Source lines 1966–1983.

```
def train_hierarchical_vdn(config, steps=4096, seed=0, macro_days=3):
    if steps < 1 or macro_days < 1:
        raise ValueError('positive simulation budget and macro duration required')
    torch.set_num_threads(1)
    torch.manual_seed(seed)
    rng = np.random.default_rng(seed)
    env = AircraftOptionEnv(config)
    model = HierarchicalVDN(env.env)
    high_opt = torch.optim.Adam(model.high_agents.parameters(), lr=3e-4)
    low_opt = torch.optim.Adam(model.low_agents.parameters(), lr=3e-4)
    obs, _ = env.reset(seed=seed)
    goals, days, episode, logs = None, 0, 0, []
    while days < steps:
        goals = model.choose_goals(obs, goals, rng, .1)
        start, window = obs.copy(), []
        for _ in range(min(macro_days, steps - days)):
            actions = model.choose_actions(obs, goals, rng, .1)
            following, reward, done, truncated, info = env.step(actions)
```

Line-by-line explanation.

1966 Define `train_hierarchical_vdn(config, steps=4096, seed=0, macro_days=3)`. Collect macro budget choices and daily catalog requests, update additive high-level TD targets, and train the low level with the disclosed advantage-shaped reward.

1967 Test `steps < 1 or macro_days < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1968 Raise `ValueError('positive simulation budget and macro duration required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1969 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

1970 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

1971 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

1972 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

1973 Set `model` by call HierarchicalVDN with the shown positional and keyword arguments. It holds the chosen controller architecture.

1974 Set `high_opt` by create an Adam optimizer for the specified parameter collection.

1975 Set `low_opt` by create an Adam optimizer for the specified parameter collection.

1976 Set `(obs, _)` by initialize a new episode using the stated seed/options.

1977 Set `(goals, days, episode, logs)` by construct the shown tuple in its stated order.

1978 Repeat the indented block while `days < steps` remains true; this condition controls the simulation/collection loop here.

1979 Set `goals` by call model.choose_goals with the shown positional and keyword arguments. It holds per-aircraft upper-level replacement budgets.

1980 Set `(start, window)` by construct the shown tuple in its stated order.

1981 Iterate `_` over `range(min(macro_days, steps - days))`. The following indented statements run once for each selected element.

1982 Set `actions` by call model.choose_actions with the shown positional and keyword arguments.

1983 Set `(following, reward, done, truncated, info)` by execute this object's step with the shown arguments.

Source lines 1984–2001.

```
            terminal = done or truncated
            window.append((obs.copy(), goals.copy(), actions.copy(), reward, following.copy(),
            ↪  terminal))
            days += 1
            obs = following
            # End the macro on a new urgency alarm so zero-budget goals never block safety.
            rows, _ = env.env.split_observation(obs)
            urgent = rows[:, 24:36].min(-1) <= config.mandatory_health
            if terminal or np.any(urgent & env.decision_mask() & (goals == 0)):
                break
        next_goals = model.choose_goals(obs, goals, rng)
        self_n = config.aircraft
        selected_goals = torch.tensor(goals).reshape(1, self_n, 1)
        high_value = model.mix(model.high_values(start).gather(-1, selected_goals).squeeze(-1))
        with torch.no_grad():
            next_high = model.mix(model.high_values(obs).gather(-1,
            ↪  torch.tensor(next_goals).reshape(1, self_n, 1)).squeeze(-1))
            target = hrl_option_target([r[3] for r in window], float(next_high[0]), terminal)
        high_advantage = (target - high_value).detach()
        high_loss = .5 * (high_value - target).square().mean()
```

Line-by-line explanation.

1984 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

1985 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1986 Add `1` to `days`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1987 Set `obs` by evaluate following. It holds the observed belief vector, rather than hidden physical health.

1988 Source comment: End the macro on a new urgency alarm so zero-budget goals never block safety. This line documents the following code and does not execute.

1989 Set `(rows, _)` by call env.env.split_observation with the shown positional and keyword arguments.

1990 Set `urgent` by evaluate the comparison rows[:, 24:36].min(-1) <= config.mandatory_health.

1991 Test `terminal or np.any(urgent & env.decision_mask() & (goals == 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1992 Exit the nearest loop immediately; execution resumes after that loop.

1993 Set `next_goals` by call model.choose_goals with the shown positional and keyword arguments.

1994 Set `self_n` by evaluate config.aircraft.

1995 Set `selected_goals` by reinterpret the shape while retaining element order.

1996 Set `high_value` by call model.mix with the shown positional and keyword arguments.

1997 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

1998 Set `next_high` by call model.mix with the shown positional and keyword arguments.

1999 Set `target` by discount daily rewards and bootstrap over the actual aircraft-decision duration. It holds a bootstrapped learning target.

2000 Set `high_advantage` by stop gradients through this result.

2001 Set `high_loss` by multiply 0.5 and (high_value - target).square().mean().

Source lines 2002–2019.

```
        low_loss = torch.zeros(())
        for t, (state, old_goals, action, reward, following, ended) in enumerate(window):
            next_goal = next_goals if t == len(window) - 1 else old_goals
            q = model.mix(model.low_values(state, torch.tensor(old_goals).unsqueeze(0)).gather(
                -1, torch.tensor(action).reshape(1, self_n, 1)).squeeze(-1))
            with torch.no_grad():
                valid = hrl_catalog_mask(following, self_n, env.env.local_size,
                ↪  config.mandatory_health,
                                         torch.tensor(next_goal).unsqueeze(0))
                future = model.low_values(following, torch.tensor(next_goal).unsqueeze(0))
                next_low = model.mix(future.masked_fill(~valid, -1e9).max(-1).values)
                shaped_reward = reward + .1 * high_advantage / len(window)
                low_target = shaped_reward + .99 * (not ended) * next_low
            low_loss = low_loss + .5 * (q - low_target).square().mean() / len(window)
        if not bool(torch.isfinite(high_loss + low_loss)):
            raise RuntimeError('nonfinite hierarchical value-decomposition loss')
        high_opt.zero_grad()
        high_loss.backward()
        torch.nn.utils.clip_grad_norm_(model.high_agents.parameters(), .5)
```

Line-by-line explanation.

2002 Set `low_loss` by call torch.zeros with the shown positional and keyword arguments.

2003 Iterate `(t, (state, old_goals, action, reward, following, ended))` over `enumerate(window)`. The following indented statements run once for each selected element.

2004 Set `next_goal` by select next_goals when t == len(window) - 1, otherwise old_goals.

2005 Set `q` by call model.mix with the shown positional and keyword arguments. It holds an action or option value estimate.

2006 Continue the assignment begun at line 2005 with `-1, torch.tensor(action).reshape(1, self_n, 1)).squeeze(-1))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2007 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2008 Set `valid` by compute permitted catalog entries under commitment and optional budget constraints. It holds the categorical structural-validity mask.

2009 Continue the assignment begun at line 2008 with `torch.tensor(next_goal).unsqueeze(0))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2010 Set `future` by call model.low_values with the shown positional and keyword arguments.

2011 Set `next_low` by call model.mix with the shown positional and keyword arguments.

2012 Set `shaped_reward` by add reward and 0.1 * high_advantage / len(window).

2013 Set `low_target` by add shaped_reward and 0.99 * (not ended) * next_low.

2014 Set `low_loss` by add low_loss and 0.5 * (q - low_target).square().mean() / len(window).

2015 Test `not bool(torch.isfinite(high_loss + low_loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2016 Raise `RuntimeError('nonfinite hierarchical value-decomposition loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2017 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2018 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

2019 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

Source lines 2020–2033.

```
        high_opt.step()
        low_opt.zero_grad()
        low_loss.backward()
        torch.nn.utils.clip_grad_norm_(model.low_agents.parameters(), .5)
        low_opt.step()
        logs.append({'record_type': 'macro_update', 'simulated_days': days, 'duration': len(window),
                     'high_loss': float(high_loss.detach()), 'low_loss': float(low_loss.detach()),
                     'high_advantage': float(high_advantage[0]), 'goals': goals.tolist()})
        if terminal:
            logs.append({'record_type': 'episode', 'simulated_days': days, **info['metrics']})
            episode += 1
            obs, _ = env.reset(seed=seed + episode)
            goals = None
    return model, logs, days
```

Line-by-line explanation.

2020 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2021 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2022 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

2023 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

2024 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2025 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2026 Continue the expression begun at line 2025 with `'high_loss': float(high_loss.detach()), 'low_loss': float(low_loss.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2027 Continue the expression begun at line 2025 with `'high_advantage': float(high_advantage[0]), 'goals': goals.tolist()})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2028 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2029 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2030 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2031 Set `(obs, _)` by initialize a new episode using the stated seed/options.

2032 Set `goals` by use the literal None. It holds per-aircraft upper-level replacement budgets.

2033 Return `(model, logs, days)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.150. evaluate_hrl

Execute each new hierarchy without learning, retaining the necessary option IDs, goals or macro clock and collecting the common physical metrics.

Source lines 2036–2053.

```
def evaluate_hrl(config, method, model, seeds):
    """Evaluate hierarchy controllers with a daily physical clock and fixed weights."""
    if method not in HRL_METHODS or model is None:
        raise ValueError('registered hierarchy and trained model required')
    records = []
    for seed in seeds:
```

```
        env = AircraftOptionEnv(config)
        obs, _ = env.reset(seed=int(seed))
        rng = np.random.default_rng(int(seed))
        decisions, free_decisions, terminations = 0, 0, 0
        if method == 'maxq':
            _, _, trace, _ = model.execute('Root', env, obs, False, config.horizon, rng, learn=False)
            decisions = len(trace)
        else:
            goals, option_ids, macro_clock = None, None, 0
            while True:
                free_decisions += int(env.decision_mask().sum())
                with torch.no_grad():
```

Line-by-line explanation.

2036 Define `evaluate_hrl(config, method, model, seeds)`. Execute each new hierarchy without learning, retaining the necessary option IDs, goals or macro clock and collecting the common physical metrics.

2037 Documentation text: Evaluate hierarchy controllers with a daily physical clock and fixed weights.. This states the contract or qualification for `evaluate_hrl`; it does not update the simulator or model.

2038 Test `method not in HRL_METHODS or model is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2039 Raise `ValueError('registered hierarchy and trained model required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2040 Set `records` by construct the shown list in its stated order.

2041 Iterate `seed` over `seeds`. The following indented statements run once for each selected element.

2042 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

2043 Set `(obs, _)` by initialize a new episode using the stated seed/options.

2044 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

2045 Set `(decisions, free_decisions, terminations)` by construct the shown tuple in its stated order.

2046 Test `method == 'maxq'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2047 Set `(_, _, trace, _)` by call model.execute with the shown positional and keyword arguments.

2048 Set `decisions` by count entries.

2049 Execute this alternative only when the corresponding preceding condition did not select its branch.

2050 Set `(goals, option_ids, macro_clock)` by construct the shown tuple in its stated order.

2051 Repeat the indented block while `True` remains true; this condition controls the simulation/collection loop here.

2052 Add `int(env.decision_mask().sum())` to `free_decisions`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2053 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

Source lines 2054–2071.

```
                if method in HRL_PPO_METHODS:
                    action = model.act(obs, deterministic=True)[0]
                    choices = model.catalog_actions(obs, action)
                elif method == 'option_critic':
                    if option_ids is None:
                        option_ids = model.q_values(obs)[0].argmax(-1).numpy()
                    ids = torch.tensor(option_ids).unsqueeze(0)
                    choices = model.policy(obs, ids, deterministic=True)[0][0].numpy()
                else:
                    if goals is None or macro_clock == 0:
                        goals = model.choose_goals(obs, goals, rng)
                    choices = model.choose_actions(obs, goals, rng)
            obs, _, done, truncated, info = env.step(choices)
            decisions += 1
            if method == 'option_critic' and not (done or truncated):
                with torch.no_grad():
                    beta = model.beta(obs, torch.tensor(option_ids).unsqueeze(0))[0].numpy()
                    next_ids = model.q_values(obs)[0].argmax(-1).numpy()
```

Line-by-line explanation.

2054 Test `method in HRL_PPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2055 Set `action` by select the indexed/sliced entries of model.act(obs, deterministic=True). It holds the sampled or stored action whose probability is evaluated.

2056 Set `choices` by call model.catalog_actions with the shown positional and keyword arguments.

2057 Test `method == 'option_critic'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2058 Test `option_ids is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2059 Set `option_ids` by expose this CPU tensor as a NumPy array. It holds the current learned planning mode for each aircraft.

2060 Set `ids` by call torch.tensor(option_ids).unsqueeze with the shown positional and keyword arguments.

2061 Set `choices` by expose this CPU tensor as a NumPy array.

2062 Execute this alternative only when the corresponding preceding condition did not select its branch.

2063 Test `goals is None or macro_clock == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2064 Set `goals` by call model.choose_goals with the shown positional and keyword arguments. It holds per-aircraft upper-level replacement budgets.

2065 Set `choices` by call model.choose_actions with the shown positional and keyword arguments.

2066 Set `(obs, _, done, truncated, info)` by execute this object's step with the shown arguments.

2067 Add `1` to `decisions`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2068 Test `method == 'option_critic' and (not (done or truncated))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2069 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2070 Set `beta` by expose this CPU tensor as a NumPy array.

2071 Set `next_ids` by expose this CPU tensor as a NumPy array.

Source lines 2072–2087.

```
2072                     change = (beta >= .5) & env.decision_mask()
2073                     option_ids = np.where(change, next_ids, option_ids)
2074                     terminations += int(change.sum())
2075                 if method == 'hierarchical_vdn':
2076                     macro_clock = (macro_clock + 1) % 3
2077                     rows, _ = env.env.split_observation(obs)
2078                     if np.any((rows[:, 24:36].min(-1) <= config.mandatory_health) &
                    ↪  env.decision_mask() & (goals == 0)):
2079                         macro_clock = 0
2080                 if done or truncated:
2081                     break
2082         metrics = env.last_info['metrics']
2083         records.append({'seed': int(seed), 'decisions': decisions,
2084                         'learned_planning_opportunities': free_decisions,
2085                         'planning_mode_terminations': terminations,
2086                         'availability': metrics['flights'] / (config.aircraft * config.horizon),
                        ↪  **metrics})
2087     return records
```

Line-by-line explanation.

2072 Set `change` by apply a bitwise intersection to beta >= 0.5 and env.decision_mask().

2073 Set `option_ids` by call np.where with the shown positional and keyword arguments. It holds the current learned planning mode for each aircraft.

2074 Add `int(change.sum())` to `terminations`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2075 Test `method == 'hierarchical_vdn'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2076 Set `macro_clock` by take the remainder of macro_clock + 1 and 3.

2077 Set `(rows, _)` by call env.env.split_observation with the shown positional and keyword arguments.

2078 Test `np.any((rows[:, 24:36].min(-1) <= config.mandatory_health) & env.decision_mask() & (goals == 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2079 Set `macro_clock` by use the literal 0.

2080 Test `done or truncated`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2081 Exit the nearest loop immediately; execution resumes after that loop.

2082 Set `metrics` by select the indexed/sliced entries of env.last_info.

2083 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2084 Continue the expression begun at line 2083 with `'learned_planning_opportunities': free_decisions,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2085 Continue the expression begun at line 2083 with `'planning_mode_terminations': terminations,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2086 Continue the expression begun at line 2083 with `'availability': metrics['flights'] / (config.aircraft * config.horizon), **metrics})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2087 Return `records` to the caller by evaluate records. No further statements in this call execute.

10.151. Module declarations and integration

Source lines 2090–2091.

```
2090 ALL_LEARNED_METHODS = ('flat', 'conditional', 'shared', 'options', 'learned_options',
2091                        *REQUEST_METHODS, *HRL_METHODS)
```

Line-by-line explanation.

2090 Set `ALL_LEARNED_METHODS` by construct the shown tuple in its stated order.

2091 Continue the assignment begun at line 2090 with `*REQUEST_METHODS, *HRL_METHODS)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

10.152. execute_option

Execute the original fleet-wide option call for several primitive days. Accepted repairs remain committed and other aircraft can fly, but no new learned fleet decision is made until this executor returns.

Source lines 2094–2112.

```
2094 def execute_option(env, option, obs, gamma=0.99, max_days=14, planned_action=None):
2095     """Plan once, then keep all active jobs committed while the fleet advances.
2097     Manager returns at first task completion, a new observed urgency alarm,
2098     episode end, or max_days. It never consumes true event diagnostics.
2099     Waiting without any accepted job lasts one day.
2100     Other operating aircraft continue their modeled flights during this option.
2101     Longer manager intervals change decision opportunities, so this is a timing
2102     experiment as well as an architecture experiment, not a matched PPO ablation.
2103     """
2104     option = int(option)
2105     if option not in range(4):
```

```
2106            raise ValueError('unknown option')
2107        if max_days < 1:
2108            raise ValueError('max_days must be positive')
2109        thresholds = {1: (0.62, 0.62), 2: (0.65, 0.77), 3: (0.78, 0.85)}
2110        action = planned_action if planned_action is not None else env.idle_action() if option == 0 else
            ↪  threshold_action(env, obs, *thresholds[option])
2111        before, _ = env.split_observation(obs)
2112        before_alarm = before[:, 24:36].min(1) <= env.config.mandatory_health
```

Line-by-line explanation.

2094 Define `execute_option(env, option, obs, gamma=0.99, max_days=14, planned_action=None)`. Execute the original fleet-wide option call for several primitive days. Accepted repairs remain committed and other aircraft can fly, but no new learned fleet decision is made until this executor returns.

2095 Documentation text: Plan once, then keep all active jobs committed while the fleet advances.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

2097 Documentation text: Manager returns at first task completion, a new observed urgency alarm,. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

2098 Documentation text: episode end, or max_days. It never consumes true event diagnostics.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

2099 Documentation text: Waiting without any accepted job lasts one day.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

2100 Documentation text: Other operating aircraft continue their modeled flights during this option.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

2101 Documentation text: Longer manager intervals change decision opportunities, so this is a timing. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

2102 Documentation text: experiment as well as an architecture experiment, not a matched PPO ablation.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

2103 Close the documentation string for `execute_option`. This delimiter ends the descriptive text and performs no simulator or learning update.

2104 Set `option` by convert the result to a Python integer.

2105 Test `option not in range(4)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2106 Raise `ValueError('unknown option')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2107 Test `max_days < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2108 Raise `ValueError('max_days must be positive')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2109 Set `thresholds` by construct the shown dict in its stated order.

2110 Set `action` by select planned_action when planned_action is not None, otherwise env.idle_action() if option == 0 else threshold_action(env, obs, *thresholds[option]). It holds the sampled or stored action whose probability is evaluated.

2111 Set `(before, _)` by call env.split_observation with the shown positional and keyword arguments.

2112 Set `before_alarm` by evaluate the comparison before[:, 24:36].min(1) <= env.config.mandatory_health.

Source lines 2113–2123.

```
2113    rewards = []
2114    while True:
2115        obs, reward, done, truncated, info = env.step(action)
2116        rewards.append(reward)
2117        rows, _ = env.split_observation(obs)
2118        new_alarm = np.any((rows[:, 24:36].min(1) <= env.config.mandatory_health) & ~before_alarm)
2119        if option == 0 or done or truncated or info['completed'] or new_alarm or (len(rewards) >=
            ↪ max_days) or (not env.jobs):
2120            break
2121        action = env.idle_action()
2122    discounted = sum((gamma ** i * r for i, r in enumerate(rewards)))
2123    return (obs, discounted, done, truncated, info, len(rewards), sum(rewards))
```

Line-by-line explanation.

2113 Set `rewards` by construct the shown list in its stated order. It holds the primitive reward sequence used for return accounting.

2114 Repeat the indented block while `True` remains true; this condition controls the simulation/collection loop here.

2115 Set `(obs, reward, done, truncated, info)` by execute this object's step with the shown arguments.

2116 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2117 Set `(rows, _)` by call env.split_observation with the shown positional and keyword arguments.

2118 Set `new_alarm` by call np.any with the shown positional and keyword arguments.

2119 Test `option == 0 or done or truncated or info['completed'] or new_alarm or (len(rewards) >= max_days) or (not env.jobs)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2120 Exit the nearest loop immediately; execution resumes after that loop.

2121 Set `action` by call env.idle_action with the shown positional and keyword arguments. It holds the sampled or stored action whose probability is evaluated.

2122 Set `discounted` by add the terms of the supplied iterable.

2123 Return `(obs, discounted, done, truncated, info, len(rewards), sum(rewards))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.153. Module declarations and integration

Source lines 2126–2134.

```
# ======================================================================
# fleet_maintenance/train.py
# ======================================================================
from dataclasses import asdict
from pathlib import Path
import argparse
import json
```

Line-by-line explanation.

2126 A visual section separator in the source. It has no executable effect.

2127 Source comment: fleet_maintenance/train.py This line documents the following code and does not execute.

2128 A visual section separator in the source. It has no executable effect.

2131 Import `from dataclasses import asdict`. Dataclasses provide structured configuration and repair records.

2132 Import `from pathlib import Path`. Path handles output/checkpoint paths.

2133 Import `import argparse`. Argument parsing configures command-line experiments.

2134 Import `import json`. JSON stores settings and diagnostic records.

10.154. make_model

Construct the policy class that matches the method registry, observation dimensions, fleet size and ownership. A method label selects a concrete implementation, not a cosmetic plotting name.

Source lines 2136–2151.

```
def make_model(env, method):
    if method in HRL_PPO_METHODS:
        return AircraftPlanningHierarchy(env, method)
    if method == 'maxq':
        return MAXQMaintenance(env)
    if method == 'option_critic':
        return FleetOptionCritic(env)
    if method == 'hierarchical_vdn':
        return HierarchicalVDN(env)
    if method in DCMAC_METHODS:
        return DCMACActorCritic(env, request_assignment(method), method.startswith('ddmac_'))
    if method in MAPPO_METHODS:
        return MultiAgentActorCritic(env, 'aircraft' if method == 'mappo_aircraft' else 'component')
    if method == 'learned_options':
        return LearnedOptionActorCritic(env)
    return OptionActorCritic(env) if method == 'options' else FleetActorCritic(env, method)
```

Line-by-line explanation.

2136 Define `make_model(env, method)`. Construct the policy class that matches the method registry, observation dimensions, fleet size and ownership. A method label selects a concrete implementation, not a cosmetic plotting name.

2137 Test `method in HRL_PPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2138 Return `AircraftPlanningHierarchy(env, method)` to the caller by call AircraftPlanningHierarchy with the shown positional and keyword arguments. No further statements in this call execute.

2139 Test `method == 'maxq'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2140 Return `MAXQMaintenance(env)` to the caller by call MAXQMaintenance with the shown positional and keyword arguments. No further statements in this call execute.

2141 Test `method == 'option_critic'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2142 Return `FleetOptionCritic(env)` to the caller by call FleetOptionCritic with the shown positional and keyword arguments. No further statements in this call execute.

2143 Test `method == 'hierarchical_vdn'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2144 Return `HierarchicalVDN(env)` to the caller by call HierarchicalVDN with the shown positional and keyword arguments. No further statements in this call execute.

2145 Test `method in DCMAC_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2146 Return `DCMACActorCritic(env, request_assignment(method), method.startswith('ddmac_'))` to the caller by call DCMACActorCritic with the shown positional and keyword arguments. No further statements in this call execute.

2147 Test `method in MAPPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2148 Return `MultiAgentActorCritic(env, 'aircraft' if method == 'mappo_aircraft' else 'component')` to the caller by call MultiAgentActorCritic with the shown positional and keyword arguments. No further statements in this call execute.

2149 Test `method == 'learned_options'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2150 Return `LearnedOptionActorCritic(env)` to the caller by call LearnedOptionActorCritic with the shown positional and keyword arguments. No further statements in this call execute.

2151 Return `OptionActorCritic(env) if method == 'options' else FleetActorCritic(env, method)` to the caller by select OptionActorCritic(env) when method == 'options', otherwise FleetActorCritic(env, method). No further statements in this call execute.

10.155. load_checkpoint

Read saved configuration, method and model state; construct the matching architecture and restore weights or MAXQ tables.

Source lines 2153–2162.

```
def load_checkpoint(path):
    """Reload one of this package's own checkpoints for independent evaluation."""
    checkpoint = torch.load(path, map_location='cpu', weights_only=True)
    if checkpoint.get('observation_version') != OBSERVATION_VERSION:
        raise ValueError('Checkpoint uses an incompatible observation/action layout. Retrain with
        ↪  individual crew masks and resource factors (version 3).')
    config = Config(**checkpoint['config'])
    model = make_model(FleetMaintenanceEnv(config), checkpoint['method'])
    model.load_state_dict(checkpoint['state_dict'])
    model.eval()
    return (config, checkpoint['method'], model)
```

Line-by-line explanation.

2153 Define `load_checkpoint(path)`. Read saved configuration, method and model state; construct the matching architecture and restore weights or MAXQ tables.

2154 Documentation text: Reload one of this package's own checkpoints for independent evaluation.. This states the contract or qualification for `load_checkpoint`; it does not update the simulator or model.

2155 Set `checkpoint` by call torch.load with the shown positional and keyword arguments.

2156 Test `checkpoint.get('observation_version') != OBSERVATION_VERSION`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2157 Raise `ValueError('Checkpoint uses an incompatible observation/action layout. Retrain with individual crew masks and resource factors (version 3).')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2158 Set `config` by call Config with the shown positional and keyword arguments.

2159 Set `model` by call make_model with the shown positional and keyword arguments. It holds the chosen controller architecture.

2160 Restore weights and registered extra state. The exact receiver and arguments are shown on this source line.

2161 Call model.eval with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2162 Return `(config, checkpoint['method'], model)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.156. smdp_gae

Compute generalized advantages with discount and trace factors raised to each transition duration for the original fleet-level option trainer.

Source lines 2164–2177.

```
def smdp_gae(rewards, values, next_values, dones, durations, gamma=0.99, lam=0.95):
    """Backward GAE with gamma**duration and lambda**duration per decision.
    rewards already contain the discounted primitive rewards inside each option.
    A one-day decision recovers ordinary GAE. Episode boundaries block bootstrap.
    """
```

```
    advantage = np.zeros(len(rewards), dtype=np.float32)
    carry = 0.0
    for t in range(len(rewards) - 1, -1, -1):
        live = 1.0 - float(dones[t])
        delta = rewards[t] + gamma ** durations[t] * next_values[t] * live - values[t]
        carry = delta + (gamma * lam) ** durations[t] * live * carry
        advantage[t] = carry
    return (advantage, advantage + np.asarray(values))
```

Line-by-line explanation.

2164 Define `smdp_gae(rewards, values, next_values, dones, durations, gamma=0.99, lam=0.95)`. Compute generalized advantages with discount and trace factors raised to each transition duration for the original fleet-level option trainer.

2165 Documentation text: Backward GAE with gamma**duration and lambda**duration per decision.. This states the contract or qualification for `smdp_gae`; it does not update the simulator or model.

2167 Documentation text: rewards already contain the discounted primitive rewards inside each option.. This states the contract or qualification for `smdp_gae`; it does not update the simulator or model.

2168 Documentation text: A one-day decision recovers ordinary GAE. Episode boundaries block bootstrap.. This states the contract or qualification for `smdp_gae`; it does not update the simulator or model.

2169 Close the documentation string for `smdp_gae`. This delimiter ends the descriptive text and performs no simulator or learning update.

2170 Set `advantage` by call np.zeros with the shown positional and keyword arguments.

2171 Set `carry` by use the literal 0.0.

2172 Iterate `t` over `range(len(rewards) - 1, -1, -1)`. The following indented statements run once for each selected element.

2173 Set `live` by subtract 1.0 and float(dones[t]).

2174 Set `delta` by subtract rewards[t] + gamma ** durations[t] * next_values[t] * live and values[t].

2175 Set `carry` by add delta and (gamma * lam) ** durations[t] * live * carry.

2176 Set `advantage[t]` by evaluate carry.

2177 Return `(advantage, advantage + np.asarray(values))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.157. train

Dispatch new method families to their dedicated trainers. For the original five methods, collect transitions, account for duration, and optimize the fleet-joint PPO objective.

Source lines 2179–2196.

```
def train(config, method='conditional', steps=4096, seed=0, rollout_size=128):
    """Interaction budget counts actual simulated days, not manager decisions."""
    if method in HRL_PPO_METHODS:
```

```
2182            return train_aircraft_hrl(config, method, steps, seed, rollout_size)
2183        if method == 'maxq':
2184            return train_maxq(config, steps, seed)
2185        if method == 'option_critic':
2186            return train_option_critic(config, steps, seed)
2187        if method == 'hierarchical_vdn':
2188            return train_hierarchical_vdn(config, steps, seed)
2189        if method in DCMAC_METHODS:
2190            return train_dcmac(config, method, steps, seed, rollout_size)
2191        if method in MAPPO_METHODS:
2192            return train_mappo(config, method, steps, seed, rollout_size)
2193        if steps < 1 or rollout_size < 2:
2194            raise ValueError('positive budget and rollout>=2 required')
2195        torch.set_num_threads(1)
2196        torch.manual_seed(seed)
```

Line-by-line explanation.

2179 Define `train(config, method='conditional', steps=4096, seed=0, rollout_size=128)`. Dispatch new method families to their dedicated trainers. For the original five methods, collect transitions, account for duration, and optimize the fleet-joint PPO objective.

2180 Documentation text: Interaction budget counts actual simulated days, not manager decisions.. This states the contract or qualification for `train`; it does not update the simulator or model.

2181 Test `method in HRL_PPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2182 Return `train_aircraft_hrl(config, method, steps, seed, rollout_size)` to the caller by call train_aircraft_hrl with the shown positional and keyword arguments. No further statements in this call execute.

2183 Test `method == 'maxq'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2184 Return `train_maxq(config, steps, seed)` to the caller by call train_maxq with the shown positional and keyword arguments. No further statements in this call execute.

2185 Test `method == 'option_critic'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2186 Return `train_option_critic(config, steps, seed)` to the caller by call train_option_critic with the shown positional and keyword arguments. No further statements in this call execute.

2187 Test `method == 'hierarchical_vdn'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2188 Return `train_hierarchical_vdn(config, steps, seed)` to the caller by call train_hierarchical_vdn with the shown positional and keyword arguments. No further statements in this call execute.

2189 Test `method in DCMAC_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2190 Return `train_dcmac(config, method, steps, seed, rollout_size)` to the caller by call train_dcmac with the shown positional and keyword arguments. No further statements in this call execute.

2191 Test `method in MAPPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2192 Return `train_mappo(config, method, steps, seed, rollout_size)` to the caller by call train_mappo with the shown positional and keyword arguments. No further statements in this call execute.

2193 Test `steps < 1 or rollout_size < 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2194 Raise `ValueError('positive budget and rollout>=2 required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2195 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

2196 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

Source lines 2197–2214.

```
    env = FleetMaintenanceEnv(config)
    model = make_model(env, method)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.0003)
    rng = np.random.default_rng(seed)
    episode = 0
    obs, _ = env.reset(seed=seed)
    simulated = 0
    logs = []
    while simulated < steps:
        data = {k: [] for k in ('obs', 'action', 'logp', 'value', 'next_value', 'reward', 'done',
        ↪  'duration')}
        while len(data['obs']) < rollout_size and simulated < steps:
            with torch.no_grad():
                action, logp, _, value = model.act(torch.tensor(obs))
            chosen = action[0].numpy()
            if method in ('options', 'learned_options'):
                macro = int(chosen) if method == 'options' else int(chosen[0])
                plan = None if method == 'options' else chosen[1:].reshape(config.aircraft, 8)
                next_obs, reward, done, truncated, info, duration, _ = execute_option(env, macro, obs,
                ↪  max_days=min(14, steps - simulated), planned_action=plan)
```

Line-by-line explanation.

2197 Set `env` by call FleetMaintenanceEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

2198 Set `model` by call make_model with the shown positional and keyword arguments. It holds the chosen controller architecture.

2199 Set `optimizer` by create an Adam optimizer for the specified parameter collection.

2200 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

2201 Set `episode` by use the literal 0.

2202 Set `(obs, _)` by initialize a new episode using the stated seed/options.

2203 Set `simulated` by use the literal 0.

2204 Set `logs` by construct the shown list in its stated order. It holds episode/update diagnostics for later inspection.

2205 Repeat the indented block while `simulated < steps` remains true; this condition controls the simulation/collection loop here.

2206 Set `data` by construct the collection by iterating the shown comprehension and applying its filters.

2207 Repeat the indented block while `len(data['obs']) < rollout_size and simulated < steps` remains true; this condition controls the simulation/collection loop here.

2208 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2209 Set `(action, logp, _, value)` by call model.act with the shown positional and keyword arguments.

2210 Set `chosen` by expose this CPU tensor as a NumPy array.

2211 Test `method in ('options', 'learned_options')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2212 Set `macro` by select int(chosen) when method == 'options', otherwise int(chosen[0]).

2213 Set `plan` by select None when method == 'options', otherwise chosen[1:].reshape(config.aircraft, 8).

2214 Set `(next_obs, reward, done, truncated, info, duration, _)` by call execute_option with the shown positional and keyword arguments.

Source lines 2215–2232.

```
            else:
                next_obs, reward, done, truncated, info = env.step(chosen)
                duration = 1
            terminal = done or truncated
            with torch.no_grad():
                nv = float(model.value(torch.tensor(next_obs).unsqueeze(0))[0])
            for key, item in [('obs', obs.copy()), ('action', np.asarray(chosen).copy()), ('logp',
            ↪  float(logp[0])), ('value', float(value[0])), ('next_value', nv), ('reward', reward),
            ↪  ('done', terminal), ('duration', duration)]:
                data[key].append(item)
            simulated += duration
            obs = next_obs
            if terminal:
                logs.append({'episode': episode, 'simulated_days': simulated, **info['metrics']})
                episode += 1
                obs, _ = env.reset(seed=seed + episode)
        advantages, targets = smdp_gae(data['reward'], data['value'], data['next_value'],
        ↪  data['done'], data['duration'])
        tensors = {k: torch.tensor(np.asarray(v)) for k, v in data.items() if k in ('obs', 'action',
        ↪  'logp')}
        advantages = torch.tensor(advantages)
        if len(advantages) > 1:
```

Line-by-line explanation.

2215 Execute this alternative only when the corresponding preceding condition did not select its branch.

2216 Set `(next_obs, reward, done, truncated, info)` by execute this object's step with the shown arguments.

2217 Set `duration` by use the literal 1. It holds the number of primitive days in this decision.

Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

Set `nv` by convert the result to a Python floating-point number.

Iterate `(key, item)` over `[('obs', obs.copy()), ('action', np.asarray(chosen).copy()), ('logp', float(logp[0])), ('value', float(value[0])), ('next_value', nv), ('reward', reward), ('done', terminal), ('duration', duration)]`. The following indented statements run once for each selected element.

Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

Add `duration` to `simulated`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Set `obs` by evaluate next_obs. It holds the observed belief vector, rather than hidden physical health.

Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Set `(obs, _)` by initialize a new episode using the stated seed/options.

Set `(advantages, targets)` by call smdp_gae with the shown positional and keyword arguments.

Set `tensors` by construct the collection by iterating the shown comprehension and applying its filters.

Set `advantages` by call torch.tensor with the shown positional and keyword arguments. It holds the policy credit signal with its retained aircraft/agent axis.

Test `len(advantages) > 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 2233–2249.

```
            advantages = (advantages - advantages.mean()) / (advantages.std(unbiased=False) + 1e-08)
        targets = torch.tensor(targets, dtype=torch.float32)
        for _ in range(4):
            ids = rng.permutation(len(targets))
            for start in range(0, len(ids), 64):
                idx = ids[start:start + 64]
                _, newlog, entropy, values = model.act(tensors['obs'][idx], tensors['action'][idx])
                ratio = torch.exp(newlog - tensors['logp'][idx])
                surrogate = torch.minimum(ratio * advantages[idx], ratio.clamp(0.8, 1.2) *
                ↪  advantages[idx])
                loss = -surrogate.mean() + 0.5 * ((values - targets[idx]) ** 2).mean() - 0.001 *
                ↪  entropy.mean()
                if not torch.isfinite(loss):
                    raise RuntimeError('nonfinite PPO loss')
                optimizer.zero_grad()
                loss.backward()
```

```
                torch.nn.utils.clip_grad_norm_(model.parameters(), 0.5)
                optimizer.step()
    return (model, logs, simulated)
```

Line-by-line explanation.

2233 Set `advantages` by divide advantages - advantages.mean() and advantages.std(unbiased=False) + 1e-08. It holds the policy credit signal with its retained aircraft/agent axis.

2234 Set `targets` by call torch.tensor with the shown positional and keyword arguments.

2235 Iterate `_` over `range(4)`. The following indented statements run once for each selected element.

2236 Set `ids` by call rng.permutation with the shown positional and keyword arguments.

2237 Iterate `start` over `range(0, len(ids), 64)`. The following indented statements run once for each selected element.

2238 Set `idx` by select the indexed/sliced entries of ids.

2239 Set `(_, newlog, entropy, values)` by call model.act with the shown positional and keyword arguments.

2240 Set `ratio` by exponentiate log values, including log-probability ratios. It holds the current-to-behavior probability ratio.

2241 Set `surrogate` by call torch.minimum with the shown positional and keyword arguments.

2242 Set `loss` by subtract -surrogate.mean() + 0.5 * ((values - targets[idx]) ** 2).mean() and 0.001 * entropy.mean().

2243 Test `not torch.isfinite(loss)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2244 Raise `RuntimeError('nonfinite PPO loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2245 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2246 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

2247 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

2248 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2249 Return `(model, logs, simulated)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

10.158. evaluate

Dispatch hierarchy evaluation or roll out the original/request controllers on held-out seeds without gradient updates. Aggregate metrics retain the common environment definitions.

Source lines 2251–2268.

```
def evaluate(config, method, model=None, seeds=range(10000, 10010)):
    """Evaluate fixed weights on held-out seeds; proposal methods use their common broker."""
    if method in HRL_METHODS:
        return evaluate_hrl(config, method, model, seeds)
    if method != 'threshold' and model is None:
        raise ValueError('evaluation requires a trained model')
    records = []
    for seed in seeds:
        if method in REQUEST_METHODS:
            env = MultiAgentFleetEnv(config, request_assignment(method))
            _, info = env.reset(seed=int(seed))
            obs = info['central_observation']
        else:
            env = FleetMaintenanceEnv(config)
            obs, _ = env.reset(seed=int(seed))
        decisions = 0
        while True:
            if method == 'threshold':
```

Line-by-line explanation.

2251 Define `evaluate(config, method, model=None, seeds=range(10000, 10010))`. Dispatch hierarchy evaluation or roll out the original/request controllers on held-out seeds without gradient updates. Aggregate metrics retain the common environment definitions.

2252 Documentation text: Evaluate fixed weights on held-out seeds; proposal methods use their common broker.. This states the contract or qualification for `evaluate`; it does not update the simulator or model.

2253 Test `method in HRL_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2254 Return `evaluate_hrl(config, method, model, seeds)` to the caller by call evaluate_hrl with the shown positional and keyword arguments. No further statements in this call execute.

2255 Test `method != 'threshold' and model is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2256 Raise `ValueError('evaluation requires a trained model')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2257 Set `records` by construct the shown list in its stated order.

2258 Iterate `seed` over `seeds`. The following indented statements run once for each selected element.

2259 Test `method in REQUEST_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2260 Set `env` by call MultiAgentFleetEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

2261 Set `(_, info)` by initialize a new episode using the stated seed/options.

2262 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health.

2263 Execute this alternative only when the corresponding preceding condition did not select its branch.

2264 Set `env` by call FleetMaintenanceEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

2265 Set `(obs, _)` by initialize a new episode using the stated seed/options.

2266 Set `decisions` by use the literal 0.

2267 Repeat the indented block while `True` remains true; this condition controls the simulation/collection loop here.

2268 Test `method == 'threshold'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 2269–2286.

```
            obs, _, done, truncated, info = env.step(threshold_action(env, obs))
        else:
            with torch.no_grad():
                if method in REQUEST_METHODS:
                    views = np.stack(list(env.observations().values())) if method in MAPPO_METHODS
                    ↪  else obs
                    action, _, _ = model.policy(torch.tensor(views).unsqueeze(0) if method in
                    ↪  MAPPO_METHODS else torch.tensor(views), deterministic=True)
                else:
                    action, _, _, _ = model.act(torch.tensor(obs), deterministic=True)
            if method in REQUEST_METHODS:
                _, _, done, truncated, info = env.step(model.action_dict(action[0]))
                obs = info['central_observation']
            elif method in ('options', 'learned_options'):
                chosen = action[0].numpy()
                macro = int(chosen) if method == 'options' else int(chosen[0])
                plan = None if method == 'options' else chosen[1:].reshape(config.aircraft, 8)
                obs, _, done, truncated, info, _, _ = execute_option(env, macro, obs,
                ↪  planned_action=plan)
            else:
                obs, _, done, truncated, info = env.step(action[0].numpy())
```

Line-by-line explanation.

2269 Set `(obs, _, done, truncated, info)` by execute this object's step with the shown arguments.

2270 Execute this alternative only when the corresponding preceding condition did not select its branch.

2271 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2272 Test `method in REQUEST_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2273 Set `views` by select np.stack(list(env.observations().values())) when method in MAPPO_METHODS, otherwise obs.

2274 Set `(action, _, _)` by call model.policy with the shown positional and keyword arguments.

2275 Execute this alternative only when the corresponding preceding condition did not select its branch.

2276 Set `(action, _, _, _)` by call model.act with the shown positional and keyword arguments.

2277 Test `method in REQUEST_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2278 Set `(_, _, done, truncated, info)` by execute this object's step with the shown arguments.

2279 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health.

2280 Test `method in ('options', 'learned_options')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2281 Set `chosen` by expose this CPU tensor as a NumPy array.

2282 Set `macro` by select int(chosen) when method == 'options', otherwise int(chosen[0]).

2283 Set `plan` by select None when method == 'options', otherwise chosen[1:].reshape(config.aircraft, 8).

2284 Set `(obs, _, done, truncated, info, _, _)` by call execute_option with the shown positional and keyword arguments.

2285 Execute this alternative only when the corresponding preceding condition did not select its branch.

2286 Set `(obs, _, done, truncated, info)` by execute this object's step with the shown arguments.

Source lines 2287–2291.

```
2287             decisions += 1
2288             if done or truncated:
2289                 break
2290         records.append({'seed': int(seed), 'decisions': decisions, 'availability':
             ↪  info['metrics']['flights'] / (config.aircraft * config.horizon), **info['metrics']})
2291     return records
```

Line-by-line explanation.

2287 Add `1` to `decisions`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2288 Test `done or truncated`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2289 Exit the nearest loop immediately; execution resumes after that loop.

2290 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2291 Return `records` to the caller by evaluate records. No further statements in this call execute.

10.159. main

Parse command-line configuration and run requested methods, seeds, checkpoint saving and evaluation. The all-method path explicitly records the one-aircraft MAXQ restriction.

Source lines 2293–2310.

```
2293 def main():
2294     parser = argparse.ArgumentParser()
2295     parser.add_argument('--method', choices=[*ALL_LEARNED_METHODS, 'all'], default='all')
2296     parser.add_argument('--steps', type=int, default=4096)
2297     parser.add_argument('--aircraft', type=int, default=7)
2298     parser.add_argument('--crews', type=int, default=3)
2299     parser.add_argument('--workshops', type=int, default=2)
2300     parser.add_argument('--crew-time-factors', type=float, nargs='+', default=())
2301     parser.add_argument('--workshop-time-factors', type=float, nargs='+', default=())
2302     parser.add_argument('--horizon', type=int, default=90)
2303     parser.add_argument('--seed', type=int, default=0)
2304     parser.add_argument('--eval-episodes', type=int, default=10)
2305     parser.add_argument('--output', default='runs/smoke')
2306     args = parser.parse_args()
2307     config = Config(aircraft=args.aircraft, horizon=args.horizon, crews=args.crews,
        ↪  workshops=args.workshops, crew_time_factors=tuple(args.crew_time_factors),
        ↪  workshop_time_factors=tuple(args.workshop_time_factors))
2308     destination = Path(args.output)
2309     destination.mkdir(parents=True, exist_ok=True)
2310     result = {'config': asdict(config), 'training_seed': args.seed, 'requested_steps': args.steps,
        ↪  'observation_version': OBSERVATION_VERSION, 'status': 'preliminary teaching experiment; not a
        ↪  paper reproduction or convergence claim', 'comparison_note': 'new HRL methods share a
        ↪  restricted 22-action catalog and broker; hrl_flat matches planning_hierarchy authority;
        ↪  concurrent_options has a fixed worker; target/option/VDN clocks and objectives differ; MAXQ is
        ↪  single-aircraft; legacy and MAPPO/DCMAC use broader action spaces', 'evaluation':
        ↪  {'threshold': evaluate(config, 'threshold', seeds=range(10000, 10000 + args.eval_episodes))}}
```

Line-by-line explanation.

2293 Define `main()`. Parse command-line configuration and run requested methods, seeds, checkpoint saving and evaluation. The all-method path explicitly records the one-aircraft MAXQ restriction.

2294 Set `parser` by call argparse.ArgumentParser with the shown positional and keyword arguments.

2295 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2296 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2297 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2298 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2299 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2300 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2301 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2302 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2303 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2304 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2305 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2306 Set `args` by call parser.parse_args with the shown positional and keyword arguments.

2307 Set `config` by call Config with the shown positional and keyword arguments.

2308 Set `destination` by call Path with the shown positional and keyword arguments.

2309 Call destination.mkdir with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2310 Set `result` by construct the shown dict in its stated order.

Source lines 2311–2327.

```
    methods = list(ALL_LEARNED_METHODS) if args.method == 'all' else [args.method]
    if args.method == 'all' and config.aircraft != 1:
        methods.remove('maxq')
        result['skipped_methods'] = {'maxq': 'requires aircraft=1'}
    for method in methods:
        model, logs, days = train(config, method, args.steps, args.seed)
        torch.save({'state_dict': model.state_dict(), 'config': asdict(config), 'method': method,
        ↪  'training_seed': args.seed, 'simulated_days': days, 'observation_version':
        ↪  OBSERVATION_VERSION}, destination / f'{method}.pt')
        result['evaluation'][method] = evaluate(config, method, model, seeds=range(10000, 10000 +
        ↪  args.eval_episodes))
        result[method + '_training'] = logs
        print(method, 'days', days, 'mean test return', np.mean([r['return'] for r in
        ↪  result['evaluation'][method]]), flush=True)
    (destination / 'results.json').write_text(json.dumps(result, indent=2), encoding='utf-8')
    summary = ['# Preliminary comparison', '', result['status'], '', result['comparison_note'], '', '|
    ↪  Method | Return | Availability | Failed flights | Rejected plans | Decisions |',
    ↪  '|---|---:|---:|---:|---:|---:|']
    for method, records in result['evaluation'].items():
        mean = lambda k: np.mean([r[k] for r in records])
        summary.append(f"| {method} | {mean('return'):.2f} | {mean('availability'):.3f} |
        ↪  {mean('flight_failures'):.1f} | {mean('rejections'):.1f} | {mean('decisions'):.1f} |")
    (destination / 'summary.md').write_text('\n'.join(summary) + '\n', encoding='utf-8')
    print('Saved', destination.resolve())
```

Line-by-line explanation.

2311 Set `methods` by select list(ALL_LEARNED_METHODS) when args.method == 'all', otherwise [args.method].

2312 Test `args.method == 'all' and config.aircraft != 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2313 Call methods.remove with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2314 Set `result['skipped_methods']` by construct the shown dict in its stated order.

2315 Iterate `method` over `methods`. The following indented statements run once for each selected element.

2316 Set `(model, logs, days)` by call train with the shown positional and keyword arguments.

2317 Call torch.save with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2318 Set `result['evaluation'][method]` by call evaluate with the shown positional and keyword arguments.

2319 Set `result[method + '_training']` by evaluate logs.

2320 Call print with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2321 Call (destination / 'results.json').write_text with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2322 Set `summary` by construct the shown list in its stated order.

2323 Iterate `(method, records)` over `result['evaluation'].items()`. The following indented statements run once for each selected element.

2324 Set `mean` by evaluate lambda k: np.mean([r[k] for r in records]).

2325 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2326 Call (destination / 'summary.md').write_text with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2327 Call print with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

10.160. Module declarations and integration

Source lines 2330–2331.

```
OPERATE_UNTIL_LOW = 0
REPLACE_NOW = 1
```

Line-by-line explanation.

2330 Set `OPERATE_UNTIL_LOW` by use the literal 0.

2331 Set `REPLACE_NOW` by use the literal 1.

11. Closing interpretation

The tutorial contains multi-agent learning interfaces and the scoped hierarchy implementations described above. Sharing weights, factorizing outputs, using global information, learning option duration, and coordinating resources are distinct design choices. The current code is most useful when these choices are inspected separately and compared under matched authority and information. Its software coverage is substantially broader than the earlier report, while physical calibration and controlled research conclusions remain future work.

References

[1] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, second edition. MIT Press, 2018. http://incompleteideas.net/book/the-book-2nd.html.

[2] R. S. Sutton, D. Precup and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112:181–211, 1999. https://doi.org/10.1016/S0004-3702(99)00052-1.

[3] J. Schulman et al. Proximal Policy Optimization Algorithms. 2017. https://arxiv.org/abs/1707.06347.

[4] Stable-Baselines3 contributors. Custom Policy Network documentation. Accessed 26 October 2026. https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html.

[5] C. P. Andriotis and K. G. Papakonstantinou. Managing engineering systems with large state and action spaces through deep reinforcement learning. *Reliability Engineering & System Safety*, 191:106483, 2019. https://doi.org/10.1016/j.ress.2019.04.036.

[6] C. P. Andriotis and K. G. Papakonstantinou. Deep reinforcement learning driven inspection and maintenance planning under incomplete information and constraints. *Reliability Engineering & System Safety*, 212:107551, 2021. https://doi.org/10.1016/j.ress.2021.107551.

[7] C. Yu et al. The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games. *Advances in Neural Information Processing Systems*, 35, 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract.html.

[8] T. G. Dietterich. Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition. *Journal of Artificial Intelligence Research*, 13:227–303, 2000. https://www.cs.cmu.edu/afs/cs/project/jair/pub/volume13/dietterich00a.pdf.

[9] P.-L. Bacon, J. Harb and D. Precup. The Option-Critic Architecture. *Proceedings of AAAI*, 31, 2017. https://doi.org/10.1609/aaai.v31i1.10916.

[10] Z. Xu, Y. Bai, B. Zhang, D. Li and G. Fan. HAVEN: Hierarchical Cooperative Multi-Agent Reinforcement Learning with Dual Coordination Mechanism. *Proceedings of AAAI*, 37(10):11735–11743, 2023. https://doi.org/10.1609/aaai.v37i10.26386.

[11] P. Sunehag et al. Value-Decomposition Networks For Cooperative Multi-Agent Learning. *Proceedings of AAMAS*, 2018. https://arxiv.org/abs/1706.05296.